

Documentation — Toutes les fonctions du projet NewApp

Ce document liste **chaque fonction** du code source TypeScript/React du projet, regroupée par catégorie, avec son **emplacement précis** (fichier:ligne) et son **code complet** reproduit sans modification. Complément du fichier **DOCUMENTATION-NEWAPP.md** (qui contient l'explication fonctionnelle).

Table des matières

- [Entrée et configuration](#)
- [Contextes \(états globaux React\)](#)
- [Hooks personnalisés](#)
- [Utilitaires](#)
- [Types](#)
- [Services — Authentification](#)
- [Services — Produits & Catalogue](#)
- [Services — Stock](#)
- [Services — Clients & Commandes](#)
- [Services — Import & Validation](#)
- [Composants — Layouts & Routing](#)
- [Composants — Authentification](#)
- [Composants — Produits \(BO\)](#)
- [Composants — Imports](#)
- [Composants — Commandes & Stock \(BO\)](#)
- [Composants — Boutique \(FrontOffice\)](#)
- [Composants — Tableau de bord](#)

Entrée et configuration

Fichier : `src/main.tsx`

Lignes : 12 • Fonctions détectées : 0

Aucune fonction détectée — contenu complet du fichier :

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/design-system.css'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Fichier : `src/App.tsx`

Lignes : 226 • Fonctions détectées : 3

`resolvePageTitle()` — ligne 29

Description (commentaire d'origine) : Résout le titre de page selon la route courante

```
function resolvePageTitle(pathname: string): string {
  if (pathname === '/products') return 'Produits';
  if (pathname === '/products/add') return 'Ajouter un produit';
  if (pathname === '/products/import') return 'Import CSV Produits';
  if (pathname.startsWith('/products/')) return 'Modifier le produit';
  if (pathname === '/import') return 'Import CSV Catalogue';
  if (pathname === '/import/fichiers') return 'Import Fichiers';
  if (pathname === '/orders') return 'Commandes';
  if (pathname === '/reset') return 'Réinitialisation';
  if (pathname === '/audit/import') return 'Audit import';
  if (pathname === '/dashboard') return 'Tableau de bord';
  return '';
}
```

`LayoutWrapper()` — ligne 44

Description (commentaire d'origine) : Layout wrapper qui lit l'URL pour passer le bon titre

```
function LayoutWrapper({ children }: { children: React.ReactNode }) {
  const location = useLocation();
  return (
    <AppLayout pageTitle={resolvePageTitle(location.pathname)}>
      {children}
    </AppLayout>
  );
}
```

`App()` — ligne 52

Fonction : `app` — voir le code ci-dessous.

```

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <CustomerProvider>
          <CartProvider>
            <Routes>
              { /* ■■ Page de selection utilisateur ■■ */ }
              <Route path="/" element={ <UserSelectPage /> } />

              { /* ■■ Route publique login ■■ */ }
              <Route path="/login" element={ <Login /> } />

              { /* ■■ FrontOffice (public, sans sidebar) ■■ */ }
              <Route
                path="/shop"
                element={
                  <ShopLayout>
                    <ShopHome />
                  </ShopLayout>
                }
              />
              <Route
                path="/shop/:id"
                element={
                  <ShopLayout>
                    <ProductDetail />
                  </ShopLayout>
                }
              />

              { /* ■■ BackOffice (protégé, avec sidebar) ■■ */ }
              <Route
                path="/products"
                element={
                  <ProtectedRoute>
                    <LayoutWrapper>
                      <ProductList />
                    </LayoutWrapper>
                  </ProtectedRoute>
                }
              />
              <Route
                path="/products/add"
                element={
                  <ProtectedRoute>
                    <LayoutWrapper>
                      <ProductCreate />
                    </LayoutWrapper>
                  </ProtectedRoute>
                }
              />
              <Route
                path="/products/import"
                element={
                  <ProtectedRoute>
                    <LayoutWrapper>
                      <ProductImport />
                    </LayoutWrapper>
                  </ProtectedRoute>
                }
              />
              <Route
                path="/products/:id"
                element={
                  <ProtectedRoute>
                    <LayoutWrapper>
                      <ProductCreate />
                    </LayoutWrapper>
                  </ProtectedRoute>
                }
              />
            </Routes>
          </CartProvider>
        </CustomerProvider>
      </AuthProvider>
    </BrowserRouter>
  )
}

```

```

/>
<Route
  path="/import"
  element={
    <ProtectedRoute>
      <LayoutWrapper>
        <CatalogImport />
      </LayoutWrapper>
    </ProtectedRoute>
  }
/>
<Route
  path="/import/fichiers"
  element={
    <ProtectedRoute>
      <LayoutWrapper>
        <FichiersImport />
      </LayoutWrapper>
    </ProtectedRoute>
  }
/>
<Route
  path="/orders"
  element={
    <ProtectedRoute>
      <LayoutWrapper>
        <OrderList />
      </LayoutWrapper>
    </ProtectedRoute>
  }
/>
<Route
  path="/stock"
  element={
    <ProtectedRoute>
      <LayoutWrapper>
        <StockUpdate />
      </LayoutWrapper>
    </ProtectedRoute>
  }
/>
<Route
  path="/reset"
  element={
    <ProtectedRoute>
      <LayoutWrapper>
        <DataReset />
      </LayoutWrapper>
    </ProtectedRoute>
  }
/>
<Route
  path="/audit/import"
  element={
    <ProtectedRoute>
      <LayoutWrapper>
        <ImportAudit />
      </LayoutWrapper>
    </ProtectedRoute>
  }
/>

{/* ■■ FrontOffice – panier & tunnel d'achat ■■ */}
<Route
  path="/shop/cart"
  element={<ShopLayout><CartPage /></ShopLayout>}
/>
<Route
  path="/shop/auth"
  element={<CustomerAuthPage />}
/>

```

```

    <Route
      path="/shop/checkout"
      element={<ShopLayout><CheckoutPage /></ShopLayout>}
    />
    <Route
      path="/shop/confirmation/:id"
      element={<ShopLayout><OrderConfirmation /></ShopLayout>}
    />
    <Route
      path="/dashboard"
      element={
        <ProtectedRoute>
          <LayoutWrapper>
            <DashboardPage />
          </LayoutWrapper>
        </ProtectedRoute>
      }
    />
    <Route
      path="/shop/my-orders"
      element={<ShopLayout><MyOrders /></ShopLayout>}
    />
  </Routes>
</CartProvider>
</CustomerProvider>
</AuthProvider>
</BrowserRouter>
);
}

```

Contextes (états globaux React)

Fichier : `src/contexts/AuthContext.tsx`

Lignes : 58 • Fonctions détectées : 4

`AuthProvider()` — ligne 23

Description (commentaire d'origine) : ===== 3.
PROVIDER =====

```
export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [user, setUser] = useState<AuthUser | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  // Restaure la session depuis localStorage au montage
  useEffect(() => {
    setUser(authService.getCurrentUser());
    setIsLoading(false);
  }, []);

  const login = async (email: string, password: string): Promise<void> => {
    const loggedUser = await authService.login({ email, password });
    setUser(loggedUser);
  };

  const logout = async (): Promise<void> => {
    await authService.logout();
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, isLoading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

`login()` — ligne 32

Fonction : `login` — voir le code ci-dessous.

```
const login = async (email: string, password: string): Promise<void> => {
  const loggedUser = await authService.login({ email, password });
  setUser(loggedUser);
}
```

`logout()` — ligne 37

Fonction : `logout` — voir le code ci-dessous.

```
const logout = async (): Promise<void> => {
  await authService.logout();
  setUser(null);
}
```

`useAuth()` — ligne 53

Description (commentaire d'origine) : ===== 4. HOOK
D'ACCÈS AU CONTEXTE =====

```
export const useAuth = (): AuthContextType => {  
  const ctx = useContext(AuthContext);  
  if (!ctx) throw new Error('useAuth doit être utilisé à l'intérieur de AuthProvider');  
  return ctx;  
}
```

Fichier :

src/contexts/CustomerContext.tsx

Lignes : 49 • Fonctions détectées : 2

CustomerProvider() — ligne 13

Fonction : **customer provider** — voir le code ci-dessous.

```
export const CustomerProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {  
  const [customer, setCustomerState] = useState<Customer | null>(() => {  
    try {  
      const raw = sessionStorage.getItem(STORAGE_KEY);  
      return raw ? (JSON.parse(raw) as Customer) : null;  
    } catch { return null; }  
  });  
  
  useEffect(() => {  
    if (customer) {  
      sessionStorage.setItem(STORAGE_KEY, JSON.stringify(customer));  
      // Also store the secure_key separately for quick access by cart sync  
      if (customer.secureKey) {  
        sessionStorage.setItem('customerSecureKey', customer.secureKey);  
      }  
    } else {  
      sessionStorage.removeItem(STORAGE_KEY);  
      sessionStorage.removeItem('customerSecureKey');  
    }  
  }, [customer]);  
  
  const logout = () => setCustomerState(null);  
  
  return (  
    <CustomerContext.Provider value={{ customer, setCustomer: setCustomerState, logout }}>  
      {children}  
    </CustomerContext.Provider>  
  );  
}
```

useCustomer() — ligne 43

Fonction : **use customer** — voir le code ci-dessous.

```
export function useCustomer(): CustomerContextType {  
  const ctx = useContext(CustomerContext);  
  if (!ctx) throw new Error('useCustomer must be used inside CustomerProvider');  
  return ctx;  
}
```

Fichier : src/contexts/CartContext.tsx

writeCartToSession() — ligne 87

Fonction : `write cart to session` — voir le code ci-dessous.

```
function writeCartToSession(items: CartItem[]): void {
    sessionStorage.setItem(CART_KEY, JSON.stringify(items));
}
```

CartProvider() — ligne 93

[illegible]

```

export const CartProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const { customer } = useCustomer();
  const [items, setItems] = useState<CartItem[]>(readCartFromSession);
  const [remoteCartId, setRemoteCartId] = useState<string | null>(null);
  const [loadingRemoteCart, setLoadingRemoteCart] = useState(false);
  const syncTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const prevCustomerIdRef = useRef<string | null>(null);

  // ■■ Persist to sessionStorage on every change ■■
  useEffect(() => {
    writeCartToSession(items);
  }, [items]);

  // ■■ Load remote cart when customer logs in ■■
  useEffect(() => {
    const customerId = customer?.id ?? null;

    // Skip if customer hasn't changed
    if (customerId === prevCustomerIdRef.current) return;
    prevCustomerIdRef.current = customerId;

    if (!customerId) {
      // Customer logged out → clear cart
      setItems([]);
      setRemoteCartId(null);
      sessionStorage.removeItem(CART_KEY);
      return;
    }

    // Customer logged in → fetch their remote cart
    let cancelled = false;
    setLoadingRemoteCart(true);

    (async () => {
      try {
        const remote = await fetchCustomerCart(customerId);
        if (cancelled) return;

        if (remote && remote.items.length > 0) {
          setRemoteCartId(remote.cartId);

          // Load full product details for each remote cart item
          const loadedItems: CartItem[] = [];
          for (const row of remote.items) {
            try {
              const detail = await fetchShopProductDetail(row.productId);
              if (cancelled) return;
              if (detail) {
                let priceHt = detail.product.priceHt;
                let priceTtc = detail.product.priceTtc;
                let variantLabel: string | undefined;

                if (row.attributeId) {
                  const combo = detail.combinations.find(c => c.id === row.attributeId);
                  if (combo) {

```

```

        priceHt = detail.product.priceHt + combo.priceImpact;
        priceTtc = priceHt * (1 + detail.product.taxRate);
        variantLabel = combo.label;
    }
}

loadedItems.push({
    id: detail.product.id,
    attributeId: row.attributeId,
    variantLabel,
    name: detail.product.name,
    priceHt,
    priceTtc,
    taxRate: detail.product.taxRate,
    qty: row.quantity,
    imageUrl: detail.images[0],
});
}
} catch {
    // Skip products that can't be loaded
    console.warn('[CartContext] Could not load product', row.productId);
}
}

if (!cancelled && loadedItems.length > 0) {
    setItems(loadedItems);
    console.log('[CartContext] Loaded remote cart with', loadedItems.length, 'items');
}
} else {
    // No remote cart – start fresh
    setItems([]);
    setRemoteCartId(null);
}
} catch (err) {
    console.error('[CartContext] Failed to load remote cart:', err);
} finally {
    if (!cancelled) setLoadingRemoteCart(false);
}
})();

return () => { cancelled = true; };
}, [customer]);

// ■■ Debounced remote sync ■■
const scheduleSyncRef = useRef<((newItems: CartItem[]) => void) | undefined>(undefined);

const scheduleSync = useCallback((newItems: CartItem[]) => {
    if (!customer) return;
    if (syncTimerRef.current) clearTimeout(syncTimerRef.current);

    syncTimerRef.current = setTimeout(async () => {
        const cartId = await syncRemoteCart({
            customerId: customer.id,
            items: newItems,
            existingCartId: remoteCartId ?? undefined,
        });
        if (cartId) {
            setRemoteCartId(cartId);
        }
    }, 500);
}, [customer, remoteCartId]);

// Keep ref in sync
useEffect(() => {
    scheduleSyncRef.current = scheduleSync;
}, [scheduleSync]);

// ■■ Cart operations ■■

const addItem = (item: Omit<CartItem, 'qty'>, qty = 1) => {
    setItems(prev => {

```

```

    const key = itemKey(item.id, item.attributeId);
    const idx = prev.findIndex(i => itemKey(i.id, i.attributeId) === key);
    let next: CartItem[];
    if (idx >= 0) {
      next = [...prev];
      next[idx] = { ...next[idx], qty: next[idx].qty + qty };
    } else {
      next = [...prev, { ...item, qty }];
    }
    scheduleSyncRef.current?.(next);
    return next;
  });
};

const removeItem = (id: string, attributeId?: string) => {
  const key = itemKey(id, attributeId);
  setItems(prev => {
    const next = prev.filter(i => itemKey(i.id, i.attributeId) !== key);
    scheduleSyncRef.current?.(next);
    return next;
  });
};

const updateQty = (id: string, qty: number, attributeId?: string) => {
  if (qty <= 0) { removeItem(id, attributeId); return; }
  const key = itemKey(id, attributeId);
  setItems(prev => {
    const next = prev.map(i => itemKey(i.id, i.attributeId) === key ? { ...i, qty } : i);
    scheduleSyncRef.current?.(next);
    return next;
  });
};

const clear = () => {
  setItems([]);
  setRemoteCartId(null);
  sessionStorage.removeItem(CART_KEY);
};

// ■■ Computed values ■■

const totalPriceHt = roundMoney(items.reduce((sum, i) => sum + i.priceHt * i.qty, 0));
const totalPrice = roundMoney(items.reduce((sum, i) => sum + i.priceTtc * i.qty, 0));
const totalTax = roundMoney(totalPrice - totalPriceHt);
const totalItems = items.reduce((sum, i) => sum + i.qty, 0);

return (
  <CartContext.Provider value={{
    items,
    addItem,
    removeItem,
    updateQty,
    clear,
    totalPrice,
    totalPriceHt,
    totalTax,
    totalItems,
    remoteCartId,
    loadingRemoteCart,
  }}>
    {children}
  </CartContext.Provider>
);
}

```

addItem() — ligne 217

Description (commentaire d'origine) : ■■ Cart operations ■■

```

const addItem = (item: Omit<CartItem, 'qty'>, qty = 1) => {
  setItems(prev => {
    const key = itemKey(item.id, item.attributeId);
    const idx = prev.findIndex(i => itemKey(i.id, i.attributeId) === key);
    let next: CartItem[];
    if (idx >= 0) {
      next = [...prev];
      next[idx] = { ...next[idx], qty: next[idx].qty + qty };
    } else {
      next = [...prev, { ...item, qty }];
    }
    scheduleSyncRef.current?.(next);
    return next;
  });
}

```

removeItem() — ligne 233

Fonction : `remove item` — voir le code ci-dessous.

```

const removeItem = (id: string, attributeId?: string) => {
  const key = itemKey(id, attributeId);
  setItems(prev => {
    const next = prev.filter(i => itemKey(i.id, i.attributeId) !== key);
    scheduleSyncRef.current?.(next);
    return next;
  });
}

```

updateQty() — ligne 242

Fonction : `update qty` — voir le code ci-dessous.

```

const updateQty = (id: string, qty: number, attributeId?: string) => {
  if (qty <= 0) { removeItem(id, attributeId); return; }
  const key = itemKey(id, attributeId);
  setItems(prev => {
    const next = prev.map(i => itemKey(i.id, i.attributeId) === key ? { ...i, qty } : i);
    scheduleSyncRef.current?.(next);
    return next;
  });
}

```

clear() — ligne 252

Fonction : `clear` — voir le code ci-dessous.

```

const clear = () => {
  setItems([]);
  setRemoteCartId(null);
  sessionStorage.removeItem(CART_KEY);
}

```

useCart() — ligne 284

Fonction : `use cart` — voir le code ci-dessous.

```

export function useCart(): CartContextType {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart must be used within CartProvider');
  return ctx;
}

```

Hooks personnalisés

Fichier :

src/hooks/useDashboardData.ts

Lignes : 62 • Fonctions détectées : 2

aggregateOrdersByDay() — ligne 4

Fonction : `aggregate orders by day` — voir le code ci-dessous.

```
function aggregateOrdersByDay(orders: OrderFromApi[]): DailyOrderStat[] {
  const map = new Map<string, { count: number; amount: number }>();
  orders.forEach(order => {
    const date = order.date_add.split(' ')[0];
    const amount = parseFloat(order.total_paid_tax_incl);
    const existing = map.get(date);
    if (existing) {
      existing.count++;
      existing.amount += amount;
    } else {
      map.set(date, { count: 1, amount });
    }
  });
  return Array.from(map.entries())
    .map(([date, { count, amount }]) => ({ date, orderCount: count, totalAmount: amount }))
    .sort((a, b) => a.date.localeCompare(b.date));
}
```

useDashboardData() — ligne 22

Fonction : `use dashboard data` — voir le code ci-dessous.

```
export function useDashboardData(filters: DashboardFilters, autoRefreshIntervalMs = 60000) {
  const [data, setData] = useState<DashboardData | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const loadData = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      // Récupère toutes les commandes à chaque appel (pas de cache d'état pour éviter boucle)
      const allOrders = await fetchOrders();

      // Filtre selon selectedDate
      const filteredOrders = filters.selectedDate
        ? allOrders.filter(order => order.date_add.split(' ')[0] === filters.selectedDate)
        : allOrders;

      const dailyStats = aggregateOrdersByDay(allOrders);
      const totalOrders = filteredOrders.length;
      const totalRevenue = filteredOrders.reduce((sum, o) => sum + parseFloat(o.total_paid_tax_incl), 0);
      const averageOrderValue = totalOrders === 0 ? 0 : totalRevenue / totalOrders;

      setData({ stats: { totalOrders, totalRevenue, averageOrderValue }, dailyStats });
    } catch (err: any) {
      setError(err.message || 'Erreur de chargement');
    } finally {
      setLoading(false);
    }
  }, [filters.selectedDate]);
}
```

```

    }, [filters]); // plus de dépendance à allOrders

    useEffect(() => {
      loadData();
      if (autoRefreshIntervalMs > 0) {
        const interval = setInterval(loadData, autoRefreshIntervalMs);
        return () => clearInterval(interval);
      }
    }, [loadData, autoRefreshIntervalMs]);

    return { data, loading, error, refetch: loadData };
  }

```

Utilitaires

Fichier : `src/utls/formatCurrency.ts`

Lignes : 3 • Fonctions détectées : 1

`formatCurrency()` — ligne 1

Fonction : `format currency` — voir le code ci-dessous.

```
export function formatCurrency(amount: number): string {  
  return new Intl.NumberFormat('fr-FR', { style: 'currency', currency: 'EUR' }).format(amount);  
}
```

Fichier : `src/utls/productBadges.ts`

Lignes : 79 • Fonctions détectées : 3

`parseLocalDateParts()` — ligne 10

Fonction : `parse local date parts` — voir le code ci-dessous.

```
function parseLocalDateParts(value: string): Date | null {  
  const match = value.match(  
    /^(\d{4})-(\d{2})-(\d{2})(?:[ T](\d{2}):(\d{2})(?::(\d{2}))?)?$/  
  );  
  if (!match) return null;  
  const year = Number(match[1]);  
  const month = Number(match[2]) - 1;  
  const day = Number(match[3]);  
  const hour = Number(match[4] ?? '0');  
  const minute = Number(match[5] ?? '0');  
  const second = Number(match[6] ?? '0');  
  const date = new Date(year, month, day, hour, minute, second);  
  return Number.isNaN(date.getTime()) ? null : date;  
}
```

`parseAvailabilityDate()` — ligne 25

Fonction : `parse availability date` — voir le code ci-dessous.

```
export function parseAvailabilityDate(value?: string | null): Date | null {  
  if (!value) return null;  
  const trimmed = value.trim();  
  if (!trimmed) return null;  
  if (trimmed.startsWith('0000-00-00')) {  
    console.log(`[parseAvailabilityDate] rejeté (0000-00-00) : "${trimmed}"`);  
    return null;  
  }  
  
  const hasTimeZone = /z$|[-+]\d{2}:?\d{2}$/i.test(trimmed);  
  if (hasTimeZone) {  
    const parsed = new Date(trimmed);  
    const result = Number.isNaN(parsed.getTime()) ? null : parsed;  
    console.log(`[parseAvailabilityDate] timezone détecté : "${trimmed}" →`, result);  
    return result;  
  }  
}
```

```

const local = parseLocalDateParts(trimmed);
if (local) {
  console.log(`[parseAvailabilityDate] local parts : "${trimmed}" →`, local.toISOString());
  return local;
}

const parsed = new Date(trimmed);
const result = Number.isNaN(parsed.getTime()) ? null : parsed;
console.log(`[parseAvailabilityDate] fallback Date() : "${trimmed}" →`, result?.toISOString() ?? 'null');
return result;
}

```

getProductBadge() — ligne 54

Fonction : `get product badge` — voir le code ci-dessous.

```

export function getProductBadge(
  dateAvailability?: string | null,
  nowMs: number = Date.now()
): ProductBadge | null {
  const date = parseAvailabilityDate(dateAvailability);
  if (!date) return null;
  const ageMs = nowMs - date.getTime();
  const ageDays = (ageMs / (24 * 60 * 60 * 1000)).toFixed(2);
  console.log(`[getProductBadge] date="${dateAvailability}" | now=${new Date(nowMs).toISOString()} | âge=${ageDays}`);
  if (ageMs < 0) {
    console.log(`[getProductBadge] → null (date dans le futur)`);
    return null;
  }

  for (const rule of BADGE_RULES) {
    if (ageMs < rule.maxAgeMs) {
      console.log(`[getProductBadge] → ${rule.badge}`);
      return rule.badge;
    }
  }

  console.log(`[getProductBadge] → null (trop ancien)`);
  return null;
}

```

Types

Fichier : `src/types/dashboard.types.ts`

Lignes : 18 • Fonctions détectées : 0

Aucune fonction détectée — contenu complet du fichier :

```
export interface DailyOrderStat {
  date: string;
  orderCount: number;
  totalAmount: number;
}

export interface DashboardData {
  stats: {
    totalOrders: number;
    totalRevenue: number;
    averageOrderValue: number;
  };
  dailyStats: DailyOrderStat[];
}

export interface DashboardFilters {
  selectedDate: string | null;
}
```

Services — Authentication

Fichier : `src/services/authService.ts`

Lignes : 151 • Fonctions détectées : 3

`persistUser()` — ligne 59

Fonction : `persist user` — voir le code ci-dessous.

```
function persistUser(user: AuthUser): void {  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(user));  
}
```

`clearUser()` — ligne 63

Fonction : `clear user` — voir le code ci-dessous.

```
function clearUser(): void {  
  localStorage.removeItem(STORAGE_KEY);  
}
```

`extractAdminToken()` — ligne 67

Fonction : `extract admin token` — voir le code ci-dessous.

```
function extractAdminToken(redirectUrl: string): string {  
  const match = redirectUrl.match(/token=([a-f0-9]+)/i);  
  return match?.[1] ?? '';  
}
```

Fichier :

`src/services/PrestashopApi.tsx`

Lignes : 15 • Fonctions détectées : 1

`uploadProductImage()` — ligne 10

Fonction : `upload product image` — voir le code ci-dessous.

```
export const uploadProductImage = (reference: string, imageBlob: Blob) => {  
  const formData = new FormData();  
  formData.append('image', imageBlob);  
  formData.append('reference', reference);  
  return axios.post(`${API_BASE}/products/image`, formData);  
}
```

Services — Produits & Catalogue

Fichier : `src/services/produitApi.ts`

Lignes : 487 • Fonctions détectées : 9

`parseXMLToJson()` — ligne 384

Description (commentaire d'origine) : ===== 5.

PARSER DOM =====

```
function parseXMLToJson(xmlString: string): unknown {
  const parser = new DOMParser();
  const xmlDoc = parser.parseFromString(xmlString, "text/xml");
  function parseNode(node: Element): unknown {
    if (node.children.length === 0) return node.textContent || "";
    const obj: Record<string, unknown> = {};
    for (let i = 0; i < node.children.length; i++) {
      const child = node.children[i];
      const nodeName = child.nodeName;
      const value = parseNode(child);
      if (obj[nodeName]) {
        const current = obj[nodeName];
        if (!Array.isArray(current)) obj[nodeName] = [current];
        (obj[nodeName] as unknown[]).push(value);
      } else {
        obj[nodeName] = value;
      }
    }
    return obj;
  }
  return parseNode(xmlDoc.documentElement);
}
```

`parseNode()` — ligne 387

Fonction : `parse node` — voir le code ci-dessous.

```
function parseNode(node: Element): unknown {
  if (node.children.length === 0) return node.textContent || "";
  const obj: Record<string, unknown> = {};
  for (let i = 0; i < node.children.length; i++) {
    const child = node.children[i];
    const nodeName = child.nodeName;
    const value = parseNode(child);
    if (obj[nodeName]) {
      const current = obj[nodeName];
      if (!Array.isArray(current)) obj[nodeName] = [current];
      (obj[nodeName] as unknown[]).push(value);
    } else {
      obj[nodeName] = value;
    }
  }
  return obj;
}
```

`getPrestashopRoot()` — ligne 411

Fonction : `get prestashop root` — voir le code ci-dessous.

```
function getPrestashopRoot(xmlData: unknown): Record<string, unknown> | null {
```

```

    if (!isRecord(xmlData)) return null;
    const root = isRecord(xmlData.prestashop) ? xmlData.prestashop : xmlData;
    return isRecord(root) ? root : null;
}

```

normalizeText() — ligne 417

Fonction : `normalize text` — voir le code ci-dessous.

```

function normalizeText(value?: string): string {
    return value?.trim() ?? '';
}

```

buildRange() — ligne 421

Fonction : `build range` — voir le code ci-dessous.

```

function buildRange(min?: number, max?: number): string | null {
    if (min == null && max == null) return null;
    const start = typeof min === 'number' ? min : 0;
    const end = typeof max === 'number' ? max : MAX_RANGE_VALUE;
    return `[${start},${end}]`;
}

```

isRecord() — ligne 428

Fonction : `is record` — voir le code ci-dessous.

```

function isRecord(value: unknown): value is Record<string, unknown> {
    return typeof value === 'object' && value !== null;
}

```

extractId() — ligne 432

Fonction : `extract id` — voir le code ci-dessous.

```

function extractId(value: unknown): string | null {
    if (Array.isArray(value)) {
        return value.length > 0 ? extractId(value[0]) : null;
    }
    if (!isRecord(value)) return null;
    const id = value.id;
    if (typeof id === 'string' || typeof id === 'number') {
        return String(id);
    }
    return null;
}

```

applyIdRange() — ligne 444

Fonction : `apply id range` — voir le code ci-dessous.

```

function applyIdRange(ids: string[], min?: number, max?: number): string[] {
    if (min == null && max == null) return ids;
    return ids.filter((id) => {
        const idValue = Number(id);
        if (Number.isNaN(idValue)) return false;
        if (typeof min === 'number' && idValue < min) return false;
        if (typeof max === 'number' && idValue > max) return false;
        return true;
    });
}

```

getProductIdsByStockRange() — ligne 455

Fonction : `get product ids by stock range` — voir le code ci-dessous.

```
async function getProductIdsByStockRange(
  min?: number,
  max?: number
): Promise<string[] | null> {
  const range = buildRange(min, max);
  if (!range) return null;

  const params = new URLSearchParams();
  params.set('display', '[id_product]');
  params.set('filter[quantity]', range);

  const response = await api.get(`/stock_availability?${params.toString()}`);
  const xmlData = parseXMLToJSON(response.data);
  const root = isRecord(xmlData) ? xmlData.prestashop ?? xmlData : null;
  const stockRoot = isRecord(root) ? root.stock_availability : null;
  const rawStock = isRecord(stockRoot) ? stockRoot.stock_available : null;

  if (!rawStock) return [];

  const stockArray = Array.isArray(rawStock) ? rawStock : [rawStock];
  return stockArray
    .map((item) => {
      if (!isRecord(item)) return null;
      const idProduct = item.id_product;
      if (typeof idProduct === 'string' || typeof idProduct === 'number') {
        return String(idProduct);
      }
      return null;
    })
    .filter((id): id is string => Boolean(id));
}
```

Fichier : `src/services/shopService.ts`

Lignes : 236 • Fonctions détectées : 11

roundMoney() — ligne 28

Fonction : `round money` — voir le code ci-dessous.

```
function roundMoney(value: number): number {
    return Math.round((value + Number.EPSILON) * 1_000_000) / 1_000_000;
}
```

calcTtc() — ligne 32

Fonction : `calc ttc` — voir le code ci-dessous.

```
function calcTtc(priceHt: number, taxRate: number): number {
  return roundMoney(priceHt * (1 + taxRate));
}
```

parseStockXml() — ligne 38

[illegible]

```
function parseStockXml(xml: string): Map<string, number> {
```

fetchStockMap() — ligne 65

fetchProductImages() — ligne 76

fetchShopCategories() — ligne 91

fetchShopProducts() — ligne 109

fetchProductCombinations() — ligne 142

Page 23

```

type RawCombo = { id: string; priceImpact: number; optionValueIds: string[] };
const rawCombos: RawCombo[] = combEls.map(el => ({
  id: el.querySelector(':scope > id')?.textContent?.trim() ?? '',
  priceImpact: parseFloat(el.querySelector(':scope > price')?.textContent ?? '0'),
  optionValueIds: Array.from(el.querySelectorAll('product_option_value id'))
    .map(x => x.textContent?.trim() ?? '').filter(Boolean),
})).filter(c => c.id);

// Parse stock per combination
const stockDoc = new DOMParser().parseFromString(stockRes.data, 'text/xml');
const stockMap = new Map<string, number>();
stockDoc.querySelectorAll('stock_available').forEach(el => {
  const attr = el.querySelector('id_product_attribute')?.textContent?.trim() ?? '';
  const qty = parseInt(el.querySelector('quantity')?.textContent ?? '0', 10);
  if (attr && attr !== '0') stockMap.set(attr, qty);
});

// Parse option value names
const ovDoc = new DOMParser().parseFromString(ovRes.data, 'text/xml');
const valueNames = new Map<string, string>();
ovDoc.querySelectorAll('product_option_value').forEach(el => {
  const id = el.querySelector(':scope > id')?.textContent?.trim() ?? '';
  const name = el.querySelector('name language')?.textContent?.trim()
    ?? el.querySelector('name')?.textContent?.trim() ?? '';
  if (id && name) valueNames.set(id, name);
});

return rawCombos.map(c => ({
  id: c.id,
  label: c.optionValueIds.map(vid => valueNames.get(vid) ?? vid).join(' / ') || `Déclinaison ${c.id}`,
  priceImpact: c.priceImpact,
  quantity: stockMap.get(c.id) ?? 0,
}));
} catch {
  return [];
}
}

```

Description	(commentaire	d'origine)	:	■■	Product	detail

```
    } catch { return null; }
}
```

formatPrice() — ligne 226

[illegible]

```
export function formatPrice(price: number): string {
  return price.toLocaleString('fr-FR', { style: 'currency', currency: 'EUR' });
}
```

stockStatus() — ligne 230

Fonction : `stock status` — voir le code ci-dessous.

```
export function stockStatus(qty: number): { label: string; level: 'ok' | 'low' | 'out' }
```

Fichier : `src/services/taxService.ts`

Lignes : 188 • Fonctions détectées : 11

rateKey() — ligne 21

Fonction : `rate key` — voir le code ci-dessous.

```
function rateKey(ratePercent: number): string {
    return ratePercent.toFixed(3);
}
```

parseTaxRules() — ligne 25

Fonction : `parse tax rules` — voir le code ci-dessous.

```
function parseTaxRules(xml: string): TaxRule[] {
  const doc = new DOMParser().parseFromString(xml, 'text/xml');
  const rules: TaxRule[] = [];
  doc.querySelectorAll('tax_rule').forEach((el) => {
    const groupId = parseInt(el.querySelector('id_tax_rules_group')?.textContent ?? '0', 10);
    const taxId = parseInt(el.querySelector('id_tax')?.textContent ?? '0', 10);
    if (groupId && taxId) rules.push({ groupId, taxId });
  });
  return rules;
}
```

parseTaxes() — ligne 36

Fonction : `parse taxes` — voir le code ci-dessous.

```
function parseTaxes(xml: string): Map<number, number> {
  const doc = new DOMParser().parseFromString(xml, 'text/xml');
  const map = new Map<number, number>();
  doc.querySelectorAll('tax').forEach((el) => {
    const id = parseInt(el.querySelector('id')?.textContent ?? '0', 10);
    const rate = parseFloat(el.querySelector('rate')?.textContent ?? '0');
    if (id) map.set(id, rate);
  });
  return map;
}
```

parseCreatedId() — ligne 47

Fonction : `parse created id` — voir le code ci-dessous.

```
function parseCreatedId(xml: string): number | null {
  try {
    const doc = new DOMParser().parseFromString(xml, 'text/xml');
    const id = doc.querySelector('id')?.textContent?.trim();
    if (!id) return null;
    const parsed = parseInt(id, 10);
    return Number.isNaN(parsed) ? null : parsed;
  } catch {
    return null;
  }
}
```

createTax() — ligne 59

Fonction : `create tax` — voir le code ci-dessous.

```
async function createTax(ratePercent: number): Promise<number> {
  const label = `Auto ${ratePercent.toFixed(3)}%`;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <tax>
    <rate><![CDATA[${ratePercent.toFixed(3)}]]></rate>
    <active><![CDATA[1]]></active>
    <name><language id="1"><![CDATA[${label}]]></language></name>
  </tax>
</prestashop>`;
  const res = await api.post('/taxes', xml);
  const id = parseCreatedId(res.data);
  if (!id) throw new Error('Creation taxe: ID manquant');
  return id;
}
```

createTaxRuleGroup() — ligne 75

Fonction : `create tax rule group` — voir le code ci-dessous.

```
async function createTaxRuleGroup(ratePercent: number): Promise<number> {
  const label = `Auto ${ratePercent.toFixed(3)}%`;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <tax_rule_group>
    <name><![CDATA[${label}]]></name>
    <active><![CDATA[1]]></active>
  </tax_rule_group>
</prestashop>`;
  const res = await api.post('/tax_rule_groups', xml);
  const id = parseCreatedId(res.data);
  if (!id) throw new Error('Creation groupe de taxe: ID manquant');
  return id;
}
```

createTaxRule() — ligne 90

Fonction : `create tax rule` — voir le code ci-dessous.

```
async function createTaxRule(groupId: number, taxId: number): Promise<number> {
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <tax_rule>
    <id_tax_rules_group><![CDATA[${groupId}]]></id_tax_rules_group>
    <id_country><![CDATA[${DEFAULT_COUNTRY_ID}]]></id_country>
  </tax_rule>
</prestashop>`;
  const res = await api.post('/tax_rules', xml);
  const id = parseCreatedId(res.data);
  if (!id) throw new Error('Creation règle de taxe: ID manquant');
  return id;
}
```

```

    <id_state><![CDATA[${DEFAULT_STATE_ID}]]></id_state>
    <zipcode_from><![CDATA[${DEFAULT_ZIP_FROM}]]></zipcode_from>
    <zipcode_to><![CDATA[${DEFAULT_ZIP_TO}]]></zipcode_to>
    <id_tax><![CDATA[${taxId}]]></id_tax>
    <behavior><![CDATA[${DEFAULT_BEHAVIOR}]]></behavior>
    <description><![CDATA[Auto tax rule]]></description>
  </tax_rule>
</prestashop>`;
const res = await api.post('/tax_rules', xml);
const id = parseCreatedId(res.data);
if (!id) throw new Error('Creation regle de taxe: ID manquant');
return id;
}

```

loadTaxCaches() — ligne 110

Fonction : load tax caches — voir le code ci-dessous.

```

async function loadTaxCaches(): Promise<void> {
  if (taxCache.loaded) return;
  const [taxRulesRes, taxesRes] = await Promise.all([
    api.get('/tax_rules?display=[id_tax,id_tax_rules_group]'),
    api.get('/taxes?display=[id,rate]'),
  ]);

  const rules = parseTaxRules(taxRulesRes.data);
  const taxRates = parseTaxes(taxesRes.data);

  for (const rule of rules) {
    if (taxCache.groupRate.has(rule.groupId)) continue;
    const ratePercent = taxRates.get(rule.taxId);
    if (ratePercent == null) continue;
    taxCache.groupRate.set(rule.groupId, ratePercent);
    const key = rateKey(ratePercent);
    if (!taxCache.rateGroup.has(key)) {
      taxCache.rateGroup.set(key, rule.groupId);
    }
  }

  taxCache.loaded = true;
}

```

getTaxRateByGroup() — ligne 134

Fonction : get tax rate by group — voir le code ci-dessous.

```

export async function getTaxRateByGroup(groupId: number): Promise<number> {
  if (!groupId) return 0;
  try {
    await loadTaxCaches();
  } catch {
    return 0;
  }
  const ratePercent = taxCache.groupRate.get(groupId) ?? 0;
  return ratePercent / 100;
}

```

resolveTaxRulesGroupIdByRate() — ligne 145

Fonction : resolve tax rules group id by rate — voir le code ci-dessous.

```

export async function resolveTaxRulesGroupIdByRate(rate: number): Promise<number | null> {
  try {
    await loadTaxCaches();
  } catch (err) {
    console.error('Failed to load tax rules for rate lookup', err);
  }
}

```

```

    return null;
  }
  const percent = Math.max(0, rate * 100);
  const key = rateKey(percent);
  const groupId = taxCache.rateGroup.get(key) ?? null;
  if (!groupId) {
    console.error('No tax group found for rate', {
      rate: percent,
      availableRates: Array.from(taxCache.rateGroup.keys()),
    });
  }
  return groupId;
}

```

ensureTaxRulesGroupIdByRate() — ligne 164

Fonction : `ensure tax rules group id by rate` — voir le code ci-dessous.

```

export async function ensureTaxRulesGroupIdByRate(rate: number): Promise<number | null> {
  const existing = await resolveTaxRulesGroupIdByRate(rate);
  if (existing) return existing;

  const ratePercent = Math.max(0, rate * 100);
  try {
    console.log('Creating tax group for rate', ratePercent);
    const [taxId, groupId] = await Promise.all([
      createTax(ratePercent),
      createTaxRuleGroup(ratePercent),
    ]);
    await createTaxRule(groupId, taxId);
    taxCache.groupRate.set(groupId, ratePercent);
    taxCache.rateGroup.set(rateKey(ratePercent), groupId);
    return groupId;
  } catch (err) {
    console.error('Failed to auto-create tax group', {
      rate: ratePercent,
      error: err,
    });
    return null;
  }
}

```

Services — Stock

Fichier : `src/services/stockApi.ts`

Lignes : 200 • Fonctions détectées : 4

`parseXMLToJSON()` — ligne 71

Description (commentaire d'origine) : ===== 4.
HELPERS (parsing XML, extraction) =====

```
function parseXMLToJSON(xmlString: string): unknown {
  const parser = new DOMParser();
  const xmlDoc = parser.parseFromString(xmlString, "text/xml");
  function parseNode(node: Element): unknown {
    if (node.children.length === 0) return node.textContent || "";
    const obj: Record<string, unknown> = {};
    for (let i = 0; i < node.children.length; i++) {
      const child = node.children[i];
      const nodeName = child.nodeName;
      const value = parseNode(child);
      if (obj[nodeName]) {
        const current = obj[nodeName];
        if (!Array.isArray(current)) obj[nodeName] = [current];
        (obj[nodeName] as unknown[]).push(value);
      } else {
        obj[nodeName] = value;
      }
    }
    return obj;
  }
  return parseNode(xmlDoc.documentElement);
}
```

`parseNode()` — ligne 74

Fonction : `parse node` — voir le code ci-dessous.

```
function parseNode(node: Element): unknown {
  if (node.children.length === 0) return node.textContent || "";
  const obj: Record<string, unknown> = {};
  for (let i = 0; i < node.children.length; i++) {
    const child = node.children[i];
    const nodeName = child.nodeName;
    const value = parseNode(child);
    if (obj[nodeName]) {
      const current = obj[nodeName];
      if (!Array.isArray(current)) obj[nodeName] = [current];
      (obj[nodeName] as unknown[]).push(value);
    } else {
      obj[nodeName] = value;
    }
  }
  return obj;
}
```

`getPrestashopRoot()` — ligne 93

Fonction : `get prestashop root` — voir le code ci-dessous.

```
function getPrestashopRoot(xmlData: unknown): Record<string, unknown> | null {
```

```

    if (typeof xmlData !== 'object' || xmlData === null) return null;
    const root = (xmlData as any).prestashop ?? xmlData;
    return typeof root === 'object' && root !== null ? root as Record<string, unknown> : null;
}

```

isRecord() — ligne 99

Fonction : `is record` — voir le code ci-dessous.

```

function isRecord(value: unknown): value is Record<string, unknown> {
    return typeof value === 'object' && value !== null;
}

```

Fichier : src/services/stockService.ts

Lignes : 581 • **Fonctions détectées :** 15

sendMovementToAPI() — ligne 56

Fonction : `send movement to api` — voir le code ci-dessous.

```

async function sendMovementToAPI(movement: {
    id: string;
    key: string;
    id_product: number;
    product_name: string;
    combination_label: string;
    id_stock_available: number;
    quantity_before: number;
    quantity_added: number;
    quantity_after: number;
    note: string;
    date?: string;
}): Promise<void> {
    const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop>
  <movement>
    <id><![CDATA[${movement.id}]]></id>
    <key><![CDATA[${movement.key}]]></key>
    <id_product><![CDATA[${movement.id_product}]]></id_product>
    <product_name><![CDATA[${movement.product_name}]]></product_name>
    <combination_label><![CDATA[${movement.combination_label}]]></combination_label>
    <id_stock_available><![CDATA[${movement.id_stock_available}]]></id_stock_available>
    <quantity_before><![CDATA[${movement.quantity_before}]]></quantity_before>
    <quantity_added><![CDATA[${movement.quantity_added}]]></quantity_added>
    <quantity_after><![CDATA[${movement.quantity_after}]]></quantity_after>
    <date><![CDATA[${movement.date || new Date().toISOString()}]]></date>
    <note><![CDATA[${movement.note}]]></note>
  </movement>
</prestashop>`;

    await moduleApi.post('/stockapi.php?action=add_movement', xml);
}

```

parseXML() — ligne 92

Description (commentaire d'origine) : ===== 3.
HELPERS XML =====

```

function parseXML(xmlString: string): unknown {
    const parser = new DOMParser();
    const doc = parser.parseFromString(xmlString, 'text/xml');
}

```

```

function walk(node: Element): unknown {
  if (node.children.length === 0) return node.textContent ?? '';
  const obj: Record<string, unknown> = {};
  for (let i = 0; i < node.children.length; i++) {
    const child = node.children[i];
    const key = child.nodeName;
    const val = walk(child);
    if (key in obj) {
      if (!Array.isArray(obj[key])) obj[key] = [obj[key]];
      (obj[key] as unknown[]).push(val);
    } else {
      obj[key] = val;
    }
  }
  return obj;
}

return walk(doc.documentElement);
}

```

walk() — ligne 95

Fonction : `walk` — voir le code ci-dessous.

```

function walk(node: Element): unknown {
  if (node.children.length === 0) return node.textContent ?? '';
  const obj: Record<string, unknown> = {};
  for (let i = 0; i < node.children.length; i++) {
    const child = node.children[i];
    const key = child.nodeName;
    const val = walk(child);
    if (key in obj) {
      if (!Array.isArray(obj[key])) obj[key] = [obj[key]];
      (obj[key] as unknown[]).push(val);
    } else {
      obj[key] = val;
    }
  }
  return obj;
}

```

buildStockUpdateXml() — ligne 115

Fonction : `build stock update xml` — voir le code ci-dessous.

```

function buildStockUpdateXml(
  idProduct: string | number,
  idProductAttribute: string | number,
  delta: number
): string {
  return '<?xml version="1.0" encoding="UTF-8"?>' +
    '<prestashop><stock_update>' +
    '<id_product><![CDATA[' + idProduct + ']]></id_product>' +
    '<id_product_attribute><![CDATA[' + idProductAttribute + ']]></id_product_attribute>' +
    '<delta><![CDATA[' + delta + ']]></delta>' +
    '</stock_update></prestashop>';
}

```

parseStockUpdateResponse() — ligne 128

Fonction : `parse stock update response` — voir le code ci-dessous.

```

function parseStockUpdateResponse(xmlString: string): {
  success: boolean;
  newQty: number;
  error?: string;
}

```

}

getRoot() — ligne 143

Fonction : `get root` — voir le code ci-dessous.

```
function getRoot(parsed: unknown): Record<string, unknown> | null {
  if (!isObj(parsed)) return null;
  const ps = isObj(parsed.prestashop) ? parsed.prestashop : parsed;
  return isObj(ps) ? ps : null;
}
```

isObj() — ligne 149

Fonction : `is obj` — voir le code ci-dessous.

```
function isObj(v: unknown): v is Record<string, unknown> {
    return typeof v === 'object' && v !== null;
}
```

getLang() — ligne 153

Fonction : `get lang` — voir le code ci-dessous.

```
function getLang(field: unknown): string {
  if (!isObj(field)) return typeof field === 'string' ? field : '';
  const lang = field.language;
  if (!lang) return '';
  const target = Array.isArray(lang) ? lang[0] : lang;
  if (!isObj(target)) return typeof target === 'string' ? target : '';
  // Gère _cdata (parfois injecté), _text, ou string direct
  return String((target as any)._cdata ?? (target as any)._text ?? target ?? '');
}
```

toArray() — ligne 163

Fonction : `to array` — voir le code ci-dessous.

```
function toArray<T>(v: T | T[] | undefined | null): T[] {
  if (v == null) return [];
  return Array.isArray(v) ? v : [v];
}
```

fetchProducts() — ligne 198

Fonction : `fetch_products` — voir le code ci-dessous.

```
async function fetchProducts(): Promise<Map<string, RawProduct>> {  
    const res = await api.get('/products?display=full');  
    const root = getRoot(parseXML(res.data));  
    const container = root && isObj(root.products) ? root.products : null;  
    const raw = toArray((container as any)?.product);  
  
    const map = new Map<string, RawProduct>();  
    for (const p of raw) {  
        if (!isObj(p)) continue;  
        const id = String(p.id ?? '');  
        if (!id) continue;  
  
        // ■■ Détection du type  
        // PS peut renvoyer : "combinations", "configurable", "1", "2"...  
        // On s'appuie aussi sur la présence de l'association "combinations"  
        const rawType = String(  

```

```

    (p as any).type ?? (p as any).product_type ?? ''
  ).toLowerCase();

  const comboAssoc = toArray(
    (p as any).associations?.combinations?.combination
  );

  const isCombinations =
    rawType === 'combinations' ||
    rawType === 'configurable' ||
    rawType === '2' || // valeur numérique PS 1.7/8
    comboAssoc.length > 0;

  // IDs de déclinaisons extraits des associations (plus fiable que le
  // filtre global sur /combinations)
  const combinationIds = comboAssoc
    .map((c: any) => String(isObj(c) ? (c.id ?? '') : c))
    .filter(Boolean);

  map.set(id, {
    id,
    name: getLang(p.name),
    reference: String(p.reference ?? ''),
    ean13: String(p.ean13 ?? ''),
    type: isCombinations ? 'combinations' : 'simple',
    active: String(p.active ?? '0'),
    combinationIds,
  });
}
return map;
}

```

fetchCombinations() — ligne 246

Fonction : `fetch combinations` — voir le code ci-dessous.

```

async function fetchCombinations(): Promise<RawCombination[]> {
  try {
    const res = await api.get('/combinations?display=full');
    const root = getRoot(parseXML(res.data));
    const container = root && isObj(root.combinations) ? root.combinations : null;
    const raw = toArray((container as any)?.combination);

    return raw.filter(isObj).map((c: any) => {
      // Extrait les IDs des valeurs d'options (attributs: couleur, taille...)
      const pov = isObj(c.associations?.product_option_values)
        ? c.associations.product_option_values
        : null;
      const ovItems = toArray((pov as any)?.product_option_value);
      const optionValueIds = ovItems
        .map((ov: any) => String(isObj(ov) ? (ov.id ?? '') : ov))
        .filter(Boolean);

      return {
        id: String(c.id ?? ''),
        id_product: String(c.id_product ?? ''),
        reference: String(c.reference ?? ''),
        ean13: String(c.ean13 ?? ''),
        optionValueIds,
      } as RawCombination;
    }).filter(c => c.id && c.id_product);
  } catch (err) {
    console.warn('[stockService] /combinations introuvable ou vide', err);
    return [];
  }
}

```

fetchStockDisponibles() — ligne 277

Fonction : `fetch stock disponibles` — voir le code ci-dessous.

```
async function fetchStockDisponibles(): Promise<RawStockAvailable[]> {
  const res = await api.get('/stock_disponibles?display=full');
  const root = getRoot(parseXML(res.data));
  const container = root && isObj(root.stock_disponibles) ? root.stock_disponibles : null;
  const raw = toArray((container as any)?.stock_disponible);

  return raw.filter(isObj).map((s: any) => ({
    id: String(s.id ?? ''),
    id_product: String(s.id_product ?? ''),
    id_product_attribute: String(s.id_product_attribute ?? '0'),
    quantity: parseInt(String(s.quantity ?? '0'), 10),
  }));
}
```

`fetchOptionValueNames()` — ligne 291

Fonction : `fetch option value names` — voir le code ci-dessous.

```
async function fetchOptionValueNames(): Promise<Map<string, string>> {
  try {
    const res = await api.get('/product_option_values?display=full');
    const root = getRoot(parseXML(res.data));
    const container = root && isObj(root.product_option_values) ? root.product_option_values : null;
    const raw = toArray((container as any)?.product_option_value);

    const map = new Map<string, string>();
    for (const ov of raw) {
      if (!isObj(ov)) continue;
      const id = String(ov.id ?? '');
      const name = getLang(ov.name);
      if (id) map.set(id, name);
    }
    return map;
  } catch (err) {
    console.warn('[stockService] /product_option_values introuvable', err);
    return new Map();
  }
}
```

`readMovements()` — ligne 318

Fonction : `read movements` — voir le code ci-dessous.

```
function readMovements(): StockMovement[] {
  try {
    const raw = localStorage.getItem(MOVEMENTS_KEY);
    return raw ? (JSON.parse(raw) as StockMovement[]) : [];
  } catch {
    return [];
  }
}
```

`writeMovement()` — ligne 327

Fonction : `write movement` — voir le code ci-dessous.

```
function writeMovement(m: StockMovement): void {
  const all = readMovements();
  all.unshift(m);
  try {
    localStorage.setItem(MOVEMENTS_KEY, JSON.stringify(all.slice(0, MAX_MOVEMENTS)));
  } catch {
    console.error('[stockService] Impossible de sauvegarder le mouvement');
  }
}
```

}


```
);
const doc = new DOMParser().parseFromString(res.data, 'text/xml');
const el = doc.querySelector('customer');
if (!el) return null;
const id = el.querySelector('id')?.textContent?.trim();
if (!id) return null;
const secureKey = await fetchSecureKey(id);
return {
  id,
  email: el.querySelector('email')?.textContent?.trim() ?? email,
  firstname: el.querySelector('firstname')?.textContent?.trim() ?? '',
  lastname: el.querySelector('lastname')?.textContent?.trim() ?? '',
  secureKey,
};
} catch { return null; }
}
```

registerCustomer() — ligne 125

Fonction : `register customer` — voir le code ci-dessous.

```
export async function registerCustomer(data: {
  email: string;
  firstname: string;
  lastname: string;
  passwd: string;
}): Promise<Customer> {
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
<customer>
  <id_default_group><![CDATA[3]]></id_default_group>
  <id_lang><![CDATA[1]]></id_lang>
  <passwd><![CDATA[${data.passwd}]]></passwd>
  <lastname><![CDATA[${data.lastname}]]></lastname>
  <firstname><![CDATA[${data.firstname}]]></firstname>
  <email><![CDATA[${data.email}]]></email>
  <id_gender><![CDATA[1]]></id_gender>
  <newsletter><![CDATA[0]]></newsletter>
  <optin><![CDATA[0]]></optin>
  <active><![CDATA[1]]></active>
  <is_guest><![CDATA[0]]></is_guest>
  <id_shop><![CDATA[1]]></id_shop>
  <id_shop_group><![CDATA[1]]></id_shop_group>
</customer>
</prestashop>`;
  const res = await api.post('/customers', xml);
  const doc = new DOMParser().parseFromString(res.data, 'text/xml');
  const el = doc.querySelector('customer');
  if (!el) throw new Error('Échec de création du compte');
  const id = el.querySelector('id')?.textContent?.trim();
  if (!id) throw new Error('Échec de création du compte');
  const secureKey = await fetchSecureKey(id);
  return { id, email: data.email, firstname: data.firstname, lastname: data.lastname, secureKey };
}
```

getCustomerAddresses() — ligne 161

[illegible]

```
export async function getCustomerAddresses(customerId: string): Promise<Address[]> {
  try {
    const res = await api.get(
      `/addresses?display=[id,alias,firstname,lastname,address1,address2,postcode,city,phone]&filter[id_customer]=${customerId}`
    );
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    const list: Address[] = [];
  }
}
```



```

        <id_address_delivery><![CDATA[${addressId}]]></id_address_delivery>
        <id_customization><![CDATA[0]]></id_customization>
        <quantity><![CDATA[${item.qty}]]></quantity>
    </cart_row>`).join('');

    console.log('[createPSCart] Debug payload:', {
        customerId: key, addressId, carrierId,
        currencyId: 1, langId: 1, shopId: 1, shopGroupId: 1,
        cartRowsCount: items.length,
    });

    const xml = `<?xml version="1.0" encoding="UTF-8"?>
    <prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
    <cart>
        <id_address_delivery><![CDATA[${addressId}]]></id_address_delivery>
        <id_address_invoice><![CDATA[${addressId}]]></id_address_invoice>
        <id_currency><![CDATA[1]]></id_currency>
        <id_customer><![CDATA[${customerId}]]></id_customer>
        <id_guest><![CDATA[0]]></id_guest>
        <id_lang><![CDATA[1]]></id_lang>
        <id_shop_group><![CDATA[1]]></id_shop_group>
        <id_shop><![CDATA[1]]></id_shop>
        <id_carrier><![CDATA[${carrierId}]]></id_carrier>
        <secure_key><![CDATA[${key}]]></secure_key>
        <recyclable><![CDATA[0]]></recyclable>
        <gift><![CDATA[0]]></gift>
        <mobile_theme><![CDATA[0]]></mobile_theme>
        <allow_seperated_package><![CDATA[0]]></allow_seperated_package>
        <associations>
            <cart_rows>${rowsXml}</cart_rows>
        </associations>
    </cart>
    </prestashop>`;

    const res = await api.post('/carts', xml);
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    const id = doc.querySelector('cart > id')?.textContent?.trim();
    if (!id) throw new Error('Échec de création du panier');
    return id;
}

```

createPSOrder() — ligne 269

Fonction : `create psorder` — voir le code ci-dessous.

```

export async function createPSOrder(params: {
    customerId: string;
    addressId: string;
    cartId: string;
    carrierId: string;
    items: CheckoutItem[];
    shippingCost: number;
    dateAdd?: string;
}): Promise<string> {
    const totalProductsHt = params.items.reduce((s, i) => s + i.priceHt * i.qty, 0);
    const totalProductsTtc = params.items.reduce((s, i) => s + i.priceTtc * i.qty, 0);
    const shippingTaxRate = 0;
    const shippingHt = params.shippingCost;
    const shippingTtc = params.shippingCost;
    const totalPaidTtc = totalProductsTtc + shippingTtc;
    const totalPaidHt = totalProductsHt + shippingHt;
    const now = params.dateAdd || new Date().toISOString().slice(0, 19).replace('T', ' ');

    const secureKey = Array.from(crypto.getRandomValues(new Uint8Array(16)))
        .map(b => b.toString(16).padStart(2, '0')).join('');

    const rowsXml = params.items.map(item => `
        <order_row>
            <product_id><![CDATA[${item.id}]]></product_id>

```

```

        <product_attribute_id><![CDATA[${item.attributeId ?? '0'}]></product_attribute_id>
        <product_quantity><![CDATA[${item.qty}]></product_quantity>
        <product_name><![CDATA[${item.name}]></product_name>
        <product_reference><![CDATA[]]></product_reference>
        <product_ean13><![CDATA[]]></product_ean13>
        <product_isbn><![CDATA[]]></product_isbn>
        <product_upc><![CDATA[]]></product_upc>
        <product_price><![CDATA[${item.priceHt.toFixed(6)}]></product_price>
        <id_customization><![CDATA[0]]></id_customization>
        <unit_price_tax_incl><![CDATA[${item.priceTtc.toFixed(6)}]></unit_price_tax_incl>
        <unit_price_tax_excl><![CDATA[${item.priceHt.toFixed(6)}]></unit_price_tax_excl>
    </order_row>`).join('');

    const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
<order>
    <id_address_delivery><![CDATA[${params.addressId}]]></id_address_delivery>
    <id_address_invoice><![CDATA[${params.addressId}]]></id_address_invoice>
    <id_cart><![CDATA[${params.cartId}]]></id_cart>
    <id_currency><![CDATA[1]]></id_currency>
    <id_lang><![CDATA[1]]></id_lang>
    <id_customer><![CDATA[${params.customerId}]]></id_customer>
    <id_carrier><![CDATA[${params.carrierId}]]></id_carrier>
    <current_state><![CDATA[1]]></current_state>
    <module><![CDATA[ps_checkpayment]]></module>
    <payment><![CDATA[ Paiement accepté]]></payment>
    <invoice_number><![CDATA[0]]></invoice_number>
    <invoice_date><![CDATA[0000-00-00 00:00:00]]></invoice_date>
    <delivery_number><![CDATA[0]]></delivery_number>
    <delivery_date><![CDATA[0000-00-00 00:00:00]]></delivery_date>
    <valid><![CDATA[0]]></valid>
    <date_add><![CDATA[${now}]]></date_add>
    <date_upd><![CDATA[${now}]]></date_upd>
    <shipping_number><![CDATA[]]></shipping_number>
    <note><![CDATA[]]></note>
    <id_shop_group><![CDATA[1]]></id_shop_group>
    <id_shop><![CDATA[1]]></id_shop>
    <secure_key><![CDATA[${secureKey}]]></secure_key>
    <recyclable><![CDATA[0]]></recyclable>
    <gift><![CDATA[0]]></gift>
    <gift_message><![CDATA[]]></gift_message>
    <mobile_theme><![CDATA[0]]></mobile_theme>
    <total_discounts><![CDATA[0.000000]]></total_discounts>
    <total_discounts_tax_incl><![CDATA[0.000000]]></total_discounts_tax_incl>
    <total_discounts_tax_excl><![CDATA[0.000000]]></total_discounts_tax_excl>
    <total_paid><![CDATA[${totalPaidTtc.toFixed(6)}]]></total_paid>
    <total_paid_tax_incl><![CDATA[${totalPaidTtc.toFixed(6)}]]></total_paid_tax_incl>
    <total_paid_tax_excl><![CDATA[${totalPaidHt.toFixed(6)}]]></total_paid_tax_excl>
    <total_paid_real><![CDATA[0.000000]]></total_paid_real>
    <total_products><![CDATA[${totalProductsHt.toFixed(6)}]]></total_products>
    <total_products_wt><![CDATA[${totalProductsTtc.toFixed(6)}]]></total_products_wt>
    <total_shipping><![CDATA[${shippingTtc.toFixed(6)}]]></total_shipping>
    <total_shipping_tax_incl><![CDATA[${shippingTtc.toFixed(6)}]]></total_shipping_tax_incl>
    <total_shipping_tax_excl><![CDATA[${shippingHt.toFixed(6)}]]></total_shipping_tax_excl>
    <carrier_tax_rate><![CDATA[${(shippingTaxRate * 100).toFixed(6)}]]></carrier_tax_rate>
    <total_wrapping><![CDATA[0.000000]]></total_wrapping>
    <total_wrapping_tax_incl><![CDATA[0.000000]]></total_wrapping_tax_incl>
    <total_wrapping_tax_excl><![CDATA[0.000000]]></total_wrapping_tax_excl>
    <round_mode><![CDATA[2]]></round_mode>
    <round_type><![CDATA[1]]></round_type>
    <conversion_rate><![CDATA[1.000000]]></conversion_rate>
    <reference><![CDATA[]]></reference>
    <associations>
        <order_rows>${rowsXml}
        </order_rows>
    </associations>
</order>
</prestashop>`;

    // Création de la commande
    const res = await api.post('/orders', xml);

```

```

const doc = new DOMParser().parseFromString(res.data, 'text/xml');
const id = doc.querySelector('order > id')?.textContent?.trim();

if (!id) throw new Error('Échec de création de la commande');

// Force l'état 2 "Paieement accepté" via order_history – PS ignore current_state au POST
try {
  const stateXml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <order_history>
    <id_order><![CDATA[${id}]]></id_order>
    <id_order_state><![CDATA[2]]></id_order_state>
    <id_employee><![CDATA[0]]></id_employee>
  </order_history>
</prestashop>`;
  await api.post('/order_histories', stateXml);
} catch (stateErr: any) {
  console.warn('[createPSOrder] Impossible de forcer l\'état 2:', stateErr?.response?.data ?? stateErr.message);
}

// Enregistre les mouvements de sortie dans ps_stock_mvt (visible dans le backoffice)
// PS décrémente le stock via order_histories mais ne crée pas la ligne de mouvement via l'API
for (const item of params.items) {
  const attributeId = item.attributeId ?? '0';
  try {
    const stockRes = await api.get(
      `/stock_availability?display=[id]&filter[id_product]=[${item.id}]&filter[id_product_attribute]=[${attributeId}]`
    );
    const stockDoc = new DOMParser().parseFromString(stockRes.data, 'text/xml');
    const stockId = stockDoc.querySelector('stock_availability > id')?.textContent?.trim();
    if (!stockId) continue;
    const mvtXml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <stock_mvt>
    <id_employee><![CDATA[0]]></id_employee>
    <id_stock><![CDATA[${stockId}]]></id_stock>
    <id_stock_mvt_reason><![CDATA[2]]></id_stock_mvt_reason>
    <physical_quantity><![CDATA[${item.qty}]]></physical_quantity>
    <sign><![CDATA[-1]]></sign>
    <price_te><![CDATA[0]]></price_te>
    <date_add><![CDATA[${new Date().toISOString().slice(0, 19).replace('T', ' ')}]]></date_add>
  </stock_mvt>
</prestashop>`;
    await api.post('/stock_movements', mvtXml);
  } catch (mvtErr: any) {
    console.warn('[createPSOrder] Mouvement sortie produit ${item.id}:', mvtErr?.response?.status);
  }
}

// ■ FIX : Forcer la date après création car PrestaShop ignore date_add au POST
if (params.dateAdd) {
  try {
    console.log(`Récupération commande ${id} pour mise à jour date...`);

    // 1. Récupérer le XML complet de la commande créée
    const getRes = await api.get(`/orders/${id}`);
    let orderXml = getRes.data as string;

    // 2. Remplacer date_add et date_upd dans le XML retourné
    orderXml = orderXml
      .replace(
        /<date_add><![CDATA\[.*?\]]></date_add>/,
        `<date_add><![CDATA[${params.dateAdd}]]></date_add>`
      )
      .replace(
        /<date_upd><![CDATA\[.*?\]]></date_upd>/,
        `<date_upd><![CDATA[${params.dateAdd}]]></date_upd>`
      );

    // 3. PUT avec le XML complet modifié
    await api.put(`/orders/${id}`, orderXml);
  }
}

```

updateStockAfterOrder() — ligne 452

getCustomerOrders() — ligne 484

Page 42

```

        orders.push({
            id,
            reference: el.querySelector('reference')?.textContent?.trim() ?? `#${id}`,
            totalPaid: parseFloat(el.querySelector('total_paid')?.textContent ?? '0'),
            currentState: parseInt(el.querySelector('current_state')?.textContent ?? '0', 10),
            dateAdd: el.querySelector('date_add')?.textContent?.trim() ?? '',
        });
    });
    return orders;
} catch (error) {
    console.error('Failed to fetch customer orders:', error);
    return [];
}
}

```

Fichier : src/services/cartSyncService.ts

Lignes : 194 • Fonctions détectées : 4

resolveAddressId() — ligne 22

Description (commentaire d'origine) : Get the first valid address for a customer, or '0' if none found.

```

async function resolveAddressId(customerId: string): Promise<string> {
    try {
        const addresses = await getCustomerAddresses(customerId);
        return addresses.length > 0 ? addresses[0].id : '0';
    } catch {
        return '0';
    }
}

```

resolveSecureKey() — ligne 34

Description (commentaire d'origine) : Get the secure_key: from sessionStorage first, then fetch if missing.

```

async function resolveSecureKey(customerId: string): Promise<string> {
    const cached = sessionStorage.getItem('customerSecureKey');
    if (cached) return cached;
    const key = await fetchSecureKey(customerId);
    if (key) sessionStorage.setItem('customerSecureKey', key);
    return key;
}

```

fetchCustomerCart() — ligne 48

Description (commentaire d'origine) : Fetch the most recent cart for a given customer from PrestaShop. Returns the cart ID and its product rows, or null if no cart exists.

```

export async function fetchCustomerCart(customerId: string): Promise<RemoteCartResult | null> {
    try {
        const res = await api.get(
            `/carts?display=full&filter[id_customer]=[${customerId}]&sort=[id_DESC]&limit=1`
        );
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        const cartEl = doc.querySelector('cart');
        if (!cartEl) return null;
    }
}

```

```

const cartId = cartEl.querySelector('id')?.textContent?.trim();
if (!cartId) return null;

const items: RemoteCartResult['items'] = [];
cartEl.querySelectorAll('cart_row').forEach(row => {
  const productId = row.querySelector('id_product')?.textContent?.trim();
  const quantity = parseInt(row.querySelector('quantity')?.textContent ?? '0', 10);
  const attrId = row.querySelector('id_product_attribute')?.textContent?.trim();
  if (productId && quantity > 0) {
    items.push({
      productId,
      quantity,
      attributeId: attrId && attrId !== '0' ? attrId : undefined,
    });
  }
});

console.log('[fetchCustomerCart] Found remote cart:', { cartId, itemCount: items.length });
return { cartId, items };
} catch (err) {
  console.error('[fetchCustomerCart] Error:', err);
  return null;
}
}

```

syncRemoteCart() — ligne 94

Description (commentaire d'origine) : Create or update a remote PrestaShop cart. Returns the cart ID on success, or null on failure.

```

export async function syncRemoteCart(params: SyncParams): Promise<string | null> {
  const { customerId, items, existingCartId } = params;

  if (items.length === 0) {
    console.log('[syncRemoteCart] Cart is empty, skipping sync.');
    return existingCartId ?? null;
  }

  try {
    // Resolve all required fields
    const secureKey = await resolveSecureKey(customerId);
    const addressId = await resolveAddressId(customerId);

    // Validate required fields
    if (!secureKey) {
      console.error('[syncRemoteCart] Missing secure_key – aborting sync.');
      return null;
    }

    const carrierId = '1'; // Default: Click and collect
    const currencyId = '1';
    const langId = '1';
    const shopId = '1';
    const shopGroupId = '1';

    // Debug log before API call
    console.log('[syncRemoteCart] Debug payload:', {
      customerId,
      secureKey,
      addressId,
      currencyId,
      langId,
      carrierId,
      shopId,
      shopGroupId,
      cartRowsCount: items.length,
      existingCartId: existingCartId ?? 'none (will create)',
    });
  }
}

```

```

// Build cart_rows XML
const rowsXml = items.map(item => `
  <cart_row>
    <id_product><![CDATA[${item.id}]]></id_product>
    <id_product_attribute><![CDATA[${item.attributeId ?? '0'}]]></id_product_attribute>
    <id_address_delivery><![CDATA[${addressId}]]></id_address_delivery>
    <id_customization><![CDATA[0]]></id_customization>
    <quantity><![CDATA[${item.qty}]]></quantity>
  </cart_row>`).join('');

// Build full cart XML
const cartIdTag = existingCartId
? `<id><![CDATA[${existingCartId}]]></id>`
: '';

const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
<cart>
  ${cartIdTag}
  <id_shop><![CDATA[${shopId}]]></id_shop>
  <id_shop_group><![CDATA[${shopGroupId}]]></id_shop_group>
  <id_currency><![CDATA[${currencyId}]]></id_currency>
  <id_lang><![CDATA[${langId}]]></id_lang>
  <id_customer><![CDATA[${customerId}]]></id_customer>
  <id_address_delivery><![CDATA[${addressId}]]></id_address_delivery>
  <id_address_invoice><![CDATA[${addressId}]]></id_address_invoice>
  <id_carrier><![CDATA[${carrierId}]]></id_carrier>
  <secure_key><![CDATA[${secureKey}]]></secure_key>
  <id_guest><![CDATA[0]]></id_guest>
  <recyclable><![CDATA[0]]></recyclable>
  <gift><![CDATA[0]]></gift>
  <mobile_theme><![CDATA[0]]></mobile_theme>
  <allow_seperated_package><![CDATA[0]]></allow_seperated_package>
  <associations>
    <cart_rows>${rowsXml}</cart_rows>
  </associations>
</cart>
</prestashop>`;

let res;
if (existingCartId) {
  res = await api.put(`/carts/${existingCartId}`, xml);
} else {
  res = await api.post('/carts', xml);
}

const doc = new DOMParser().parseFromString(res.data, 'text/xml');
const newCartId = doc.querySelector('cart > id')?.textContent?.trim();
if (!newCartId) {
  console.error('[syncRemoteCart] No cart ID in response.');
```

```

  return null;
}
```

```

  console.log('[syncRemoteCart] Success – cart ID:', newCartId);
  return newCartId;
} catch (err) {
  console.error('[syncRemoteCart] Error syncing cart:', err);
  return null;
}
```

```

}
```

Fichier : src/services/orderService.ts

Lignes : 411 • Fonctions détectées : 13

isTransitionAllowed() — ligne 49

Description (commentaire d'origine) : ■ Vérifier si une transition est autorisée

```
export function isTransitionAllowed(oldState: number, newState: number): boolean {
  // Règle 1: Le panier (1) peut devenir payé (2) ou annulé (6)
  if (oldState === 1 && (newState === 2 || newState === 6)) {
    return true;
  }

  // Règle 2: Payé (2) peut devenir annulé (6)
  if (oldState === 2 && newState === 6) {
    return true;
  }

  // Règle 3: Annulé (6) peut redevenir payé (2) - cas rare mais possible
  if (oldState === 6 && newState === 2) {
    return true;
  }

  // Règle 4: Même état → pas de changement
  if (oldState === newState) {
    return false;
  }

  // Règle 5: TOUT VERS "dans le panier" (1) est INTERDIT
  if (newState === 1) {
    return false;
  }

  // Autres transitions non autorisées
  return false;
}
```

fetchCustomerName() — ligne 82

Description (commentaire d'origine) : =====
PRESTASHOP ORDERS =====

```
async function fetchCustomerName(customerId: string): Promise<string> {
  try {
    const res = await api.get(`/customers/${customerId}?display=[firstname,lastname]`);
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    const first = doc.querySelector('firstname')?.textContent?.trim() ?? '';
    const last = doc.querySelector('lastname')?.textContent?.trim() ?? '';
    return `${first} ${last}`.trim() || `Client #${customerId}`;
  } catch {
    return `Client #${customerId}`;
  }
}
```

parseOrdersXml() — ligne 96

Fonction : `parse orders xml` — voir le code ci-dessous.

```
function parseOrdersXml(xmlString: string): (Omit<PSOrder, 'customerName'> & { cartId?: string })
```

fetchPSOrders() — ligne 125

Fonction : `fetch psorders` — voir le code ci-dessous.

```
export async function fetchPSOrders(): Promise<PSOrder[]> {
  try {
    // 1■■ Récupérer toutes les commandes
    const ordersRes = await api.get('/orders?display=full');
```

```

const orders = parseOrdersXml(ordersRes.data);

// 2■■■ Extraire les IDs des paniers déjà transformés en commandes
const cartIdsAlreadyOrdered = new Set(
  orders
    .map(o => o.cartId)      // Il faut récupérer id_cart depuis la commande
    .filter(id => id)        // Éliminer les vides
);

// 3■■■ Récupérer les paniers
const cartsRes = await api.get('/carts?display=full');
const allCarts = await parseCartsXml(cartsRes.data);

// 4■■■ ■ FILTERER : garder uniquement les paniers NON transformés en commande
const pendingCarts = allCarts.filter(
  cart => !cartIdsAlreadyOrdered.has(cart.id)
);

// 5■■■ Fusionner commandes + paniers en attente
const allItems = [...orders, ...pendingCarts];

// 6■■■ Enrichir avec noms clients
const enriched = await Promise.all(
  allItems.map(async (item) => ({
    ...item,
    customerName: await fetchCustomerName(item.customerId),
  })))
);

return enriched;
} catch {
  return [];
}
}

```

fetchProductPrice() — ligne 168

Fonction : `fetch product price` — voir le code ci-dessous.

```

async function fetchProductPrice(productId: string): Promise<number> {
  if (productPriceCache.has(productId)) {
    return productPriceCache.get(productId)!;
  }

  try {
    const res = await api.get(`/products/${productId}?display=full`);
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');

    // PrestaShop expose price HT + price_ttc selon la config.
    // Préfère price_ttc, sinon price (HT).
    const priceTtc = doc.querySelector('price_tax_incl')?.textContent?.trim();
    const priceHt = doc.querySelector('price')?.textContent?.trim();

    const price = parseFloat(priceTtc ?? priceHt ?? '0') || 0;
    productPriceCache.set(productId, price);
    return price;
  } catch {
    console.warn(`Impossible de récupérer le prix du produit ${productId}`);
    return 0;
  }
}

```

fetchCombinationPriceImpact() — ligne 191

Fonction : `fetch combination price impact` — voir le code ci-dessous.

```

async function fetchCombinationPriceImpact(comboId: string): Promise<number> {
  if (combinationPriceCache.has(comboId)) {

```

```

    return combinationPriceCache.get(comboId)!!;
  }
  try {
    const res = await api.get(`/combinations/${comboId}?display=[price]`);
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    const impact = parseFloat(doc.querySelector('price')?.textContent ?? '0') || 0;
    combinationPriceCache.set(comboId, impact);
    return impact;
  } catch {
    return 0;
  }
}

```

parseCartsXml() — ligne 206

Fonction : `parse carts xml` — voir le code ci-dessous.

```

async function parseCartsXml(
  xmlString: string
): Promise<Omit<PSOrder, 'customerName'>[]> {
  const doc = new DOMParser().parseFromString(xmlString, 'text/xml');
  const cartEls = Array.from(doc.querySelectorAll('cart'));

  const carts = await Promise.all(
    cartEls.map(async (el) => {
      const id = el.querySelector('id')?.textContent?.trim() ?? '';
      if (!id) return null;

      const customerId = el.querySelector('id_customer')?.textContent?.trim() ?? '';
      const date = el.querySelector('date_add')?.textContent?.trim() ?? '';

      // On garantit ici que ce sont des strings (jamais undefined)
      const id_address_delivery = el.querySelector('id_address_delivery')?.textContent?.trim() ?? '0';
      const id_address_invoice = el.querySelector('id_address_invoice')?.textContent?.trim() ?? '0';
      const id_carrier = el.querySelector('id_carrier')?.textContent?.trim() ?? '0';
      const id_currency = el.querySelector('id_currency')?.textContent?.trim() ?? '1';

      const rows = Array.from(
        el.querySelectorAll('associations cart_rows cart_row, cart_rows cart_row, cart_row')
      );

      const cartRows: CartRow[] = rows
        .map(row => ({
          productId: row.querySelector('id_product')?.textContent?.trim() ?? '',
          combinationId: row.querySelector('id_product_attribute')?.textContent?.trim() ?? '0',
          quantity: parseInt(row.querySelector('quantity')?.textContent?.trim() ?? '0', 10),
        }))
        .filter(r => r.productId && r.quantity > 0);

      const rowTotals = await Promise.all(
        cartRows.map(async (row) => {
          if (!row.productId || row.quantity === 0) return 0;
          const basePrice = await fetchProductPrice(row.productId);
          const priceImpact = row.combinationId !== '0' ? await fetchCombinationPriceImpact(row.combinationId) : 0;
          return (basePrice + priceImpact) * row.quantity;
        })
      );

      const totalPaid = rowTotals.reduce((sum, v) => sum + v, 0);

      // On retourne l'objet directement
      return {
        id,
        reference: `PANIER-${id}`,
        customerId,
        totalPaid,
        date,
        currentState: 1,
        id_address_delivery,

```


Fonction : `update psorder status` — voir le code ci-dessous.

```
export async function updatePSOrderStatus(orderId: string, stateId: number): Promise<boolean> {
  // ■ Vérification supplémentaire avant envoi à l'API
  // On ne devrait jamais envoyer une transition vers panier (1)
  // car c'est interdit par isTransitionAllowed, mais sécurité supplémentaire

  try {
    const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <order_history>
    <id_order><![CDATA[${orderId}]]></id_order>
    <id_order_state><![CDATA[${stateId}]]></id_order_state>
    <id_employee><![CDATA[1]]></id_employee>
  </order_history>
</prestashop>`;

    await api.post('/order_histories', xml);
    return true;
  } catch (error) {
    console.error(`Erreur updatePSOrderStatus pour ${orderId} -> ${stateId}:`, error);
    return false;
  }
}
```

deletePSCart() — ligne 348

Description (commentaire d'origine) : =====
SUPPRESSION DES PANIERS =====

```
export async function deletePSCart(cartId: string): Promise<boolean> {
  try {
    await api.delete(`/carts/${cartId}`);
    return true;
  } catch (error) {
    console.error(`Erreur suppression panier ${cartId}:`, error);
    return false;
  }
}
```

deleteZombieCarts() — ligne 358

Fonction : `delete zombie carts` — voir le code ci-dessous.

```
export async function deleteZombieCarts(orders: PSOrder[]): Promise<{ deleted: number; failed: number }> {
  const zombies = orders.filter(o => o.currentState === 1 && o.totalPaid === 0);
  let deleted = 0;
  let failed = 0;
  for (const cart of zombies) {
    const ok = await deletePSCart(cart.id);
    if (ok) deleted++; else failed++;
  }
  return { deleted, failed };
}
```

getOrdersStats() — ligne 373

Description (commentaire d'origine) : =====
FONCTIONS UTILITAIRES POUR LE STATISTIQUES
=====

```
export function getOrdersStats(orders: PSOrder[]) {
  const paniers = orders.filter(o => o.currentState === 1);
  const payees = orders.filter(o => o.currentState === 2);
  const annulees = orders.filter(o => o.currentState === 6);
}
```

```

const montantTotalPaye = payees.reduce((sum, o) => sum + o.totalPaid, 0);
const montantTotalPaniers = paniers.reduce((sum, o) => sum + o.totalPaid, 0);

return {
  total: orders.length,
  paniers: paniers.length,
  payees: payees.length,
  annulees: annulees.length,
  montantTotalPaye,
  montantTotalPaniers,
};
}

```

fetchOrderRows() — ligne 395

Description (commentaire d'origine) : ===== LIGNES
D'UNE COMMANDE EXISTANTE =====

```

export async function fetchOrderRows(orderId: string): Promise<CartRow[]> {
  try {
    const res = await api.get(`/orders/${orderId}?display=full`);
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    const details = Array.from(doc.querySelectorAll('order_detail, order_row'));
    return details
      .map(el => ({
        productId: el.querySelector('product_id')?.textContent?.trim() ?? '',
        combinationId: el.querySelector('product_attribute_id')?.textContent?.trim() ?? '0',
        quantity: parseInt(el.querySelector('product_quantity')?.textContent?.trim() ?? '0', 10),
      }))
      .filter(r => r.productId && r.quantity > 0);
  } catch {
    return [];
  }
}

```

Fichier : src/services/dashboardApi.ts

Lignes : 58 • Fonctions détectées : 2

parseOrdersXml() — ligne 25

Description (commentaire d'origine) : Parse la réponse XML de PrestaShop Structure typique :
<prestashop><orders><order>...</order>...</orders></prestashop>

```

function parseOrdersXml(xmlString: string): OrderFromApi[] {
  const parser = new DOMParser();
  const xmlDoc = parser.parseFromString(xmlString, 'text/xml');
  const orders: OrderFromApi[] = [];

  const orderNodes = xmlDoc.getElementsByTagName('order');
  for (let i = 0; i < orderNodes.length; i++) {
    const order = orderNodes[i];
    const id = order.getElementsByTagName('id')[0]?.textContent || '';
    const reference = order.getElementsByTagName('reference')[0]?.textContent || '';
    const total_paid_tax_incl = order.getElementsByTagName('total_paid_tax_incl')[0]?.textContent || '0';
    const date_add = order.getElementsByTagName('date_add')[0]?.textContent || '';
    const current_state = order.getElementsByTagName('current_state')[0]?.textContent || '';

    if (id && reference) {
      orders.push({ id, reference, total_paid_tax_incl, date_add, current_state });
    }
  }
  return orders;
}

```

fetchOrders() — ligne 45

Fonction : `fetch orders` — voir le code ci-dessous.

```
export async function fetchOrders(dateFrom?: string, dateTo?: string): Promise<OrderFromApi[]> {
  try {
    // AUCUN paramètre pour tester la connectivité de base
    const response = await apiClient.get('/orders?display=[id,reference,total_paid_tax_incl,date_add,current_...
    console.log('■ Réponse API brute :', response.data);
    console.log('Réponse brute :', response.data); // log partiel pour éviter d'encombrer la console
    const orders = parseOrdersXml(response.data);
    return orders;
  } catch (error) {
    console.error('Erreur fetchOrders:', error);
    throw error;
  }
}
```

Services — Import & Validation

Fichier :

`src/services/csvImportService.ts`

Lignes : 370 • Fonctions détectées : 10

`cleanCell()` — ligne 62

Fonction : `clean cell` — voir le code ci-dessous.

```
function cleanCell(value: string): string {
  return value.trim().replace(/^["]|["]$/g, '');
}

export function parseCSV(content: string): CsvRow[] {
  const lines = content.split(/\r?\n/).filter((l) => l.trim() !== '');
  // Ignore la ligne d'en-tête (index 0)
  return lines
    .slice(1)
    .map((line) => {
      const cells = line.split(';');
      return {
        id: cleanCell(cells[COL.ID] ?? ''),
        active: cleanCell(cells[COL.ACTIVE] ?? ''),
        name: cleanCell(cells[COL.NAME] ?? ''),
        categories: cleanCell(cells[COL.CATEGORIES] ?? ''),
        price: cleanCell(cells[COL.PRICE] ?? ''),
        taxRulesId: cleanCell(cells[COL.TAX_RULES_ID] ?? ''),
        wholesalePrice: cleanCell(cells[COL.WHOLESALE_PRICE] ?? ''),
        reference: cleanCell(cells[COL.REFERENCE] ?? ''),
        weight: cleanCell(cells[COL.WEIGHT] ?? ''),
        quantity: cleanCell(cells[COL.QUANTITY] ?? ''),
        summary: cleanCell(cells[COL.SUMMARY] ?? ''),
        description: cleanCell(cells[COL.DESCRPTION] ?? ''),
        condition: cleanCell(cells[COL.CONDITION] ?? ''),
      } satisfies CsvRow;
    })
    .filter((row) => row.name !== '');
}
```

`parseCSV()` — ligne 66

Fonction : `parse csv` — voir le code ci-dessous.

```
export function parseCSV(content: string): CsvRow[] {
  const lines = content.split(/\r?\n/).filter((l) => l.trim() !== '');
  // Ignore la ligne d'en-tête (index 0)
  return lines
    .slice(1)
    .map((line) => {
      const cells = line.split(';');
      return {
        id: cleanCell(cells[COL.ID] ?? ''),
        active: cleanCell(cells[COL.ACTIVE] ?? ''),
        name: cleanCell(cells[COL.NAME] ?? ''),
        categories: cleanCell(cells[COL.CATEGORIES] ?? ''),
        price: cleanCell(cells[COL.PRICE] ?? ''),
        taxRulesId: cleanCell(cells[COL.TAX_RULES_ID] ?? ''),
        wholesalePrice: cleanCell(cells[COL.WHOLESALE_PRICE] ?? ''),
        reference: cleanCell(cells[COL.REFERENCE] ?? ''),
      }
    })
    .filter((row) => row.name !== '');
}
```

```

        weight: cleanCell(cells[COL.WEIGHT] ?? ''),
        quantity: cleanCell(cells[COL.QUANTITY] ?? ''),
        summary: cleanCell(cells[COL.SUMMARY] ?? ''),
        description: cleanCell(cells[COL.DESCRPTION] ?? ''),
        condition: cleanCell(cells[COL.CONDITION] ?? ''),
    } satisfies CsvRow;
  })
  .filter((row) => row.name !== '');
}

```

validateRow() — ligne 96

Description (commentaire d'origine) : ===== 3.
 PRÉ-VALIDATION (ligne par ligne + doublons) =====

```

export function validateRow(row: CsvRow, rowIndex: number): string[] {
  const errors: string[] = [];

  if (!row.name) {
    errors.push(`Le champ "Nom" est requis`);
  }

  const price = parseFloat(row.price);
  if (row.price === '' || isNaN(price)) {
    errors.push(`"Prix HT" invalide (valeur : "${row.price}")`);
  } else if (price < 0) {
    errors.push(`"Prix HT" ne peut pas être négatif (${row.price})`);
  }

  const wholesalePrice = parseFloat(row.wholesalePrice);
  if (row.wholesalePrice !== '' && !isNaN(wholesalePrice) && wholesalePrice < 0) {
    errors.push(`"Prix d'achat" ne peut pas être négatif (${row.wholesalePrice})`);
  }

  const quantity = parseInt(row.quantity, 10);
  if (row.quantity !== '' && !isNaN(quantity) && quantity < 0) {
    errors.push(`"Quantité" ne peut pas être négative (${row.quantity})`);
  }

  const taxRulesId = parseInt(row.taxRulesId, 10);
  if (!row.taxRulesId || Number.isNaN(taxRulesId) || taxRulesId <= 0) {
    errors.push(`"Tax rules ID" invalide (valeur : "${row.taxRulesId}")`);
  }

  return errors;
}

```

validateAllRows() — ligne 133

Description (commentaire d'origine) : Valide toutes les lignes du CSV en une seule passe. Détecte les doublons de référence et les catégories textuelles.

```

export function validateAllRows(rows: CsvRow[]): RowValidation[] {
  const refMap = new Map<string, number>(); // reference → première ligne (csvLine)
  const validations: RowValidation[] = [];

  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    const csvLine = i + 2; // +2 car header=1, index 0-based
    const errors = validateRow(row, i);
    const warnings: string[] = [];

    // ■■■ Doublon de référence dans le CSV ■■■
    if (row.reference) {
      const refKey = row.reference.trim().toLowerCase();
      if (refMap.has(refKey)) {
        errors.push(

```

```

        `Référence « ${row.reference} » déjà présente à la ligne ${refMap.get(refKey)}`
    );
} else {
    refMap.set(refKey, csvLine);
}
}

// ■■ Catégorie textuelle → warning (sera créée automatiquement) ■■
const firstCategory = row.categories.split(',')[0].trim();
if (firstCategory && isNaN(parseInt(firstCategory, 10))) {
    warnings.push(
        `La catégorie « ${firstCategory} » sera créée automatiquement`
    );
}

validations.push({ rowIndex: i, errors, warnings });
}

return validations;
}

```

mapRowToProduct () — ligne 172

Description (commentaire d'origine) : ===== 4.
 MAPPING CSV → Product (WebService XML) =====

```

export function mapRowToProduct(row: CsvRow, categoryId?: number): Partial<Product> {
    const firstCategory = row.categories.split(',')[0].trim();
    const parsedCategory = parseInt(firstCategory, 10);
    const idCategory = categoryId ?? (isNaN(parsedCategory) ? 2 : parsedCategory);
    const taxRulesId = parseInt(row.taxRulesId, 10);

    return {
        name: row.name,
        price: parseFloat(row.price) || 0,
        wholesale_price: parseFloat(row.wholesalePrice) || 0,
        reference: row.reference,
        ean13: '',
        description: row.description,
        description_short: row.summary,
        meta_title: row.name,
        active: row.active === '1',
        quantity: parseInt(row.quantity, 10) || 0,
        id_category_default: idCategory,
        id_tax_rules_group: Number.isNaN(taxRulesId) ? 0 : taxRulesId,
    };
}

```

sSlug () — ligne 205

Fonction : **sSlug** — voir le code ci-dessous.

```

function slug(name: string): string {
    return name
        .toLowerCase()
        .normalize('NFD').replace(/[\u0300-\u036f]/g, '')
        .replace(/^[^a-z0-9]+/g, '-')
        .replace(/^-|-$/g, '') || 'item';
}

```

resolveOrCreateCategory () — ligne 218

Description (commentaire d'origine) : Recherche une catégorie par nom. Si elle n'existe pas, la crée.
 Retourne l'id numérique de la catégorie.

```

export async function resolveOrCreateCategory(categoryName: string): Promise<number> {
    const key = categoryName.trim().toLowerCase();
    if (categoryCache.has(key)) return categoryCache.get(key)!;

    // Chercher la catégorie existante par nom
    try {
        const res = await categoryApi.get(
            `/categories?display=[id,name]&filter[name]=%[${categoryName}]%`
        );
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        const categories = doc.querySelectorAll('category');
        for (const cat of categories) {
            const nameEl = cat.querySelector('name language') ?? cat.querySelector('name');
            const idEl = cat.querySelector('id');
            if (nameEl?.textContent?.trim().toLowerCase() === key && idEl?.textContent) {
                const id = parseInt(idEl.textContent.trim(), 10);
                categoryCache.set(key, id);
                return id;
            }
        }
    } catch {
        // API error – on tente quand même la création
    }

    // Créer la catégorie
    const lr = slug(categoryName);
    const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <category>
    <active><![CDATA[1]]></active>
    <id_parent><![CDATA[2]]></id_parent>
    <name><language id="1"><![CDATA[${categoryName}]]></language></name>
    <description><language id="1"><![CDATA[]]></language></description>
    <link_rewrite><language id="1"><![CDATA[${lr}]]></language></link_rewrite>
    <meta_title><language id="1"><![CDATA[${categoryName}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[]]></language></meta_description>
  </category>
</prestashop>`;

    const createRes = await categoryApi.post('/categories', xml);
    const createDoc = new DOMParser().parseFromString(createRes.data, 'text/xml');
    const createdId = parseInt(
        createDoc.querySelector('id')?.textContent?.trim() || '2',
        10
    );
    categoryCache.set(key, createdId);
    return createdId;
}

```

clearCategoryCache() — ligne 269

Description (commentaire d'origine) : Vide le cache de catégories (utile pour un nouvel import)

```

export function clearCategoryCache(): void {
    categoryCache.clear();
}

```

importProducts() — ligne 276

Description (commentaire d'origine) : ===== 6.

IMPORT PRODUITS — utilise productService.create() (WebService XML /api/products)

=====

```

export async function importProducts(
    file: File,
    onProgress?: ProgressCallback
)

```

```

): Promise<ImportResult[]> {
  const content = await file.text();
  const rows = parseCSV(content);
  const results: ImportResult[] = [];
  const total = rows.length;

  // Reset le cache de catégories pour un import frais
  clearCategoryCache();

  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];

    const validationErrors = validateRow(row, i);
    if (validationErrors.length > 0) {
      results.push({
        rowIndex: i,
        productName: row.name || `Ligne ${i + 2}`,
        success: false,
        error: validationErrors.join(' | '),
      });
      onProgress?.(i + 1, total);
      continue;
    }

    // Résolution de la catégorie (création automatique si textuelle)
    let categoryId: number | undefined;
    const firstCategory = row.categories.split(',')[0].trim();
    if (firstCategory && isNaN(parseInt(firstCategory, 10))) {
      try {
        categoryId = await resolveOrCreateCategory(firstCategory);
      } catch (err: any) {
        results.push({
          rowIndex: i,
          productName: row.name,
          success: false,
          error: `Impossible de créer la catégorie « ${firstCategory} » : ${err.message}`,
        });
        onProgress?.(i + 1, total);
        continue;
      }
    }

    const payload = mapRowToProduct(row, categoryId);

    try {
      const created = await productService.create(payload);
      if (!created) throw new Error('Réponse vide du serveur');

      results.push({
        rowIndex: i,
        productName: row.name,
        success: true,
        productId: created.id,
      });
    } catch (err: any) {
      const msg =
        err.response?.data
          ? extractXmlError(err.response.data)
          : err.message || 'Erreur inconnue';
      results.push({
        rowIndex: i,
        productName: row.name,
        success: false,
        error: msg,
      });
    }

    onProgress?.(i + 1, total);
  }

  return results;
}

```

```
}
```

extractXmlError() — ligne 356

Description (commentaire d'origine) : ===== 7.
UTILITAIRE — extrait le message d'erreur PrestaShop depuis le XML
=====

```
function extractXmlError(xmlString: string): string {
  try {
    const parser = new DOMParser();
    const doc = parser.parseFromString(xmlString, 'text/xml');
    const msgEl = doc.querySelector('message');
    if (msgEl?.textContent) return msgEl.textContent.trim();
    const errorEl = doc.querySelector('error');
    if (errorEl?.textContent) return errorEl.textContent.trim();
  } catch {
    // ignore parse errors
  }
  return typeof xmlString === 'string' ? xmlString.slice(0, 120) : 'Erreur API';
}
```

Fichier : src/services/fichierImportService.ts

Lignes : 1336 • Fonctions détectées : 46

parseCsvLine() — ligne 32

Description (commentaire d'origine) : Parser CSV générique : gère les champs entre guillemets et les virgules internes

```
function parseCsvLine(line: string): string[] {
  const result: string[] = [];
  let current = '';
  let inQuotes = false;
  for (let i = 0; i < line.length; i++) {
    const ch = line[i];
    if (ch === '"') {
      if (inQuotes && line[i + 1] === '"') { current += '"'; i++; }
      else { inQuotes = !inQuotes; }
    } else if (ch === ',' && !inQuotes) {
      result.push(current.trim()); current = '';
    } else {
      current += ch;
    }
  }
  result.push(current.trim());
  return result;
}
```

parseCsvContent() — ligne 50

Fonction : `parse csv content` — voir le code ci-dessous.

```
function parseCsvContent(content: string): string[][] {
  return content
    .split(/\r?\n/)
    .filter((l) => l.trim() !== '')
    .map(parseCsvLine);
}
```

```
}
```

normalizeHeader() — ligne 57

Fonction : `normalize header` — voir le code ci-dessous.

```
function normalizeHeader(value: string): string {  
  return value.trim().toLowerCase();  
}
```

resolveFichier1Columns() — ligne 61

Fonction : `resolve fichier1columns` — voir le code ci-dessous.

```
function resolveFichier1Columns(header: string[]) {  
  const lower = header.map(normalizeHeader);  
  const hasDate = lower.includes('date_produit');  
  const fallback = hasDate  
    ? { date: 0, nom: 1, reference: 2, prixTtc: 3, taxe: 4, categorie: 5, prixAchat: 6 }  
    : { date: -1, nom: 0, reference: 1, prixTtc: 2, taxe: 3, categorie: 4, prixAchat: 5 };  
  
  const findIdx = (names: string[], fallbackIdx: number) => {  
    for (const name of names) {  
      const idx = lower.indexOf(name);  
      if (idx >= 0) return idx;  
    }  
    return fallbackIdx;  
  };  
  
  return {  
    dateIdx: findIdx(['date_produit', 'date produit', 'date', 'date_availability_produit'], fallback.date),  
    nomIdx: findIdx(['nom', 'name'], fallback.nom),  
    referenceIdx: findIdx(['reference', 'référence', 'ref'], fallback.reference),  
    prixTtcIdx: findIdx(['prix_ttc', 'prix ttc', 'price_ttc'], fallback.prixTtc),  
    taxeIdx: findIdx(['taxe', 'taux_tva', 'tva', 'tax'], fallback.taxe),  
    categorieIdx: findIdx(['categorie', 'catégorie', 'category'], fallback.categorie),  
    prixAchatIdx: findIdx(['prix_achat', 'prix achat', 'wholesale_price'], fallback.prixAchat),  
  };  
}
```

findIdx() — ligne 68

Fonction : `find idx` — voir le code ci-dessous.

```
const findIdx = (names: string[], fallbackIdx: number) => {  
  for (const name of names) {  
    const idx = lower.indexOf(name);  
    if (idx >= 0) return idx;  
  }  
  return fallbackIdx;  
}
```

parseFrenchNumber() — ligne 87

Fonction : `parse french number` — voir le code ci-dessous.

```
function parseFrenchNumber(s: string): number {  
  return parseFloat(s.replace(',', '.')) || 0;  
}
```

parseNumberStrict() — ligne 91

Fonction : `parse number strict` — voir le code ci-dessous.

```
function parseNumberStrict(raw: string): number | null {
  const cleaned = raw.trim().replace('%', '').replace(',', '.');
  if (!cleaned) return null;
  const value = Number(cleaned);
  return Number.isNaN(value) ? null : value;
}
```

`normalizeYear()` — ligne 98

Fonction : `normalize year` — voir le code ci-dessous.

```
function normalizeYear(year: number): number {
  if (year >= 100) return year;
  return year >= 70 ? 1900 + year : 2000 + year;
}
```

`buildDate()` — ligne 103

Fonction : `build date` — voir le code ci-dessous.

```
function buildDate(year: number, month: number, day: number, h = 0, m = 0, s = 0): Date | null {
  const y = normalizeYear(year);
  if (month < 1 || month > 12 || day < 1 || day > 31) return null;
  const date = new Date(y, month - 1, day, h, m, s);
  if (Number.isNaN(date.getTime())) return null;
  if (date.getFullYear() !== y || date.getMonth() !== month - 1 || date.getDate() !== day) return null;
  return date;
}
```

`parseDateFlexible()` — ligne 112

Fonction : `parse date flexible` — voir le code ci-dessous.

```
function parseDateFlexible(raw: string): Date | null {
  const value = raw.trim();
  if (!value) return null;

  // YYYY-MM-DD ou YYYY/MM/DD (ISO, non ambigu) — priorité maximale
  const ymd = value.match(/^([12]\d{3})[\/\.-](\d{1,2})[\/\.-](\d{1,2})(?:\s+(\d{1,2}))(?::(\d{1,2}))?(?::(\d{1,2}))?$/);
  if (ymd) {
    const [, y, mo, d, hh, mm, ss] = ymd;
    return buildDate(
      parseInt(y, 10), parseInt(mo, 10), parseInt(d, 10),
      parseInt(hh ?? '0', 10), parseInt(mm ?? '0', 10), parseInt(ss ?? '0', 10),
    );
  }

  // DD/MM/YYYY ou DD.MM.YYYY (format français — avant Date.parse pour éviter l'inversion MM/DD)
  const dmy = value.match(/^(\d{1,2})[\/\.-](\d{1,2})[\/\.-](\d{2,4})(?:\s+(\d{1,2}))(?::(\d{1,2}))?(?::(\d{1,2}))?$/);
  if (dmy) {
    const [, p1, p2, y, hh, mm, ss] = dmy;
    const a = parseInt(p1, 10);
    const b = parseInt(p2, 10);
    const year = parseInt(y, 10);
    // Si a > 12 → forcément le jour ; si b > 12 → forcément le mois (impossible, erreur)
    // Par défaut format français : a = jour, b = mois
    const day = a <= 31 ? a : b;
    const month = a <= 31 ? b : a;
    return buildDate(
      year, month, day,
      parseInt(hh ?? '0', 10), parseInt(mm ?? '0', 10), parseInt(ss ?? '0', 10),
    );
  }
}
```

```
// Dernier recours : laisser Date.parse gérer (ISO 8601 avec timezone, etc.)
const direct = Date.parse(value);
return Number.isNaN(direct) ? null : new Date(direct);
}
```

parseTaxRate() — ligne 148

Fonction : `parse tax rate` — voir le code ci-dessous.

```
function parseTaxRate(s: string): number {
  return parseFrenchNumber(s.replace('%', '')) / 100;
}
```

ttcToHt() — ligne 152

Fonction : `ttc to ht` — voir le code ci-dessous.

```
function ttcToHt(ttc: number, taxRate: number): number {
  if (taxRate <= 0) return ttc;
  return ttc / (1 + taxRate);
}
```

slugify() — ligne 157

Fonction : `slugify` — voir le code ci-dessous.

```
function slugify(name: string): string {
  return name.toLowerCase()
    .normalize('NFD').replace(/[\p{D}]/g, '')
    .replace(/[\^a-z0-9]+/g, '-')
    .replace(/^-|-$/g, '') || 'item';
}
```

roundMoney() — ligne 163

Fonction : `round money` — voir le code ci-dessous.

```
function roundMoney(value: number): number {
  return Math.round((value + Number.EPSILON) * 1_000_000) / 1_000_000;
}
```

buildCombinationReference() — ligne 167

Fonction : `build combination reference` — voir le code ci-dessous.

```
function buildCombinationReference(reference: string, variant: string): string {
  const suffix = slugify(variant);
  return suffix ? `${reference}-${suffix}` : reference;
}
```

extractXmlError() — ligne 172

Fonction : `extract xml error` — voir le code ci-dessous.

```
function extractXmlError(xml: string): string {
  try {
    const doc = new DOMParser().parseFromString(xml, 'text/xml');
    return doc.querySelector('message')?.textContent?.trim()
      ?? doc.querySelector('error')?.textContent?.trim()
      ?? xml.slice(0, 100);
  } catch { return 'Erreur API'; }
}
```

getCreatedId() — ligne 181

Fonction : `get created id` — voir le code ci-dessous.

```
function getCreatedId(xml: string): string | null {
  try {
    return new DOMParser().parseFromString(xml, 'text/xml')
      .querySelector('id')?.textContent?.trim() ?? null;
  } catch { return null; }
}
```

postXml() — ligne 188

Fonction : `post xml` — voir le code ci-dessous.

```
async function postXml(endpoint: string, xml: string): Promise<string> {
  const res = await api.post(endpoint, xml);
  return getCreatedId(res.data) ?? '?';
}
```

fetchProductByReference() — ligne 193

Fonction : `fetch product by reference` — voir le code ci-dessous.

```
async function fetchProductByReference(reference: string): Promise<{
  id: string;
  name: string;
  priceHt: number;
  taxRulesGroupId: number;
} | null> {
  try {
    const res = await api.get(
      `/products?display=[id,name,reference,price,id_tax_rules_group]&filter[reference]=[${reference}]`
    );
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    const el = doc.querySelector('product');
    if (!el) return null;
    const id = el.querySelector('id')?.textContent?.trim() ?? '';
    const name = el.querySelector('name > language')?.textContent?.trim()
      ?? el.querySelector('name')?.textContent?.trim()
      ?? reference;
    const priceHt = parseFloat(el.querySelector('price')?.textContent ?? '0');
    const taxRulesGroupId = parseInt(el.querySelector('id_tax_rules_group')?.textContent ?? '0', 10);
    if (!id) return null;
    return { id, name, priceHt, taxRulesGroupId };
  } catch (err) {
    console.error('fetchProductByReference failed', { reference, err });
    return null;
  }
}
```

fetchCombinationByReference() — ligne 220

Fonction : `fetch combination by reference` — voir le code ci-dessous.

```
async function fetchCombinationByReference(reference: string): Promise<{
  id: string;
  productId: string;
  priceImpact: number;
} | null> {
  try {
    const res = await api.get(
      `/combinations?display=[id,id_product,reference,price]&filter[reference]=[${reference}]`
    );
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
```

```

    const el = doc.querySelector('combination');
    if (!el) return null;
    const id = el.querySelector('id')?.textContent?.trim() ?? '';
    const productId = el.querySelector('id_product')?.textContent?.trim() ?? '';
    const priceImpact = parseFloat(el.querySelector('price')?.textContent ?? '0');
    if (!id) return null;
    return { id, productId, priceImpact };
  } catch (err) {
    console.error('fetchCombinationByReference failed', { reference, err });
    return null;
  }
}

```

findOrCreateCategory() — ligne 271

Description (commentaire d'origine) : ===== FICHER

1 — Produits (date_produit,nom,reference,prix_ttc,Taxe,categorie,prix_achat)
=====

```

async function findOrCreateCategory(name: string): Promise<string> {
  if (categoryCache.has(name)) return categoryCache.get(name)!;

  // Chargement de toutes les catégories au premier appel
  if (categoryCache.size === 0) {
    try {
      const res = await api.get('/categories?display=full');
      const doc = new DOMParser().parseFromString(res.data, 'text/xml');
      doc.querySelectorAll('category').forEach((c) => {
        const id = c.querySelector(':scope > id')?.textContent?.trim();
        const nameEl = c.querySelector('name language');
        const n = nameEl?.textContent?.trim() ?? c.querySelector('name')?.textContent?.trim();
        if (id && n) categoryCache.set(n, id);
      });
    } catch { /* ignore */ }
  }

  if (categoryCache.has(name)) return categoryCache.get(name)!;

  // Créer la catégorie
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <category>
    <active><![CDATA[1]]></active>
    <id_parent><![CDATA[2]]></id_parent>
    <name><language id="1"><![CDATA[${name}]]></language></name>
    <link_rewrite><language id="1"><![CDATA[${slugify(name)}]]></language></link_rewrite>
    <description><language id="1"><![CDATA[]]></language></description>
    <meta_title><language id="1"><![CDATA[${name}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[]]></language></meta_description>
  </category>
</prestashop>`;
  const id = await postXml('/categories', xml);
  categoryCache.set(name, id);
  return id;
}

```

importFichier1() — ligne 309

Fonction : `import fichier1` — voir le code ci-dessous.

```

export async function importFichier1(
  file: File,
  onProgress?: FichierProgressCallback,
): Promise<FichierImportResult[]> {
  categoryCache = new Map();
  productRefCache = new Map();
}

```

```

taxRateCache = new Map();

const lines = parseCsvContent(await file.text());
const header = lines[0] ?? [];
const cols = resolveFichier1Columns(header);
const rows = lines.slice(1)
    .map((r, idx) => ({ row: r, csvLine: idx + 2 })))
    .filter(({ row }) => (row[cols.nomIdx] ?? '').trim() !== '');
const results: FichierImportResult[] = [];

for (let i = 0; i < rows.length; i++) {
    const { row, csvLine } = rows[i];
    const nom = row[cols.nomIdx] ?? '';
    const reference = row[cols.referenceIdx] ?? '';
    const dateValue = cols.dateIdx >= 0 ? (row[cols.dateIdx] ?? '') : '';
    const prix_ttc_str = row[cols.prixTtcIdx] ?? '';
    const taxe_str = row[cols.taxeIdx] ?? '';
    const categorie = row[cols.categorieIdx] ?? '';
    const prix_achat_str = row[cols.prixAchatIdx] ?? '';
    const label = `${nom} (${reference})`;
    onProgress?.(i, rows.length, label);
    try {
        const taxRate = parseTaxRate(taxe_str ?? '0%');
        const ttc = parseFrenchNumber(prix_ttc_str ?? '0');
        const ht = ttcToHt(ttc, taxRate);
        const wholesalePrice = parseFrenchNumber(prix_achat_str ?? '0');
        const catId = await findOrCreateCategory(categorie ?? 'Général');
        const taxGroupId = await ensureTaxRulesGroupIdByRate(taxRate);
        if (!taxGroupId) {
            console.error('Tax group not found for rate', {
                label,
                rate: taxRate,
                taxLabel: taxe_str,
            });
            throw new Error(`Aucun groupe de taxe pour le taux ${taxe_str ?? '0%'}`);
        }
        taxRateCache.set(reference, taxRate);

        const parsedDate = dateValue ? parseDateFlexible(dateValue) : null;
        const dateIso = parsedDate
            ? `${parsedDate.getFullYear()}-${String(parsedDate.getMonth() + 1).padStart(2, '0')}-${String(parsedDate.getDate().padStart(2, '0'))}`
            : '';
        const dateTag = dateIso ? `<available_date><![CDATA[${dateIso}]]></available_date>` : '';

        const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
<product>
<active><![CDATA[1]]></active>
<state><![CDATA[1]]></state>
<id_category_default><![CDATA[${catId}]]></id_category_default>
<id_tax_rules_group><![CDATA[${taxGroupId}]]></id_tax_rules_group>
<type><![CDATA[simple]]></type>
<reference><![CDATA[${reference}]]></reference>
<price><![CDATA[${ht.toFixed(6)}]]></price>
<wholesale_price><![CDATA[${wholesalePrice.toFixed(6)}]]></wholesale_price>
<name><language id="1"><![CDATA[${nom}]]></language></name>
<link_rewrite><language id="1"><![CDATA[${slugify(nom)}]]></language></link_rewrite>
<description><language id="1"><![CDATA[]]></language></description>
<description_short><language id="1"><![CDATA[]]></language></description_short>
<meta_title><language id="1"><![CDATA[${nom}]]></language></meta_title>
${dateTag}
<associations>
<categories><category><id><![CDATA[${catId}]]></id></category></categories>
</associations>
</product>
</prestashop>`;
        const id = await postXml('/products', xml);
        productRefCache.set(reference, id);
        results.push({ label, success: true, id, lineNumber: csvLine });
    } catch (err: any) {
        console.error('Import fichier1 failed', {

```

```

        label,
        error: err,
        response: err?.response?.data,
    });
    results.push({ label, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message, lineNumber: csvLine });
    }
    onProgress?.(i + 1, rows.length, label);
}
return results;
}

```

getProductIdByRef() — ligne 402

Description (commentaire d'origine) : ===== FICHIER

2 — Combinaisons & Stock (reference,specificité,karazany,stock_initial,prix_vente_ttc)
=====

```

async function getProductIdByRef(reference: string): Promise<string | null> {
    if (productRefCache.has(reference)) return productRefCache.get(reference)!;
    try {
        const res = await api.get(`/products?display=[id,reference]&filter[reference]=[${reference}]`);
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        const id = doc.querySelector('product > id')?.textContent?.trim();
        if (id) { productRefCache.set(reference, id); return id; }
    } catch { /* ignore */ }
    return null;
}

```

getStockAvailableId() — ligne 413

Fonction : `get stock available id` — voir le code ci-dessous.

```

async function getStockAvailableId(productId: string, combinationId = '0'): Promise<string | null> {
    try {
        const res = await api.get(
            `/stock_availlables?display=[id,id_product,id_product_attribute]&filter[id_product]=[${productId}]&filter[
        );
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        return doc.querySelector('stock_available > id')?.textContent?.trim() ?? null;
    } catch { return null; }
}

```

setStock() — ligne 423

Fonction : `set stock` — voir le code ci-dessous.

```

async function setStock(stockId: string, productId: string, combinationId: string, qty: number): Promise<void> {
    const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
    <stock_available>
        <id><![CDATA[${stockId}]]></id>
        <id_product><![CDATA[${productId}]]></id_product>
        <id_product_attribute><![CDATA[${combinationId}]]></id_product_attribute>
        <id_shop><![CDATA[1]]></id_shop>
        <id_shop_group><![CDATA[0]]></id_shop_group>
        <quantity><![CDATA[${qty}]]></quantity>
        <depends_on_stock><![CDATA[0]]></depends_on_stock>
        <out_of_stock><![CDATA[0]]></out_of_stock>
    </stock_available>
</prestashop>`;
    await api.put(`/stock_availlables/${stockId}`, xml);
}

```

getOrCreateOption() — ligne 440

Fonction : `get or create option` — voir le code ci-dessous.

```
async function getOrCreateOption(name: string): Promise<string> {
  const key = name.toLowerCase();
  if (optionCache.has(key)) return optionCache.get(key)!;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <product_option>
    <name><language id="1"><![CDATA[${name}]]></language></name>
    <public_name><language id="1"><![CDATA[${name}]]></language></public_name>
    <group_type><![CDATA[select]]></group_type>
    <is_color_group><![CDATA[0]]></is_color_group>
    <position><![CDATA[0]]></position>
  </product_option>
</prestashop>`;
  const id = await postXml('/product_options', xml);
  optionCache.set(key, id);
  return id;
}
```

getOrCreateOptionValue() — ligne 458

Fonction : `get or create option value` — voir le code ci-dessous.

```
async function getOrCreateOptionValue(optionId: string, valueName: string): Promise<string> {
  const key = `${optionId}:${valueName.toLowerCase()}`;
  if (optionValueCache.has(key)) return optionValueCache.get(key)!;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <product_option_value>
    <id_attribute_group><![CDATA[${optionId}]]></id_attribute_group>
    <name><language id="1"><![CDATA[${valueName}]]></language></name>
    <position><![CDATA[0]]></position>
  </product_option_value>
</prestashop>`;
  const id = await postXml('/product_option_values', xml);
  optionValueCache.set(key, id);
  return id;
}
```

importFichier2() — ligne 474

Fonction : `import fichier2` — voir le code ci-dessous.

```
export async function importFichier2(
  file: File,
  onProgress?: FichierProgressCallback,
): Promise<FichierImportResult[]> {
  optionCache = new Map();
  optionValueCache = new Map();

  const lines = parseCsvContent(await file.text());
  const rows = lines.slice(1)
    .map((r, idx) => ({ row: r, csvLine: idx + 2 }))
    .filter(({ row }) => row[0]);
  const results: FichierImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {
    const { row, csvLine } = rows[i];
    const [reference, specificite, karazany, stock_str, prix_ttc_str] = row;
    const label = `${reference}${karazany ? ' - ' + karazany : ''}`;
    onProgress?.(i, rows.length, label);

    try {
      const productId = await getProductIdByRef(reference);
```

```

        if (!productId) throw new Error('Produit "${reference}" introuvable');

        const qty = parseInt(stock_str ?? '0', 10) || 0;
        const hasVariant = specificite && karazany;

        if (hasVariant) {
            // Créer la combinaison
            const taxRate = taxRateCache.get(reference) ?? 0;
            const basePriceRes = await api.get(`/products/${productId}?display=[price]`);
            const baseDoc = new DOMParser().parseFromString(basePriceRes.data, 'text/xml');
            const baseHt = parseFloat(baseDoc.querySelector('price')?.textContent ?? '0');
            const variantTtc = parseFloat(prix_ttc_str ?? '0');
            const variantHt = variantTtc > 0 ? ttcToHt(variantTtc, taxRate) : baseHt;
            const priceImpact = (variantHt - baseHt).toFixed(6);
            const combinationRef = buildCombinationReference(reference, karazany);

            const optionId = await getOrCreateOption(specificite);
            const valId = await getOrCreateOptionValue(optionId, karazany);

            const combXml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <combination>
    <id_product><![CDATA[${productId}]]></id_product>
    <reference><![CDATA[${combinationRef}]]></reference>
    <price><![CDATA[${priceImpact}]]></price>
    <minimal_quantity><![CDATA[1]]></minimal_quantity>
    <default_on><![CDATA[0]]></default_on>
    <associations>
      <product_option_values>
        <product_option_value><id><![CDATA[${valId}]]></id></product_option_value>
      </product_option_values>
    </associations>
  </combination>
</prestashop>`;

            const combId = await postXml('/combinations', combXml);
            // Mettre à jour le stock de la combinaison
            const stockId = await getStockAvailableId(productId, combId);
            if (stockId) await setStock(stockId, productId, combId, qty);
            results.push({ label, success: true, id: combId, lineNumber: csvLine });
        } else {
            // Pas de variante : mettre le stock du produit de base
            const stockId = await getStockAvailableId(productId, '0');
            if (stockId) await setStock(stockId, productId, '0', qty);
            results.push({ label, success: true, lineNumber: csvLine });
        }
    } catch (err: any) {
        console.error('Import fichier2 failed', {
            label,
            error: err,
            response: err?.response?.data,
        });
        results.push({ label, success: false,
            error: err.response?.data ? extractXmlError(err.response.data) : err.message, lineNumber: csvLine });
    }
    onProgress?.(i + 1, rows.length, label);
}
return results;
}

```

parseAchat() — ligne 558

Description (commentaire d'origine) : ===== FICHER

3 — Clients & Commandes (date,nom,email,pwd,adresse,achat,etat)
=====

```

function parseAchat(raw: string): Array<{ reference: string; qty: number; variant: string }> {
    // raw après parse CSV: [("T_01";3;"ngoza")] ou [("T_01";2;"kely"),("M_03";1;"")]
    const cleaned = raw.trim();
    if (!cleaned.startsWith('[')) return [];
}

```

```

const inner = cleaned.slice(1, -1); // retire [ et ]
const itemStrings = inner.split('),( ');

return itemStrings.map((s) => {
  const stripped = s.replace(/^(\\|\\)$/, '');
  const parts = stripped.split(');');
  return {
    reference: parts[0]?.replace(/"/g, '').trim() ?? '',
    qty:       parseInt(parts[1]?.trim() ?? '1', 10) || 1,
    variant:   parts[2]?.replace(/"/g, '').trim() ?? '',
  };
}).filter((it) => it.reference);
}

```

splitCustomerName() — ligne 593

Fonction : `split customer name` — voir le code ci-dessous.

```
function splitCustomerName(nom: string): { firstname: string; lastname: string }
```

createCustomer() — ligne 600

Fonction : `create customer` — voir le code ci-dessous.

```

async function createCustomer(nom: string, email: string, pwd: string): Promise<ImportCustomer | null> {
  const { firstname, lastname } = splitCustomerName(nom);
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <customer>
    <active><![CDATA[1]]></active>
    <id_gender><![CDATA[0]]></id_gender>
    <email><![CDATA[${email}]]></email>
    <passwd><![CDATA[${pwd}]]></passwd>
    <lastname><![CDATA[${lastname}]]></lastname>
    <firstname><![CDATA[${firstname}]]></firstname>
    <newsletter><![CDATA[0]]></newsletter>
    <optin><![CDATA[0]]></optin>
    <id_default_group><![CDATA[3]]></id_default_group>
  </customer>
</prestashop>`;
  try {
    const id = await postXml('/customers', xml);
    if (!id || id === '?') return null;
    return { id, firstname, lastname, email };
  } catch {
    return null;
  }
}

```

resolveCustomer() — ligne 625

Fonction : `resolve customer` — voir le code ci-dessous.

```

async function resolveCustomer(nom: string, email: string, pwd: string): Promise<ImportCustomer | null> {
  const existing = await findCustomerByEmail(email);
  if (existing) return existing;
  return await createCustomer(nom, email, pwd);
}

```

ensureAddressForCustomer() — ligne 631

Fonction : `ensure address for customer` — voir le code ci-dessous.

```
async function ensureAddressForCustomer(customer: ImportCustomer, adresse: string): Promise<string | null> {
```

```

const address1 = adresse?.trim() || 'Adresse import';
const city = adresse?.trim() || 'Ville';
const postcode = '00000';
const alias = `Import ${customer.id}-${Date.now()}`;
try {
  return await createAddress({
    id_customer: customer.id,
    alias,
    firstname: customer.firstname,
    lastname: customer.lastname,
    address1,
    address2: '',
    postcode,
    city,
  });
} catch (err) {
  console.error('Address creation failed', { customer: customer.email, adresse, err });
  return null;
}
}

```

buildCheckoutItems() — ligne 653

Fonction : `build checkout items` — voir le code ci-dessous.

```

async function buildCheckoutItems(items: Array<{ reference: string; qty: number; variant: string }>): Promise<CheckoutItem[]> {
  const result: CheckoutItem[] = [];

  for (const item of items) {
    const product = await fetchProductByReference(item.reference);
    if (!product) throw new Error(`Produit "${item.reference}" introuvable`);

    let attributeId: string | undefined;
    let priceHt = product.priceHt;

    if (item.variant) {
      const combRef = buildCombinationReference(item.reference, item.variant);
      const combination = await fetchCombinationByReference(combRef);
      if (!combination) throw new Error(`Declinaison "${combRef}" introuvable`);
      if (combination.productId && combination.productId !== product.id) {
        console.warn('Combination product mismatch', { combRef, productId: product.id, comboProductId: combination.productId });
      }
      attributeId = combination.id;
      priceHt = product.priceHt + combination.priceImpact;
    }

    const taxRate = await getTaxRateByGroup(product.taxRulesGroupId);
    const priceTtc = roundMoney(priceHt * (1 + taxRate));

    result.push({
      id: product.id,
      name: product.name || item.reference,
      priceHt: roundMoney(priceHt),
      priceTtc,
      taxRate,
      qty: item.qty,
      attributeId,
    });
  }

  return result;
}

```

importFichier3() — ligne 691

Fonction : `import fichier3` — voir le code ci-dessous.

```

export async function importFichier3(

```

```

file: File,
onProgress?: FichierProgressCallback,
): Promise<FichierImportResult[]> {
  const lines = parseCsvContent(await file.text());
  const rows = lines.slice(1)
    .map((r, idx) => ({ row: r, csvLine: idx + 2 }))
    .filter(({ row }) => row[1]);
  const results: FichierImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {
    const { row, csvLine } = rows[i];
    const [date, nom, email, pwd, adresse, achat, etatRaw] = row;
    const label = `${nom} (${email})`;
    const etat = (etatRaw || '').toLowerCase().trim();

    console.log(`Traitement ligne ${csvLine}:`, { date, nom, email, etat });

    onProgress?.(i, rows.length, label);

    try {
      // 1. Gestion Client & Adresse
      const customer = await resolveCustomer(nom, email, pwd);
      if (!customer) throw new Error(`Client impossible à créer/trouver`);

      const addressId = await ensureAddressForCustomer(customer, adresse ?? '');
      if (!addressId) throw new Error(`Erreur création adresse`);

      // 2. Préparation des items
      const items = parseAchat(achat ?? '');
      const checkoutItems = await buildCheckoutItems(items);
      if (checkoutItems.length === 0) throw new Error('Aucun produit valide');

      // 3. CRÉATION DU PANIER
      const cartId = await createPSCart(customer.id, addressId, '1', checkoutItems);

      let finalId = cartId;
      let importType = "Panier";

      // 4. TRANSFORMATION EN COMMANDE
      if (etat === STATUS_MAP.PAID || etat === STATUS_MAP.CANCELLED) {
        // Conversion ROBUSTE de la date
        let orderDate = null;

        if (date && date.trim()) {
          // Support de plusieurs formats
          let day, month, year;

          // Essayer DD/MM/YYYY
          if (date.includes('/')) {
            [day, month, year] = date.split('/');
          }
          // Essayer DD-MM-YYYY
          else if (date.includes('-')) {
            [day, month, year] = date.split('-');
          }
          // Essayer YYYY-MM-DD (déjà formaté)
          else if (date.includes('-') && date[4] === '-') {
            [year, month, day] = date.split('-');
          }

          if (day && month && year) {
            // Nettoyer les valeurs
            day = day.padStart(2, '0');
            month = month.padStart(2, '0');
            year = year.padStart(4, '20');

            orderDate = `${year}-${month}-${day} 00:00:00`;
            console.log(`Date formatée: ${orderDate}`);
          }
        }
      }
    }
  }
}

```

```

// Date par défaut
const finalOrderDate = orderDate || new Date().toISOString().slice(0, 19).replace('T', ' ');
console.log(`Date utilisée pour la commande: ${finalOrderDate}`);

const orderId = await createPSOrder({
  customerId: customer.id,
  addressId,
  cartId,
  carrierId: '1',
  items: checkoutItems,
  shippingCost: 0,
  dateAdd: finalOrderDate,
});

console.log(`Commande créée avec l'ID: ${orderId}, Date: ${finalOrderDate}`);

// Appliquer le statut
const psState = (etat === STATUS_MAP.PAID)
  ? PS_STATE_PAYMENT_ACCEPTED
  : PS_STATE_CANCELED;

const statusXml = `<?xml version="1.0" encoding="UTF-8"?>
  <prestashop>
    <order_history>
      <id_order><![CDATA[${orderId}]]></id_order>
      <id_order_state><![CDATA[${psState}]]></id_order_state>
      <id_employee><![CDATA[1]]></id_employee>
      <date_add><![CDATA[${finalOrderDate}]]></date_add>
    </order_history>
  </prestashop>`;

console.log(`XML historique: ${statusXml}`);

await api.post('/order_histories', statusXml);

finalId = orderId;
importType = "Commande";
}

// ■ NE RIEN FAIRE D'AUTRE - Laisser PrestaShop gérer le stock

results.push({ label: `${label} [${importType}]`, success: true, id: finalId, lineNumber: csvLine });

} catch (err: any) {
  console.error(`Erreur pour ${label}:`, err);
  results.push({
    label,
    success: false,
    error: err.response?.data ? extractXmlError(err.response.data) : err.message,
    lineNumber: csvLine
  });
}
onProgress?.(i + 1, rows.length, label);
}
return results;
}

```

restoreStock() — ligne 822

Description (commentaire d'origine) : Fonction pour restaurer le stock

```

async function restoreStock(productId: string, attributeId: string | undefined, qty: number): Promise<void> {
  const combId = attributeId || '0';
  const stockId = await getStockAvailableId(productId, combId);

  if (stockId) {
    // Récupérer le stock actuel
    const res = await api.get(`/stock_availability/${stockId}?display=[quantity]`);
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
  }
}

```

```

    const currentQty = parseInt(doc.querySelector('quantity')?.textContent ?? '0', 10);

    // Restaurer en ajoutant la quantité
    const newQty = currentQty + qty;
    await setStock(stockId, productId, combId, newQty);
  }
}

```

uploadImage() — ligne 848

Fonction : `upload image` — voir le code ci-dessous.

```

async function uploadImage(productId: string, blob: Blob, filename: string): Promise<void> {
  const formData = new FormData();
  formData.append('image', blob, filename);
  const res = await fetch(`/api/images/products/${productId}`, {
    method: 'POST',
    body: formData,
  });
  if (!res.ok) {
    const text = await res.text();
    throw new Error(extractXmlError(text) || `HTTP ${res.status}`);
  }
}

```

importImagesZip() — ligne 861

Fonction : `import images zip` — voir le code ci-dessous.

```

export async function importImagesZip(
  file: File,
  onProgress?: (done: number, total: number, name: string) => void,
): Promise<ImageImportResult[]> {
  const zip = await JSZip.loadAsync(await file.arrayBuffer());
  const results: ImageImportResult[] = [];
  const entries = Object.entries(zip.files).filter(
    ([name, f]) => !f.dir && !name.startsWith('__MACOSX') && /\. (png|jpg|jpeg|webp|gif)$/i.test(name)
  );

  for (let i = 0; i < entries.length; i++) {
    const [path, zipFile] = entries[i];
    const filename = path.split('/').pop() ?? path;
    const reference = filename.replace(/\. [\^.] +$/, '');
    onProgress?.(i, entries.length, filename);

    try {
      const productId = await getProductIdByRef(reference);
      if (!productId) throw new Error(`Produit "${reference}" introuvable`);
      const blob = await zipFile.async('blob');
      await uploadImage(productId, blob, filename);
      results.push({ filename, reference, success: true });
    } catch (err: any) {
      console.error('Import images failed', {
        filename,
        reference,
        error: err,
      });
      results.push({ filename, reference, success: false, error: err.message });
    }
    onProgress?.(i + 1, entries.length, filename);
  }
  return results;
}

```

productExistsByRef() — ligne 921

Fonction : `product exists by ref` — voir le code ci-dessous.

```
async function productExistsByRef(reference: string, cache: Map<string, boolean>): Promise<boolean> {
  if (cache.has(reference)) return cache.get(reference)!;
  const exists = Boolean(await fetchProductByReference(reference));
  cache.set(reference, exists);
  return exists;
}
```

`combinationExistsByRef()` — ligne 928

Fonction : `combination exists by ref` — voir le code ci-dessous.

```
async function combinationExistsByRef(reference: string, cache: Map<string, boolean>): Promise<boolean> {
  if (cache.has(reference)) return cache.get(reference)!;
  const exists = Boolean(await fetchCombinationByReference(reference));
  cache.set(reference, exists);
  return exists;
}
```

`customerExistsByEmail()` — ligne 935

Fonction : `customer exists by email` — voir le code ci-dessous.

```
async function customerExistsByEmail(email: string, cache: Map<string, boolean>): Promise<boolean> {
  if (cache.has(email)) return cache.get(email)!;
  const exists = Boolean(await findCustomerByEmail(email));
  cache.set(email, exists);
  return exists;
}
```

`prevalidateFichier1Internal()` — ligne 942

Fonction : `prevalidate fichier1internal` — voir le code ci-dessous.

```
async function prevalidateFichier1Internal(
  file: File,
  onProgress: PrevalidateCallbacks['fichier1'],
  productExistsCache: Map<string, boolean>,
): Promise<{ results: FichierImportResult[]; context: PrevalidateContext }> {
  const lines = parseCsvContent(await file.text());
  const header = lines[0] ?? [];

  // Règle 1 : validation des en-têtes
  const headerErrors = validateHeaders(header, FICHER1_COLUMN_SPECS);
  if (headerErrors.length > 0) {
    return {
      results: [{ label: 'En-têtes invalides', success: false, error: headerErrors.join(' ; '), lineNumber: 1 }],
      context: { productRefs: new Set(), taxRateByRef: new Map(), combinationRefs: new Set() },
    };
  }

  const cols = resolveFichier1Columns(header);
  const rows = lines.slice(1)
    .map((r, idx) => ({ row: r, csvLine: idx + 2 }))
    .filter(({ row }) => (row[cols.nomIdx] ?? '').trim() !== '');
  const results: FichierImportResult[] = [];
  const productRefs = new Set<string>();
  const taxRateByRef = new Map<string, number>();

  for (let i = 0; i < rows.length; i++) {
    const { row, csvLine } = rows[i];
    const nom = row[cols.nomIdx] ?? '';
    const reference = row[cols.referenceIdx] ?? '';
    const dateValue = cols.dateIdx >= 0 ? (row[cols.dateIdx] ?? '') : '';
    const prix_ttc_str = row[cols.prixTtcIdx] ?? '';
```

```

const taxe_str = row[cols.taxeIdx] ?? '';
const prix_achat_str = row[cols.prixAchatIdx] ?? '';
const label = `${nom} (${reference})`;
onProgress?.(i, rows.length, label);

const errors: string[] = [];

if (!nom.trim()) errors.push(`Ligne ${csvLine} – "nom" : valeur manquante`);
if (!reference.trim()) errors.push(`Ligne ${csvLine} – "reference" : valeur manquante`);

// Règle 2 : date valide
const dateErr = validateDateField(dateValue, 'date_availability_produit', csvLine, parseDateFlexible);
if (dateErr) errors.push(dateErr);

// Règle 3 : montants strictement positifs
const prixErr = validatePositiveAmount(prix_ttc_str, 'prix_ttc', csvLine);
if (prixErr) errors.push(prixErr);

const prixAchatErr = validatePositiveAmount(prix_achat_str, 'prix_achat', csvLine);
if (prixAchatErr) errors.push(prixAchatErr);

const taxeRaw = taxe_str.trim();
const taxeParsed = taxeRaw ? parseNumberStrict(taxeRaw) : null;
if (taxeRaw && taxeParsed == null) {
  errors.push(`Ligne ${csvLine} – "taxe" : valeur non numérique "${taxe_str}"`);
}

if (reference && productRefs.has(reference)) {
  errors.push(`Ligne ${csvLine} – "reference" : la référence "${reference}" est en double dans le fichier`);
}

if (errors.length === 0 && reference) {
  const exists = await productExistsByRef(reference, productExistsCache);
  if (exists) errors.push(`Ligne ${csvLine} – "reference" : "${reference}" existe déjà dans PrestaShop`);
}

let taxRate = 0;
if (taxeParsed != null) taxRate = Math.max(0, taxeParsed) / 100;

if (errors.length === 0 && reference) {
  productRefs.add(reference);
  taxRateByRef.set(reference, taxRate);
  results.push({ label, success: true, lineNumber: csvLine });
} else {
  results.push({ label, success: false, error: errors.join(' | '), lineNumber: csvLine });
}

onProgress?.(i + 1, rows.length, label);
}

return {
  results,
  context: { productRefs, taxRateByRef, combinationRefs: new Set() },
};
}

```

prevalidateFichier2Internal() — ligne 1029

Fonction : `prevalidate fichier2internal` — voir le code ci-dessous.

```

async function prevalidateFichier2Internal(
  file: File,
  baseContext: PrevalidateContext,
  onProgress: PrevalidateCallbacks['fichier2'],
  productExistsCache: Map<string, boolean>,
  combinationExistsCache: Map<string, boolean>,
): Promise<{ results: FichierImportResult[]; combinationRefs: Set<string> }> {
  const lines = parseCsvContent(await file.text());
  const header = lines[0] ?? [];

```

```

// Règle 1 : validation des en-têtes
const headerErrors = validateHeaders(header, FICHER2_COLUMN_SPECS);
if (headerErrors.length > 0) {
  return {
    results: [{ label: 'En-têtes invalides', success: false, error: headerErrors.join(' ; '), lineNumber: 1,
    combinationRefs: new Set(),
  }];
}

// Résolution des colonnes par nom
const refIdx = resolveColumnIndex(header, ['reference', 'référence', 'ref']);
const specIdx = resolveColumnIndex(header, ['specificité', 'specificite', 'specificite']);
const karaIdx = resolveColumnIndex(header, ['karazany']);
const stockIdx = resolveColumnIndex(header, ['stock_initial', 'stock', 'quantite', 'qty']);
const prixVteIdx = resolveColumnIndex(header, ['prix_vente_ttc', 'prix_ttc', 'prix vente ttc', 'prix vente']);

const rows = lines.slice(1)
  .map((r, idx) => ({ row: r, csvLine: idx + 2 }));
  .filter(({ row }) => (row[refIdx] ?? '').trim() !== '');
const results: FichierImportResult[] = [];
const combinationRefs = new Set<string>();
const seenCombRefs = new Set<string>();

for (let i = 0; i < rows.length; i++) {
  const { row, csvLine } = rows[i];
  const reference = row[refIdx] ?? '';
  const specificite = row[specIdx] ?? '';
  const karazany = row[karaIdx] ?? '';
  const stock_str = row[stockIdx] ?? '';
  const prix_ttc_str = row[prixVteIdx] ?? '';
  const label = `${reference}${karazany ? ' - ' + karazany : ''}`;
  onProgress?.(i, rows.length, label);

  const errors: string[] = [];
  if (!reference.trim()) errors.push(`Ligne ${csvLine} - "reference" : valeur manquante`);

  const stockRaw = stock_str.trim();
  if (stockRaw) {
    const stockParsed = parseInt(stockRaw, 10);
    if (Number.isNaN(stockParsed)) {
      errors.push(`Ligne ${csvLine} - "stock_initial" : valeur non numérique "${stock_str}"`);
    } else if (stockParsed < 0) {
      errors.push(`Ligne ${csvLine} - "stock_initial" : stock négatif (valeur reçue : "${stock_str}")`);
    }
  }

  // Règle 3 : prix_vente_ttc strictement positif si renseigné
  const prixErr = validatePositiveAmount(prix_ttc_str, 'prix_vente_ttc', csvLine, false);
  if (prixErr) errors.push(prixErr);

  const hasSpecificite = Boolean(specificite.trim());
  const hasKarazany = Boolean(karazany.trim());
  if (hasSpecificite !== hasKarazany) {
    errors.push(`Ligne ${csvLine} - "specificité" et "karazany" doivent être renseignés ensemble`);
  }

  let combRef = '';
  if (hasSpecificite && hasKarazany && reference.trim()) {
    combRef = buildCombinationReference(reference.trim(), karazany.trim());
    if (seenCombRefs.has(combRef)) {
      errors.push(`Ligne ${csvLine} - déclinaison "${combRef}" en double dans le fichier`);
    }
    seenCombRefs.add(combRef);
    combinationRefs.add(combRef);
  }

  if (errors.length === 0 && reference.trim()) {
    const ref = reference.trim();
    if (!baseContext.productRefs.has(ref)) {
      const exists = await productExistsByRef(ref, productExistsCache);
      if (!exists) errors.push(`Ligne ${csvLine} - "reference" : produit "${ref}" introuvable dans PrestaShop`);
    }
  }
}

```

```

    }
  }

  if (errors.length === 0 && combRef) {
    const exists = await combinationExistsByRef(combRef, combinationExistsCache);
    if (exists) errors.push(`Ligne ${csvLine} – déclinaison "${combRef}" existe déjà dans PrestaShop`);
  }

  if (errors.length === 0) {
    results.push({ label, success: true, lineNumber: csvLine });
  } else {
    results.push({ label, success: false, error: errors.join(' | '), lineNumber: csvLine });
  }

  onProgress?.(i + 1, rows.length, label);
}

return { results, combinationRefs };
}

```

prevalidateFichier3Internal() — ligne 1130

Fonction : `prevalidate fichier3internal` — voir le code ci-dessous.

```

async function prevalidateFichier3Internal(
  file: File,
  baseContext: PrevalidateContext,
  onProgress: PrevalidateCallbacks['fichier3'],
  productExistsCache: Map<string, boolean>,
  combinationExistsCache: Map<string, boolean>,
  customerExistsCache: Map<string, boolean>,
): Promise<FichierImportResult[]> {
  const lines = parseCsvContent(await file.text());
  const header = lines[0] ?? [];

  // Règle 1 : validation des en-têtes
  const headerErrors = validateHeaders(header, FICHIER3_COLUMN_SPECS);
  if (headerErrors.length > 0) {
    return [{ label: 'En-têtes invalides', success: false, error: headerErrors.join(' ; '), lineNumber: 1 }];
  }

  // Résolution des colonnes par nom
  const dateIdx = resolveColumnIndex(header, ['date']);
  const nomIdx = resolveColumnIndex(header, ['nom', 'name']);
  const emailIdx = resolveColumnIndex(header, ['email', 'mail', 'courriel']);
  const pwdIdx = resolveColumnIndex(header, ['pwd', 'password', 'mot_de_passe']);
  const achatIdx = resolveColumnIndex(header, ['achat', 'commande', 'panier', 'achats']);

  const rows = lines.slice(1)
    .map((r, idx) => ({ row: r, csvLine: idx + 2 }))
    .filter(({ row }) => (row[nomIdx] ?? '').trim() !== '');
  const results: FichierImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {
    const { row, csvLine } = rows[i];
    const dateValue = row[dateIdx] ?? '';
    const nom = row[nomIdx] ?? '';
    const email = row[emailIdx] ?? '';
    const pwd = row[pwdIdx] ?? '';
    const achat = row[achatIdx] ?? '';
    const label = `${nom} (${email})`;
    onProgress?.(i, rows.length, label);

    const errors: string[] = [];
    const emailValue = email.trim();

    if (!nom.trim()) errors.push(`Ligne ${csvLine} – "nom" : valeur manquante`);
    if (!emailValue) errors.push(`Ligne ${csvLine} – "email" : valeur manquante`);
    if (emailValue && !/^S+@\S+\.\S+$/i.test(emailValue)) {

```

```

        errors.push(`Ligne ${csvLine} - "email" : adresse invalide "${email}"`);
    }

    // Règle 2 : date valide
    const dateErr = validateDateField(dateValue, 'date', csvLine, parseDateFlexible);
    if (dateErr) errors.push(dateErr);

    if (emailValue && /\S+@\S+\.\S+$/.test(emailValue)) {
        const customerExists = await customerExistsByEmail(emailValue, customerExistsCache);
        if (!customerExists && !pwd.trim()) {
            errors.push(`Ligne ${csvLine} - "pwd" : mot de passe manquant pour un nouveau client`);
        }
    }

    const items = parseAchat(achat);
    if (items.length === 0) {
        errors.push(`Ligne ${csvLine} - "achat" : aucun produit valide trouvé dans la colonne`);
    }

    for (const item of items) {
        if (!item.reference.trim()) {
            errors.push(`Ligne ${csvLine} - "achat" : référence produit manquante`);
            continue;
        }
        if (item.qty <= 0) {
            errors.push(`Ligne ${csvLine} - "achat" : quantité invalide (${item.qty}) pour "${item.reference}"`);
        }

        const ref = item.reference.trim();
        if (!baseContext.productRefs.has(ref)) {
            const exists = await productExistsByRef(ref, productExistsCache);
            if (!exists) errors.push(`Ligne ${csvLine} - "achat" : produit "${ref}" introuvable dans PrestaShop`);
        }

        if (item.variant?.trim()) {
            const combRef = buildCombinationReference(ref, item.variant.trim());
            if (!baseContext.combinationRefs.has(combRef)) {
                const exists = await combinationExistsByRef(combRef, combinationExistsCache);
                if (!exists) errors.push(`Ligne ${csvLine} - "achat" : déclinaison "${combRef}" introuvable dans PrestaShop`);
            }
        }
    }

    if (errors.length === 0) {
        results.push({ label, success: true, lineNumber: csvLine });
    } else {
        results.push({ label, success: false, error: errors.join(' | '), lineNumber: csvLine });
    }

    onProgress?.(i + 1, rows.length, label);
}

return results;
}

```

prevalidateImagesZipInternal() — ligne 1230

Fonction : `prevalidate images zip internal` — voir le code ci-dessous.

```

async function prevalidateImagesZipInternal(
    file: File,
    baseContext: PrevalidateContext,
    onProgress: PrevalidateCallbacks['images'],
    productExistsCache: Map<string, boolean>,
): Promise<ImageImportResult[]> {
    const zip = await JSZip.loadAsync(await file.arrayBuffer());
    const results: ImageImportResult[] = [];
    const entries = Object.entries(zip.files).filter(
        ([name, f]) => !f.dir && !name.startsWith('__MACOSX') && /\.(png|jpg|jpeg|webp|gif)$/i.test(name)
    );
}

```

```

);

if (entries.length === 0) {
  results.push({
    filename: 'Aucune image',
    reference: '',
    success: false,
    error: 'Archive vide ou sans image valide',
  });
  return results;
}

for (let i = 0; i < entries.length; i++) {
  const [path] = entries[i];
  const filename = path.split('/').pop() ?? path;
  const reference = filename.replace(/\.([^.]+)$/, '');
  onProgress?.(i, entries.length, filename);

  const errors: string[] = [];
  if (!reference.trim()) errors.push('Nom de fichier invalide');

  if (reference.trim() && !baseContext.productRefs.has(reference)) {
    const exists = await productExistsByRef(reference, productExistsCache);
    if (!exists) errors.push(`Produit "${reference}" introuvable`);
  }

  if (errors.length === 0) {
    results.push({ filename, reference, success: true });
  } else {
    results.push({ filename, reference, success: false, error: errors.join(' | ') });
  }

  onProgress?.(i + 1, entries.length, filename);
}

return results;
}

```

prevalidateFichiersImport() — ligne 1278

Fonction : `prevalidate fichiers import` — voir le code ci-dessous.

```

export async function prevalidateFichiersImport(
  files: { fichier1: File; fichier2: File; fichier3: File; images: File },
  callbacks: PrevalidateCallbacks = {},
): Promise<PrevalidateAllResult> {
  const productExistsCache = new Map<string, boolean>();
  const combinationExistsCache = new Map<string, boolean>();
  const customerExistsCache = new Map<string, boolean>();

  const fichier1Result = await prevalidateFichier1Internal(
    files.fichier1,
    callbacks.fichier1,
    productExistsCache,
  );

  const fichier2Result = await prevalidateFichier2Internal(
    files.fichier2,
    fichier1Result.context,
    callbacks.fichier2,
    productExistsCache,
    combinationExistsCache,
  );

  const fullContext: PrevalidateContext = {
    productRefs: fichier1Result.context.productRefs,
    taxRateByRef: fichier1Result.context.taxRateByRef,
    combinationRefs: fichier2Result.combinationRefs,
  };
}

```

```

const fichier3Results = await prevalidateFichier3Internal(
  files.fichier3,
  fullContext,
  callbacks.fichier3,
  productExistsCache,
  combinationExistsCache,
  customerExistsCache,
);

const imagesResults = await prevalidateImagesZipInternal(
  files.images,
  fullContext,
  callbacks.images,
  productExistsCache,
);

const hasErrors =
  fichier1Result.results.some((r) => !r.success)
  || fichier2Result.results.some((r) => !r.success)
  || fichier3Results.some((r) => !r.success)
  || imagesResults.some((r) => !r.success);

return {
  fichier1: fichier1Result.results,
  fichier2: fichier2Result.results,
  fichier3: fichier3Results,
  images: imagesResults,
  hasErrors,
};
}

```

Fichier : src/services/otherImportService.ts

Lignes : 726 • Fonctions détectées : 35

cleanCell() — ligne 32

Description (commentaire d'origine) : =====
UTILITAIRES =====

```

function cleanCell(v: string): string {
  return v.trim().replace(/^["]|[""]$/g, '');
}

function parseCsvLines(content: string): string[][] {
  return content
    .split(/\r?\n/)
    .slice(1)
    .filter((l) => l.trim() !== '')
    .map((l) => l.split(';').map(cleanCell));
}

function slug(name: string): string {
  return name
    .toLowerCase()
    .normalize('NFD').replace(/[\p{D}]/g, '')
    .replace(/^[a-z0-9]+/g, '-')
    .replace(/^-|-$/g, '') || 'item';
}

function extractXmlError(xmlString: string): string {
  try {
    const doc = new DOMParser().parseFromString(xmlString, 'text/xml');

```

```

    const msg = doc.querySelector('message')?.textContent?.trim();
    if (msg) return msg;
    const err = doc.querySelector('error')?.textContent?.trim();
    if (err) return err;
  } catch { /* ignore */ }
  return typeof xmlString === 'string' ? xmlString.slice(0, 140) : 'Erreur API';
}

function getCreatedId(xmlString: string): string | null {
  try {
    const doc = new DOMParser().parseFromString(xmlString, 'text/xml');
    return doc.querySelector('id')?.textContent?.trim() || null;
  } catch { return null; }
}

async function postEntity(endpoint: string, xml: string): Promise<string> {
  const res = await api.post(endpoint, xml);
  return getCreatedId(res.data) || '?';
}

function parseIdList(xmlString: string): string[] {
  const doc = new DOMParser().parseFromString(xmlString, 'text/xml');
  const ids: string[] = [];
  const elements = doc.getElementsByTagName('id');
  for (let i = 0; i < elements.length; i++) {
    const id = elements[i].textContent?.trim();
    if (id) ids.push(id);
  }
  return ids;
}

// =====
// NETTOYAGE GÉNÉRIQUE
// =====

export async function cleanEntities(
  endpoint: string,
  protectedIds: string[] = [],
  onProgress?: ProgressCallback,
): Promise<CleanResult> {
  const res = await api.get(`${endpoint}?display=[id]`);
  const ids = parseIdList(res.data).filter((id) => !protectedIds.includes(id));
  let deleted = 0;
  let errors = 0;

  for (let i = 0; i < ids.length; i++) {
    try {
      await api.delete(`${endpoint}/${ids[i]}`);
      deleted++;
    } catch {
      errors++;
    }
    onProgress?.(i + 1, ids.length);
  }
  return { total: ids.length, deleted, errors };
}

// Fonctions de nettoyage par entité
export const cleanCategories = (cb?: ProgressCallback) => cleanEntities('/categories', ['1', '2'], cb);
export const cleanCustomers = (cb?: ProgressCallback) => cleanEntities('/customers', [], cb);
export const cleanAddresses = (cb?: ProgressCallback) => cleanEntities('/addresses', [], cb);
export const cleanSuppliers = (cb?: ProgressCallback) => cleanEntities('/suppliers', [], cb);
export const cleanBrands = (cb?: ProgressCallback) => cleanEntities('/manufacturers', [], cb);
export const cleanCombinations = (cb?: ProgressCallback) => cleanEntities('/combinations', [], cb);
export const cleanProducts = (cb?: ProgressCallback) => cleanEntities('/products', [], cb);
export const cleanOrders = (cb?: ProgressCallback) => cleanEntities('/orders', [], cb);

// =====
// 1. CATÉGORIES
// =====

```

```

export interface CategoryRow {
  active: string;
  name: string;
  parentCategory: string;
  description: string;
  metaTitle: string;
  metaKeywords: string;
  metaDescription: string;
  linkRewrite: string;
}

export function parseCategoryCsv(content: string): CategoryRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      active: c[1] ?? '1',
      name: c[2] ?? '',
      parentCategory: c[3] ?? 'Home',
      description: c[5] ?? '',
      metaTitle: c[6] ?? '',
      metaKeywords: c[7] ?? '',
      metaDescription: c[8] ?? '',
      linkRewrite: c[9] ?? '',
    })))
    .filter((r) => r.name !== '');
}

function resolveParentId(parentName: string): number {
  if (!parentName || parentName.toLowerCase() === 'home') return 2;
  if (parentName.toLowerCase() === 'root') return 1;
  const n = parseInt(parentName, 10);
  return isNaN(n) ? 2 : n;
}

function buildCategoryXml(row: CategoryRow): string {
  const lr = row.linkRewrite || slug(row.name);
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <category>
    <active><![CDATA[${row.active} || '1']]></active>
    <id_parent><![CDATA[${resolveParentId(row.parentCategory)}]]></id_parent>
    <name><language id="1"><![CDATA[${row.name}]]></language></name>
    <description><language id="1"><![CDATA[${row.description}]]></language></description>
    <link_rewrite><language id="1"><![CDATA[${lr}]]></language></link_rewrite>
    <meta_title><language id="1"><![CDATA[${row.metaTitle}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[${row.metaKeywords}]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[${row.metaDescription}]]></language></meta_description>
  </category>
</prestashop>`;
}

export async function importCategories(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const rows = parseCategoryCsv(await file.text());
  const results: ImportResult[] = [];
  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    try {
      const id = await postEntity('/categories', buildCategoryXml(row));
      results.push({ rowIndex: i, label: row.name, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label: row.name, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

// =====

```

```

// 2. CLIENTS
// =====

export interface CustomerRow {
  active: string;
  titleId: string;
  email: string;
  password: string;
  birthday: string;
  lastName: string;
  firstName: string;
  newsletter: string;
  optin: string;
  defaultGroupId: string;
}

export function parseCustomerCsv(content: string): CustomerRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      active: c[1] ?? '1',
      titleId: c[2] ?? '0',
      email: c[3] ?? '',
      password: c[4] ?? '',
      birthday: c[5] ?? '',
      lastName: c[6] ?? '',
      firstName: c[7] ?? '',
      newsletter: c[8] ?? '0',
      optin: c[9] ?? '0',
      defaultGroupId: c[12] ?? '3',
    }))
    .filter((r) => r.email !== '');
}

function buildCustomerXml(row: CustomerRow): string {
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <customer>
    <active><![CDATA[${row.active}]]></active>
    <id_gender><![CDATA[${row.titleId}]]></id_gender>
    <email><![CDATA[${row.email}]]></email>
    <passwd><![CDATA[${row.password}]]></passwd>
    <birthday><![CDATA[${row.birthday}]]></birthday>
    <lastname><![CDATA[${row.lastName}]]></lastname>
    <firstname><![CDATA[${row.firstName}]]></firstname>
    <newsletter><![CDATA[${row.newsletter}]]></newsletter>
    <optin><![CDATA[${row.optin}]]></optin>
    <id_default_group><![CDATA[${row.defaultGroupId}]]></id_default_group>
  </customer>
</prestashop>`;
}

export async function importCustomers(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const rows = parseCustomerCsv(await file.text());
  const results: ImportResult[] = [];
  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    const label = `${row.firstName} ${row.lastName} (${row.email})`;
    try {
      const id = await postEntity('/customers', buildCustomerXml(row));
      results.push({ rowIndex: i, label, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

```

```

// =====
// 3. ADRESSES
// =====

// Cache pays : nom (minuscule) → id
async function buildCountryCache(): Promise<Map<string, string>> {
  const cache = new Map<string, string>();
  try {
    const res = await api.get('/countries?display=[id,name]');
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    doc.querySelectorAll('country').forEach((c) => {
      const id = c.querySelector('id')?.textContent?.trim();
      const name = c.querySelector('name language')?.textContent?.trim()
        ?? c.querySelector('name')?.textContent?.trim();
      if (id && name) cache.set(name.toLowerCase(), id);
    });
  } catch { /* API unreachable: cache stays empty */ }
  return cache;
}

// Cache client : email (minuscule) → id
async function buildCustomerCache(): Promise<Map<string, string>> {
  const cache = new Map<string, string>();
  try {
    const res = await api.get('/customers?display=[id,email]');
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    doc.querySelectorAll('customer').forEach((c) => {
      const id = c.querySelector('id')?.textContent?.trim();
      const email = c.querySelector('email')?.textContent?.trim();
      if (id && email) cache.set(email.toLowerCase(), id);
    });
  } catch { /* ignore */ }
  return cache;
}

export interface AddressRow {
  alias: string;
  active: string;
  customerEmail: string;
  company: string;
  lastName: string;
  firstName: string;
  address1: string;
  address2: string;
  postcode: string;
  city: string;
  country: string;
  phone: string;
  phoneMobile: string;
}

export function parseAddressCsv(content: string): AddressRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      alias: c[1] ?? '',
      active: c[2] ?? '1',
      customerEmail: c[3] ?? '',
      company: c[7] ?? '',
      lastName: c[8] ?? '',
      firstName: c[9] ?? '',
      address1: c[10] ?? '',
      address2: c[11] ?? '',
      postcode: c[12] ?? '',
      city: c[13] ?? '',
      country: c[14] ?? '',
      phone: c[17] ?? '',
      phoneMobile: c[18] ?? '',
    }))
    .filter((r) => r.lastName !== '');
}

```

```

function buildAddressXml(
  row: AddressRow,
  customerId: string,
  countryId: string,
): string {
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <address>
    <active><![CDATA[${row.active} || '1']]></active>
    <deleted><![CDATA[0]]></deleted>
    <id_customer><![CDATA[${customerId}]]></id_customer>
    <id_country><![CDATA[${countryId}]]></id_country>
    <alias><![CDATA[${row.alias}]]></alias>
    <company><![CDATA[${row.company}]]></company>
    <lastname><![CDATA[${row.lastName}]]></lastname>
    <firstname><![CDATA[${row.firstName}]]></firstname>
    <address1><![CDATA[${row.address1}]]></address1>
    <address2><![CDATA[${row.address2}]]></address2>
    <postcode><![CDATA[${row.postcode}]]></postcode>
    <city><![CDATA[${row.city}]]></city>
    <phone><![CDATA[${row.phone}]]></phone>
    <phone_mobile><![CDATA[${row.phoneMobile}]]></phone_mobile>
  </address>
</prestashop>`;
}

export async function importAddresses(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const [countryCache, customerCache] = await Promise.all([
    buildCountryCache(),
    buildCustomerCache(),
  ]);

  const rows = parseAddressCsv(await file.text());
  const results: ImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    const label = `${row.firstName} ${row.lastName}`;
    try {
      const countryId = countryCache.get(row.country.toLowerCase());
      if (!countryId) throw new Error(`Pays introuvable : "${row.country}"`);

      const customerId = customerCache.get(row.customerEmail.toLowerCase()) ?? '0';
      const xml = buildAddressXml(row, customerId, countryId);
      const id = await postEntity('/addresses', xml);
      results.push({ rowIndex: i, label, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

// =====
// 4. FOURNISSEURS
// =====

export interface SupplierRow {
  active: string;
  name: string;
  description: string;
  metaTitle: string;
  metaKeywords: string;
  metaDescription: string;
}

```

```

export function parseSupplierCsv(content: string): SupplierRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      active:      c[1] ?? '1',
      name:        c[2] ?? '',
      description: c[3] ?? '',
      metaTitle:   c[4] ?? '',
      metaKeywords: c[5] ?? '',
      metaDescription: c[6] ?? ''
    })))
    .filter((r) => r.name !== '');
}

function buildSupplierXml(row: SupplierRow): string {
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <supplier>
    <active><![CDATA[${row.active}]]></active>
    <name><![CDATA[${row.name}]]></name>
    <description><language id="1"><![CDATA[${row.description}]]></language></description>
    <meta_title><language id="1"><![CDATA[${row.metaTitle}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[${row.metaKeywords}]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[${row.metaDescription}]]></language></meta_description>
  </supplier>
</prestashop>`;
}

export async function importSuppliers(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const rows = parseSupplierCsv(await file.text());
  const results: ImportResult[] = [];
  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    try {
      const id = await postEntity('/suppliers', buildSupplierXml(row));
      results.push({ rowIndex: i, label: row.name, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label: row.name, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

// =====
// 5. MARQUES (manufacturers)
// =====

export interface BrandRow {
  active: string;
  name: string;
  description: string;
  shortDescription: string;
  metaTitle: string;
  metaKeywords: string;
  metaDescription: string;
}

export function parseBrandCsv(content: string): BrandRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      active:      c[1] ?? '1',
      name:        c[2] ?? '',
      description: c[3] ?? '',
      shortDescription: c[4] ?? '',
      metaTitle:   c[5] ?? '',
      metaKeywords: c[6] ?? '',
      metaDescription: c[7] ?? ''
    })))

```

```

    )))
    .filter((r) => r.name !== '');
}

function buildBrandXml(row: BrandRow): string {
    return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <manufacturer>
    <active><![CDATA[${row.active} || '1']]></active>
    <name><![CDATA[${row.name}]]></name>
    <description><language id="1"><![CDATA[${row.description}]]></language></description>
    <short_description><language id="1"><![CDATA[${row.shortDescription}]]></language></short_description>
    <meta_title><language id="1"><![CDATA[${row.metaTitle}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[${row.metaKeywords}]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[${row.metaDescription}]]></language></meta_description>
  </manufacturer>
</prestashop>`;
}

export async function importBrands(
    file: File,
    onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
    const rows = parseBrandCsv(await file.text());
    const results: ImportResult[] = [];
    for (let i = 0; i < rows.length; i++) {
        const row = rows[i];
        try {
            const id = await postEntity('/manufacturers', buildBrandXml(row));
            results.push({ rowIndex: i, label: row.name, success: true, id });
        } catch (err: any) {
            results.push({ rowIndex: i, label: row.name, success: false,
                error: err.response?.data ? extractXmlError(err.response.data) : err.message });
        }
        onProgress?.(i + 1, rows.length);
    }
    return results;
}

// =====
// 6. DÉCLINAISONS (combinations)
// =====

export interface CombinationRow {
    productId: string;
    attributeSpec: string; // "Color:color:0, Disk space:select:1"
    valueSpec: string; // "Blue:0, 16GB:1"
    reference: string;
    ean13: string;
    wholesalePrice: string;
    impactPrice: string;
    quantity: string;
    minimalQuantity: string;
    isDefault: string;
    availableDate: string;
}

export function parseCombinationCsv(content: string): CombinationRow[] {
    return parseCsvLines(content)
        .map((c) => ({
            productId: c[0] ?? '',
            attributeSpec: c[1] ?? '',
            valueSpec: c[2] ?? '',
            reference: c[4] ?? '',
            ean13: c[5] ?? '',
            wholesalePrice: c[7] ?? '0',
            impactPrice: c[8] ?? '0',
            quantity: c[10] ?? '0',
            minimalQuantity: c[11] ?? '1',
            isDefault: c[14] ?? '0',
            availableDate: c[15] ?? '',
        }));
}

```

```

    )))
    .filter((r) => r.productId !== '');
}

interface AttributeDef { name: string; type: string; position: number }
interface ValueDef      { value: string; position: number }

function parseAttributeSpec(spec: string): AttributeDef[] {
    return spec.split(', ').filter(Boolean).map((part) => {
        const lastColon = part.lastIndexOf(':');
        const position  = parseInt(part.slice(lastColon + 1), 10) || 0;
        const rest      = part.slice(0, lastColon);
        const midColon  = rest.lastIndexOf(':');
        const type       = rest.slice(midColon + 1);
        const name       = rest.slice(0, midColon);
        return { name, type, position };
    });
}

function parseValueSpec(spec: string): ValueDef[] {
    return spec.split(', ').filter(Boolean).map((part) => {
        const lastColon = part.lastIndexOf(':');
        const position  = parseInt(part.slice(lastColon + 1), 10) || 0;
        const value     = part.slice(0, lastColon);
        return { value, position };
    });
}

// Load existing product_options into cache (name → id)
async function loadOptionCache(): Promise<Map<string, string>> {
    const cache = new Map<string, string>();
    try {
        const res = await api.get('/product_options?display=full');
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        doc.querySelectorAll('product_option').forEach((opt) => {
            const id   = opt.querySelector('id')?.textContent?.trim();
            const name = opt.querySelector('name language')?.textContent?.trim()
                ?? opt.querySelector('name')?.textContent?.trim();
            if (id && name) cache.set(name.toLowerCase(), id);
        });
    } catch { /* ignore */ }
    return cache;
}

// Load existing product_option_values into cache ("optionId:valueName" → id)
async function loadOptionValueCache(): Promise<Map<string, string>> {
    const cache = new Map<string, string>();
    try {
        const res = await api.get('/product_option_values?display=full');
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        doc.querySelectorAll('product_option_value').forEach((v) => {
            const id          = v.querySelector('id')?.textContent?.trim();
            const groupId     = v.querySelector('id_attribute_group')?.textContent?.trim();
            const name        = v.querySelector('name language')?.textContent?.trim()
                ?? v.querySelector('name')?.textContent?.trim();
            if (id && groupId && name) cache.set(`${groupId}:${name.toLowerCase()}`, id);
        });
    } catch { /* ignore */ }
    return cache;
}

async function getOrCreateOption(
    name: string,
    type: string,
    position: number,
    cache: Map<string, string>,
): Promise<string> {
    const key = name.toLowerCase();
    if (cache.has(key)) return cache.get(key)!;
    const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">

```

```

    <product_option>
      <name><language id="1"><![CDATA[${name}]]></language></name>
      <public_name><language id="1"><![CDATA[${name}]]></language></public_name>
      <group_type><![CDATA[${type} || 'select']]></group_type>
      <is_color_group><![CDATA[${type} === 'color' ? 1 : 0]]></is_color_group>
      <position><![CDATA[${position}]]></position>
    </product_option>
  </prestashop>`;
  const id = await postEntity('/product_options', xml);
  cache.set(key, id);
  return id;
}

async function getOrCreateOptionValue(
  optionId: string,
  valueName: string,
  position: number,
  cache: Map<string, string>,
): Promise<string> {
  const key = `${optionId}:${valueName.toLowerCase()}`;
  if (cache.has(key)) return cache.get(key)!;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <product_option_value>
    <id_attribute_group><![CDATA[${optionId}]]></id_attribute_group>
    <name><language id="1"><![CDATA[${valueName}]]></language></name>
    <position><![CDATA[${position}]]></position>
  </product_option_value>
</prestashop>`;
  const id = await postEntity('/product_option_values', xml);
  cache.set(key, id);
  return id;
}

function buildCombinationXml(row: CombinationRow, optionValueIds: string[]): string {
  const assocXml = optionValueIds
    .map((id) => `<product_option_value><id><![CDATA[${id}]]></id></product_option_value>`)
    .join('\n      ');
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <combination>
    <id_product><![CDATA[${row.productId}]]></id_product>
    <reference><![CDATA[${row.reference}]]></reference>
    <ean13><![CDATA[${/^d{13}$/.test(row.ean13) ? row.ean13 : ''}]]></ean13>
    <wholesale_price><![CDATA[${row.wholesalePrice} || '0']]]></wholesale_price>
    <price><![CDATA[${row.impactPrice} || '0']]]></price>
    <minimal_quantity><![CDATA[${row.minimalQuantity} || '1']]]></minimal_quantity>
    <default_on><![CDATA[${row.isDefault} || '0']]]></default_on>
    <available_date><![CDATA[${row.availableDate}]]></available_date>
    <associations>
      <product_option_values>
        ${assocXml}
      </product_option_values>
    </associations>
  </combination>
</prestashop>`;
}

export async function importCombinations(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const [optionCache, optionValueCache] = await Promise.all([
    loadOptionCache(),
    loadOptionValueCache(),
  ]);

  const rows = parseCombinationCsv(await file.text());
  const results: ImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {

```

```

const row = rows[i];
const label = `Produit ${row.productId} - ${row.valueSpec}`;
try {
  const attrDefs = parseAttributeSpec(row.attributeSpec);
  const valueDefs = parseValueSpec(row.valueSpec);

  if (attrDefs.length !== valueDefs.length) {
    throw new Error('Nombre d\'attributs et de valeurs incohérent');
  }
}

```

parseCsvLines() — ligne 36

Fonction : `parse csv lines` — voir le code ci-dessous.

```

function parseCsvLines(content: string): string[][] {
  return content
    .split(/\r?\n/)
    .slice(1)
    .filter((l) => l.trim() !== '')
    .map((l) => l.split(';').map(cleanCell));
}

```

sSlug() — ligne 44

Fonction : `s slug` — voir le code ci-dessous.

```

function slug(name: string): string {
  return name
    .toLowerCase()
    .normalize('NFD').replace(/[\p{D}]/g, '')
    .replace(/[\p{L}-z0-9]+/g, '-')
    .replace(/^-|-$/g, '') || 'item';
}

```

extractXmlError() — ligne 52

Fonction : `extract xml error` — voir le code ci-dessous.

```

function extractXmlError(xmlString: string): string {
  try {
    const doc = new DOMParser().parseFromString(xmlString, 'text/xml');
    const msg = doc.querySelector('message')?.textContent?.trim();
    if (msg) return msg;
    const err = doc.querySelector('error')?.textContent?.trim();
    if (err) return err;
  } catch { /* ignore */ }
  return typeof xmlString === 'string' ? xmlString.slice(0, 140) : 'Erreur API';
}

```

getCreatedId() — ligne 63

Fonction : `get created id` — voir le code ci-dessous.

```

function getCreatedId(xmlString: string): string | null {
  try {
    const doc = new DOMParser().parseFromString(xmlString, 'text/xml');
    return doc.querySelector('id')?.textContent?.trim() || null;
  } catch { return null; }
}

```

postEntity() — ligne 70

Fonction : `post entity` — voir le code ci-dessous.

```
async function postEntity(endpoint: string, xml: string): Promise<string> {
  const res = await api.post(endpoint, xml);
  return getCreatedId(res.data) || '?';
}
```

`parseIdList()` — ligne 75

Fonction : `parse id list` — voir le code ci-dessous.

```
function parseIdList(xmlString: string): string[] {
  const doc = new DOMParser().parseFromString(xmlString, 'text/xml');
  const ids: string[] = [];
  const elements = doc.getElementsByTagName('id');
  for (let i = 0; i < elements.length; i++) {
    const id = elements[i].textContent?.trim();
    if (id) ids.push(id);
  }
  return ids;
}
```

`cleanEntities()` — ligne 90

Description (commentaire d'origine) : =====
NETTOYAGE GÉNÉRIQUE =====

```
export async function cleanEntities(
  endpoint: string,
  protectedIds: string[] = [],
  onProgress?: ProgressCallback,
): Promise<CleanResult> {
  const res = await api.get(`${endpoint}?display=[id]`);
  const ids = parseIdList(res.data).filter((id) => !protectedIds.includes(id));
  let deleted = 0;
  let errors = 0;

  for (let i = 0; i < ids.length; i++) {
    try {
      await api.delete(`${endpoint}/${ids[i]}`);
      deleted++;
    } catch {
      errors++;
    }
    onProgress?.(i + 1, ids.length);
  }
  return { total: ids.length, deleted, errors };
}
```

`parseCategoryCsv()` — ligne 137

Fonction : `parse category csv` — voir le code ci-dessous.

```
export function parseCategoryCsv(content: string): CategoryRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      active:      c[1] ?? '1',
      name:        c[2] ?? '',
      parentCategory: c[3] ?? 'Home',
      description:  c[5] ?? '',
      metaTitle:    c[6] ?? '',
      metaKeywords: c[7] ?? '',
      metaDescription: c[8] ?? '',
      linkRewrite:  c[9] ?? '',
    })))
    .filter((r) => r.name !== '');
}
```

```
}
```

resolveParentId() — ligne 152

Fonction : `resolve parent id` — voir le code ci-dessous.

```
function resolveParentId(parentName: string): number {
  if (!parentName || parentName.toLowerCase() === 'home') return 2;
  if (parentName.toLowerCase() === 'root') return 1;
  const n = parseInt(parentName, 10);
  return isNaN(n) ? 2 : n;
}
```

buildCategoryXml() — ligne 159

Fonction : `build category xml` — voir le code ci-dessous.

```
function buildCategoryXml(row: CategoryRow): string {
  const lr = row.linkRewrite || slug(row.name);
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <category>
    <active><![CDATA[${row.active}]]></active>
    <id_parent><![CDATA[${resolveParentId(row.parentCategory)}]]></id_parent>
    <name><language id="1"><![CDATA[${row.name}]]></language></name>
    <description><language id="1"><![CDATA[${row.description}]]></language></description>
    <link_rewrite><language id="1"><![CDATA[${lr}]]></language></link_rewrite>
    <meta_title><language id="1"><![CDATA[${row.metaTitle}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[${row.metaKeywords}]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[${row.metaDescription}]]></language></meta_description>
  </category>
</prestashop>`;
}
```

importCategories() — ligne 176

Fonction : `import categories` — voir le code ci-dessous.

```
export async function importCategories(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const rows = parseCategoryCsv(await file.text());
  const results: ImportResult[] = [];
  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    try {
      const id = await postEntity('/categories', buildCategoryXml(row));
      results.push({ rowIndex: i, label: row.name, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label: row.name, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}
```

parseCustomerCsv() — ligne 213

Fonction : `parse customer csv` — voir le code ci-dessous.

```
export function parseCustomerCsv(content: string): CustomerRow[] {
  return parseCsvLines(content)
```

```

.map((c) => ({
  active:      c[1] ?? '1',
  titleId:     c[2] ?? '0',
  email:       c[3] ?? '',
  password:    c[4] ?? '',
  birthday:    c[5] ?? '',
  lastName:    c[6] ?? '',
  firstName:   c[7] ?? '',
  newsletter:  c[8] ?? '0',
  optin:       c[9] ?? '0',
  defaultGroupId: c[12] ?? '3',
})))
.filter((r) => r.email !== '');
}

```

buildCustomerXml() — ligne 230

Fonction : `build customer xml` — voir le code ci-dessous.

```

function buildCustomerXml(row: CustomerRow): string {
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <customer>
    <active><![CDATA[${row.active}]]></active>
    <id_gender><![CDATA[${row.titleId}]]></id_gender>
    <email><![CDATA[${row.email}]]></email>
    <passwd><![CDATA[${row.password}]]></passwd>
    <birthday><![CDATA[${row.birthday}]]></birthday>
    <lastname><![CDATA[${row.lastName}]]></lastname>
    <firstname><![CDATA[${row.firstName}]]></firstname>
    <newsletter><![CDATA[${row.newsletter}]]></newsletter>
    <optin><![CDATA[${row.optin}]]></optin>
    <id_default_group><![CDATA[${row.defaultGroupId}]]></id_default_group>
  </customer>
</prestashop>`;
}

```

importCustomers() — ligne 248

Fonction : `import customers` — voir le code ci-dessous.

```

export async function importCustomers(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const rows = parseCustomerCsv(await file.text());
  const results: ImportResult[] = [];
  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    const label = `${row.firstName} ${row.lastName} (${row.email})`;
    try {
      const id = await postEntity('/customers', buildCustomerXml(row));
      results.push({ rowIndex: i, label, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

```

buildCountryCache() — ligne 275

Description (commentaire d'origine) : Cache pays : nom (minuscule) → id

```

async function buildCountryCache(): Promise<Map<string, string>> {
  const cache = new Map<string, string>();
  try {
    const res = await api.get('/countries?display=[id,name]');
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    doc.querySelectorAll('country').forEach((c) => {
      const id = c.querySelector('id')?.textContent?.trim();
      const name = c.querySelector('name language')?.textContent?.trim()
        ?? c.querySelector('name')?.textContent?.trim();
      if (id && name) cache.set(name.toLowerCase(), id);
    });
  } catch { /* API unreachable: cache stays empty */ }
  return cache;
}

```

buildCustomerCache() — ligne 291

Description (commentaire d'origine) : Cache client : email (minuscule) → id

```

async function buildCustomerCache(): Promise<Map<string, string>> {
  const cache = new Map<string, string>();
  try {
    const res = await api.get('/customers?display=[id,email]');
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    doc.querySelectorAll('customer').forEach((c) => {
      const id = c.querySelector('id')?.textContent?.trim();
      const email = c.querySelector('email')?.textContent?.trim();
      if (id && email) cache.set(email.toLowerCase(), id);
    });
  } catch { /* ignore */ }
  return cache;
}

```

parseAddressCsv() — ligne 320

Fonction : `parse address csv` — voir le code ci-dessous.

```

export function parseAddressCsv(content: string): AddressRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      alias: c[1] ?? '',
      active: c[2] ?? '1',
      customerEmail: c[3] ?? '',
      company: c[7] ?? '',
      lastName: c[8] ?? '',
      firstName: c[9] ?? '',
      address1: c[10] ?? '',
      address2: c[11] ?? '',
      postcode: c[12] ?? '',
      city: c[13] ?? '',
      country: c[14] ?? '',
      phone: c[17] ?? '',
      phoneMobile: c[18] ?? '',
    })))
    .filter((r) => r.lastName !== '');
}

```

buildAddressXml() — ligne 340

Fonction : `build address xml` — voir le code ci-dessous.

```

function buildAddressXml(
  row: AddressRow,
  customerId: string,
  countryId: string,
): string {

```

```

    return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <address>
    <active><![CDATA[${row.active} || '1']]></active>
    <deleted><![CDATA[0]]></deleted>
    <id_customer><![CDATA[${customerId}]]></id_customer>
    <id_country><![CDATA[${countryId}]]></id_country>
    <alias><![CDATA[${row.alias}]]></alias>
    <company><![CDATA[${row.company}]]></company>
    <lastname><![CDATA[${row.lastName}]]></lastname>
    <firstname><![CDATA[${row.firstName}]]></firstname>
    <address1><![CDATA[${row.address1}]]></address1>
    <address2><![CDATA[${row.address2}]]></address2>
    <postcode><![CDATA[${row.postcode}]]></postcode>
    <city><![CDATA[${row.city}]]></city>
    <phone><![CDATA[${row.phone}]]></phone>
    <phone_mobile><![CDATA[${row.phoneMobile}]]></phone_mobile>
  </address>
</prestashop>`;
}

```

importAddresses() — ligne 366

Fonction : `import addresses` — voir le code ci-dessous.

```

export async function importAddresses(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const [countryCache, customerCache] = await Promise.all([
    buildCountryCache(),
    buildCustomerCache(),
  ]);

  const rows = parseAddressCsv(await file.text());
  const results: ImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    const label = `${row.firstName} ${row.lastName}`;
    try {
      const countryId = countryCache.get(row.country.toLowerCase());
      if (!countryId) throw new Error(`Pays introuvable : "${row.country}"`);

      const customerId = customerCache.get(row.customerEmail.toLowerCase()) ?? '0';
      const xml = buildAddressXml(row, customerId, countryId);
      const id = await postEntity('/addresses', xml);
      results.push({ rowIndex: i, label, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

```

parseSupplierCsv() — ligne 411

Fonction : `parse supplier csv` — voir le code ci-dessous.

```

export function parseSupplierCsv(content: string): SupplierRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      active:      c[1] ?? '1',
      name:        c[2] ?? '',
      description: c[3] ?? '',
      metaTitle:   c[4] ?? '',
    }));
}

```

```

        metaKeywords:    c[5] ?? '',
        metaDescription: c[6] ?? '',
    })))
    .filter((r) => r.name !== '');
}

```

buildSupplierXml() — ligne 424

Fonction : `build supplier xml` — voir le code ci-dessous.

```

function buildSupplierXml(row: SupplierRow): string {
    return `<?xml version="1.0" encoding="UTF-8"?>
    <prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
    <supplier>
    <active><![CDATA[${row.active}]]></active>
    <name><![CDATA[${row.name}]]></name>
    <description><language id="1"><![CDATA[${row.description}]]></language></description>
    <meta_title><language id="1"><![CDATA[${row.metaTitle}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[${row.metaKeywords}]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[${row.metaDescription}]]></language></meta_description>
    </supplier>
    </prestashop>`;
}

```

importSuppliers() — ligne 438

Fonction : `import suppliers` — voir le code ci-dessous.

```

export async function importSuppliers(
    file: File,
    onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
    const rows = parseSupplierCsv(await file.text());
    const results: ImportResult[] = [];
    for (let i = 0; i < rows.length; i++) {
        const row = rows[i];
        try {
            const id = await postEntity('/suppliers', buildSupplierXml(row));
            results.push({ rowIndex: i, label: row.name, success: true, id });
        } catch (err: any) {
            results.push({ rowIndex: i, label: row.name, success: false,
                error: err.response?.data ? extractXmlError(err.response.data) : err.message });
        }
        onProgress?.(i + 1, rows.length);
    }
    return results;
}

```

parseBrandCsv() — ligne 472

Fonction : `parse brand csv` — voir le code ci-dessous.

```

export function parseBrandCsv(content: string): BrandRow[] {
    return parseCsvLines(content)
        .map((c) => ({
            active:    c[1] ?? '1',
            name:      c[2] ?? '',
            description: c[3] ?? '',
            shortDescription: c[4] ?? '',
            metaTitle:  c[5] ?? '',
            metaKeywords: c[6] ?? '',
            metaDescription: c[7] ?? '',
        })))
        .filter((r) => r.name !== '');
}

```

buildBrandXml() — ligne 486

Fonction : `build brand xml` — voir le code ci-dessous.

```
function buildBrandXml(row: BrandRow): string {
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <manufacturer>
    <active><![CDATA[${row.active} || '1']]></active>
    <name><![CDATA[${row.name}]]></name>
    <description><language id="1"><![CDATA[${row.description}]]></language></description>
    <short_description><language id="1"><![CDATA[${row.shortDescription}]]></language></short_description>
    <meta_title><language id="1"><![CDATA[${row.metaTitle}]]></language></meta_title>
    <meta_keywords><language id="1"><![CDATA[${row.metaKeywords}]]></language></meta_keywords>
    <meta_description><language id="1"><![CDATA[${row.metaDescription}]]></language></meta_description>
  </manufacturer>
</prestashop>`;
}
```

importBrands() — ligne 501

Fonction : `import brands` — voir le code ci-dessous.

```
export async function importBrands(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const rows = parseBrandCsv(await file.text());
  const results: ImportResult[] = [];
  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    try {
      const id = await postEntity('/manufacturers', buildBrandXml(row));
      results.push({ rowIndex: i, label: row.name, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label: row.name, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}
```

parseCombinationCsv() — ligne 539

Fonction : `parse combination csv` — voir le code ci-dessous.

```
export function parseCombinationCsv(content: string): CombinationRow[] {
  return parseCsvLines(content)
    .map((c) => ({
      productId: c[0] ?? '',
      attributeSpec: c[1] ?? '',
      valueSpec: c[2] ?? '',
      reference: c[4] ?? '',
      ean13: c[5] ?? '',
      wholesalePrice: c[7] ?? '0',
      impactPrice: c[8] ?? '0',
      quantity: c[10] ?? '0',
      minimalQuantity: c[11] ?? '1',
      isDefault: c[14] ?? '0',
      availableDate: c[15] ?? '',
    })))
    .filter((r) => r.productId !== '');
}
```

parseAttributeSpec() — ligne 560

Fonction : `parse attribute spec` — voir le code ci-dessous.

```
function parseAttributeSpec(spec: string): AttributeDef[] {
  return spec.split(', ').filter(Boolean).map((part) => {
    const lastColon = part.lastIndexOf(':');
    const position = parseInt(part.slice(lastColon + 1), 10) || 0;
    const rest = part.slice(0, lastColon);
    const midColon = rest.lastIndexOf(':');
    const type = rest.slice(midColon + 1);
    const name = rest.slice(0, midColon);
    return { name, type, position };
  });
}
```

parseValueSpec() — ligne 572

Fonction : `parse value spec` — voir le code ci-dessous.

```
function parseValueSpec(spec: string): ValueDef[] {
  return spec.split(', ').filter(Boolean).map((part) => {
    const lastColon = part.lastIndexOf(':');
    const position = parseInt(part.slice(lastColon + 1), 10) || 0;
    const value = part.slice(0, lastColon);
    return { value, position };
  });
}
```

loadOptionCache() — ligne 583

Description (commentaire d'origine) : Load existing product_options into cache (name → id)

```
async function loadOptionCache(): Promise<Map<string, string>> {
  const cache = new Map<string, string>();
  try {
    const res = await api.get('/product_options?display=full');
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    doc.querySelectorAll('product_option').forEach((opt) => {
      const id = opt.querySelector('id')?.textContent?.trim();
      const name = opt.querySelector('name language')?.textContent?.trim()
        ?? opt.querySelector('name')?.textContent?.trim();
      if (id && name) cache.set(name.toLowerCase(), id);
    });
  } catch { /* ignore */ }
  return cache;
}
```

loadOptionValueCache() — ligne 599

Description (commentaire d'origine) : Load existing product_option_values into cache ("optionId:valueName" → id)

```
async function loadOptionValueCache(): Promise<Map<string, string>> {
  const cache = new Map<string, string>();
  try {
    const res = await api.get('/product_option_values?display=full');
    const doc = new DOMParser().parseFromString(res.data, 'text/xml');
    doc.querySelectorAll('product_option_value').forEach((v) => {
      const id = v.querySelector('id')?.textContent?.trim();
      const groupId = v.querySelector('id_attribute_group')?.textContent?.trim();
      const name = v.querySelector('name language')?.textContent?.trim()
        ?? v.querySelector('name')?.textContent?.trim();
      if (id && groupId && name) cache.set(`${groupId}:${name.toLowerCase()}`, id);
    });
  }
}
```

```

    } catch { /* ignore */ }
    return cache;
}

```

getOrCreateOption() — ligne 614

Fonction : `get or create option` — voir le code ci-dessous.

```

async function getOrCreateOption(
  name: string,
  type: string,
  position: number,
  cache: Map<string, string>,
): Promise<string> {
  const key = name.toLowerCase();
  if (cache.has(key)) return cache.get(key)!;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <product_option>
    <name><language id="1"><![CDATA[${name}]]></language></name>
    <public_name><language id="1"><![CDATA[${name}]]></language></public_name>
    <group_type><![CDATA[${type} || 'select']]></group_type>
    <is_color_group><![CDATA[${type} === 'color' ? 1 : 0]]></is_color_group>
    <position><![CDATA[${position}]]></position>
  </product_option>
</prestashop>`;
  const id = await postEntity('/product_options', xml);
  cache.set(key, id);
  return id;
}

```

getOrCreateOptionValue() — ligne 637

Fonction : `get or create option value` — voir le code ci-dessous.

```

async function getOrCreateOptionValue(
  optionId: string,
  valueName: string,
  position: number,
  cache: Map<string, string>,
): Promise<string> {
  const key = `${optionId}:${valueName.toLowerCase()}`;
  if (cache.has(key)) return cache.get(key)!;
  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <product_option_value>
    <id_attribute_group><![CDATA[${optionId}]]></id_attribute_group>
    <name><language id="1"><![CDATA[${valueName}]]></language></name>
    <position><![CDATA[${position}]]></position>
  </product_option_value>
</prestashop>`;
  const id = await postEntity('/product_option_values', xml);
  cache.set(key, id);
  return id;
}

```

buildCombinationXml() — ligne 658

Fonction : `build combination xml` — voir le code ci-dessous.

```

function buildCombinationXml(row: CombinationRow, optionValueIds: string[]): string {
  const assocXml = optionValueIds
    .map((id) => `<product_option_value><id><![CDATA[${id}]]></id></product_option_value>`)
    .join('\n');
  return `<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">

```

```

<combination>
  <id_product><![CDATA[${row.productId}]]></id_product>
  <reference><![CDATA[${row.reference}]]></reference>
  <ean13><![CDATA[${/^d{13}$/.test(row.ean13) ? row.ean13 : ''}]]></ean13>
  <wholesale_price><![CDATA[${row.wholesalePrice || '0'}]]></wholesale_price>
  <price><![CDATA[${row.impactPrice || '0'}]]></price>
  <minimal_quantity><![CDATA[${row.minimalQuantity || '1'}]]></minimal_quantity>
  <default_on><![CDATA[${row.isDefault || '0'}]]></default_on>
  <available_date><![CDATA[${row.availableDate}]]></available_date>
  <associations>
    <product_option_values>
      ${assocXml}
    </product_option_values>
  </associations>
</combination>
</prestashop>;
}

```

importCombinations() — ligne 682

Fonction : `import combinations` — voir le code ci-dessous.

```

export async function importCombinations(
  file: File,
  onProgress?: ProgressCallback,
): Promise<ImportResult[]> {
  const [optionCache, optionValueCache] = await Promise.all([
    loadOptionCache(),
    loadOptionValueCache(),
  ]);

  const rows = parseCombinationCsv(await file.text());
  const results: ImportResult[] = [];

  for (let i = 0; i < rows.length; i++) {
    const row = rows[i];
    const label = `Produit ${row.productId} - ${row.valueSpec}`;
    try {
      const attrDefs = parseAttributeSpec(row.attributeSpec);
      const valueDefs = parseValueSpec(row.valueSpec);

      if (attrDefs.length !== valueDefs.length) {
        throw new Error('Nombre d\'attributs et de valeurs incohérent');
      }

      const optionValueIds: string[] = [];
      for (let j = 0; j < attrDefs.length; j++) {
        const attr = attrDefs[j];
        const val = valueDefs[j];
        const optId = await getOrCreateOption(attr.name, attr.type, attr.position, optionCache);
        const valId = await getOrCreateOptionValue(optId, val.value, val.position, optionValueCache);
        optionValueIds.push(valId);
      }

      const xml = buildCombinationXml(row, optionValueIds);
      const id = await postEntity('/combinations', xml);
      results.push({ rowIndex: i, label, success: true, id });
    } catch (err: any) {
      results.push({ rowIndex: i, label, success: false,
        error: err.response?.data ? extractXmlError(err.response.data) : err.message });
    }
    onProgress?.(i + 1, rows.length);
  }
  return results;
}

```

Fichier :

src/services/importAuditService.ts

Lignes : 244 • Fonctions détectées : 15

text() — ligne 7

Fonction : **text** — voir le code ci-dessous.

```
function text(el: Element, selector: string): string {
  return el.querySelector(selector)?.textContent?.trim() ?? '';
}
```

langText() — ligne 11

Fonction : **lang text** — voir le code ci-dessous.

```
function langText(el: Element, selector: string): string {
  const lang = el.querySelector(`${selector} > language`);
  if (lang?.textContent) return lang.textContent.trim();
  return text(el, selector);
}
```

parseList() — ligne 17

Fonction : **parse list** — voir le code ci-dessous.

```
function parseList(xml: string, tag: string): Element[] {
  const doc = new DOMParser().parseFromString(xml, 'text/xml');
  return Array.from(doc.querySelectorAll(tag));
}
```

withLimit() — ligne 22

Fonction : **with limit** — voir le code ci-dessous.

```
function withLimit(limit: number): string {
  return `0,${limit}`;
}
```

fetchProductsSample() — ligne 96

Fonction : **fetch products sample** — voir le code ci-dessous.

```
export async function fetchProductsSample(limit = 50): Promise<ProductAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,name,reference,price,id_tax_rules_group,active]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/products?${params.toString()}`);
  return parseList(res.data, 'product').map((el) => ({
    id: text(el, 'id'),
    name: langText(el, 'name'),
    reference: text(el, 'reference'),
    priceHt: text(el, 'price'),
    taxRulesGroupId: text(el, 'id_tax_rules_group'),
    active: text(el, 'active') === '1' ? 'yes' : 'no',
  }));
}
```

fetchCategoriesSample() — ligne 112

Fonction : `fetch categories sample` — voir le code ci-dessous.

```
export async function fetchCategoriesSample(limit = 50): Promise<CategoryAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,name]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/categories?${params.toString()}`);
  return parseList(res.data, 'category').map((el) => ({
    id: text(el, 'id'),
    name: langText(el, 'name'),
  }));
}
```

fetchCustomersSample() — ligne 124

Fonction : `fetch customers sample` — voir le code ci-dessous.

```
export async function fetchCustomersSample(limit = 50): Promise<CustomerAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,firstname,lastname,email]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/customers?${params.toString()}`);
  return parseList(res.data, 'customer').map((el) => ({
    id: text(el, 'id'),
    firstname: text(el, 'firstname'),
    lastname: text(el, 'lastname'),
    email: text(el, 'email'),
  }));
}
```

fetchAddressesSample() — ligne 138

Fonction : `fetch addresses sample` — voir le code ci-dessous.

```
export async function fetchAddressesSample(limit = 50): Promise<AddressAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,alias,city,postcode,id_country,id_customer]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/addresses?${params.toString()}`);
  return parseList(res.data, 'address').map((el) => ({
    id: text(el, 'id'),
    alias: text(el, 'alias'),
    city: text(el, 'city'),
    postcode: text(el, 'postcode'),
    countryId: text(el, 'id_country'),
    customerId: text(el, 'id_customer'),
  }));
}
```

fetchSuppliersSample() — ligne 154

Fonction : `fetch suppliers sample` — voir le code ci-dessous.

```
export async function fetchSuppliersSample(limit = 50): Promise<SupplierAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,name]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/suppliers?${params.toString()}`);
  return parseList(res.data, 'supplier').map((el) => ({
```

```

        id: text(el, 'id'),
        name: text(el, 'name'),
    }));
}

```

fetchBrandsSample() — ligne 166

Fonction : `fetch brands sample` — voir le code ci-dessous.

```

export async function fetchBrandsSample(limit = 50): Promise<BrandAudit[]> {
    const params = new URLSearchParams();
    params.set('display', '[id,name]');
    params.set('limit', withLimit(limit));
    params.set('sort', '[id_DESC]');
    const res = await api.get(`/manufacturers?${params.toString()}`);
    return parseList(res.data, 'manufacturer').map((el) => ({
        id: text(el, 'id'),
        name: text(el, 'name'),
    }));
}

```

fetchCombinationsSample() — ligne 178

Fonction : `fetch combinations sample` — voir le code ci-dessous.

```

export async function fetchCombinationsSample(limit = 50): Promise<CombinationAudit[]> {
    const params = new URLSearchParams();
    params.set('display', '[id,id_product,reference]');
    params.set('limit', withLimit(limit));
    params.set('sort', '[id_DESC]');
    const res = await api.get(`/combinations?${params.toString()}`);
    return parseList(res.data, 'combination').map((el) => ({
        id: text(el, 'id'),
        productId: text(el, 'id_product'),
        reference: text(el, 'reference'),
    }));
}

```

fetchStockSample() — ligne 191

Fonction : `fetch stock sample` — voir le code ci-dessous.

```

export async function fetchStockSample(limit = 50): Promise<StockAudit[]> {
    const params = new URLSearchParams();
    params.set('display', '[id,id_product,quantity]');
    params.set('filter[id_product_attribute]', '[0]');
    params.set('limit', withLimit(limit));
    params.set('sort', '[id_DESC]');
    const res = await api.get(`/stock_availability?${params.toString()}`);
    return parseList(res.data, 'stock_available').map((el) => ({
        id: text(el, 'id'),
        productId: text(el, 'id_product'),
        quantity: text(el, 'quantity'),
    }));
}

```

fetchTaxesSample() — ligne 205

Fonction : `fetch taxes sample` — voir le code ci-dessous.

```

export async function fetchTaxesSample(limit = 50): Promise<TaxAudit[]> {
    const params = new URLSearchParams();
    params.set('display', '[id,name,rate]');
    params.set('limit', withLimit(limit));
}

```

```

    params.set('sort', '[id_DESC]');
    const res = await api.get(`/taxes?${params.toString()}`);
    return parseList(res.data, 'tax').map((el) => ({
      id: text(el, 'id'),
      name: langText(el, 'name'),
      rate: text(el, 'rate'),
    }));
  }
}

```

fetchTaxRuleGroupsSample() — ligne 218

Fonction : `fetch tax rule groups sample` — voir le code ci-dessous.

```

export async function fetchTaxRuleGroupsSample(limit = 50): Promise<TaxRuleGroupAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,name]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/tax_rule_groups?${params.toString()}`);
  return parseList(res.data, 'tax_rule_group').map((el) => ({
    id: text(el, 'id'),
    name: text(el, 'name'),
  }));
}

```

fetchTaxRulesSample() — ligne 230

Fonction : `fetch tax rules sample` — voir le code ci-dessous.

```

export async function fetchTaxRulesSample(limit = 50): Promise<TaxRuleAudit[]> {
  const params = new URLSearchParams();
  params.set('display', '[id,id_tax_rules_group,id_tax,id_country]');
  params.set('limit', withLimit(limit));
  params.set('sort', '[id_DESC]');
  const res = await api.get(`/tax_rules?${params.toString()}`);
  return parseList(res.data, 'tax_rule').map((el) => ({
    id: text(el, 'id'),
    groupId: text(el, 'id_tax_rules_group'),
    taxId: text(el, 'id_tax'),
    countryId: text(el, 'id_country'),
  }));
}

```

Fichier : src/services/importValidationService.ts

Lignes : 165 • **Fonctions détectées :** 5

levenshtein() — ligne 46

Description (commentaire d'origine) : =====
 VALIDATION DES EN-TÊTES =====

```

function levenshtein(a: string, b: string): number {
  const m = a.length, n = b.length;
  const dp: number[][] = Array.from({ length: m + 1 }, (_, i) =>
    Array.from({ length: n + 1 }, (_, j) => (i === 0 ? j : j === 0 ? i : 0)));
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      dp[i][j] = a[i - 1] === b[j - 1]
    }
  }
}

```

```

        ? dp[i - 1][j - 1]
        : 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
    }
}
return dp[m][n];
}

```

validateHeaders() — ligne 66

Description (commentaire d'origine) : Vérifie que les en-têtes reçus correspondent aux colonnes attendues. Détecte : colonnes inconnues (avec suggestion si faute de frappe), colonnes obligatoires manquantes.

```

export function validateHeaders(receivedHeaders: string[], specs: ColumnSpec[]): string[] {
    const errors: string[] = [];
    const normalizedReceived = receivedHeaders.map(h => h.trim().toLowerCase());
    const allKnown = new Set(
        specs.flatMap(s => [s.canonical.toLowerCase(), ...s.aliases.map(a => a.toLowerCase())])
    );

    // Colonnes inconnues (avec détection de fautes de frappe)
    for (const h of receivedHeaders) {
        const hNorm = h.trim().toLowerCase();
        if (allKnown.has(hNorm)) continue;
        let bestMatch = '';
        let bestDist = Infinity;
        for (const k of allKnown) {
            const d = levenshtein(hNorm, k);
            if (d < bestDist) { bestDist = d; bestMatch = k; }
        }
        const suggestion = bestDist <= 3 ? ` - voulez-vous écrire "${bestMatch}" ?` : '';
        errors.push(`Colonne inconnue : "${h}"${suggestion} (en-tête ligne 1)`);
    }

    // Colonnes obligatoires manquantes
    for (const spec of specs) {
        if (!spec.required) continue;
        const allAliases = [spec.canonical.toLowerCase(), ...spec.aliases.map(a => a.toLowerCase())];
        if (!normalizedReceived.some(h => allAliases.includes(h))) {
            errors.push(`Colonne obligatoire manquante : "${spec.canonical}" (attendu dans l'en-tête, ligne 1)`);
        }
    }

    return errors;
}

```

resolveColumnIndex() — ligne 102

Description (commentaire d'origine) : =====
 RÉOLUTION D'INDEX DE COLONNE PAR NOM
 =====

```

export function resolveColumnIndex(header: string[], names: string[]): number {
    const normalized = header.map(h => h.trim().toLowerCase());
    for (const name of names) {
        const idx = normalized.indexOf(name.toLowerCase());
        if (idx >= 0) return idx;
    }
    return -1;
}

```

validateDateField() — ligne 122

Description (commentaire d'origine) : Valide une date avec message d'erreur enrichi. Retourne null si valide, sinon le message d'erreur.

```
export function validateDateField(
  raw: string,
  fieldName: string,
  lineNumber: number,
  parser: (s: string) => Date | null,
): string | null {
  if (!raw.trim()) return null;
  if (!parser(raw)) {
    return `Ligne ${lineNumber} - "${fieldName}" : date invalide "${raw}". Formats acceptés : ${DATE_FORMATS}`;
  }
  return null;
}
```

validatePositiveAmount() — ligne 144

Description (commentaire d'origine) : Vérifie qu'un montant est strictement positif (> 0), numérique et non vide. Accepte le format français (virgule comme séparateur décimal). Retourne null si valide, sinon le message d'erreur.

```
export function validatePositiveAmount(
  raw: string,
  fieldName: string,
  lineNumber: number,
  required = true,
): string | null {
  const trimmed = raw.trim();
  if (!trimmed) {
    return required
      ? `Ligne ${lineNumber} - "${fieldName}" : montant manquant (valeur obligatoire)`
      : null;
  }
  const value = Number(trimmed.replace(',', '.').replace('%', ''));
  if (isNaN(value)) {
    return `Ligne ${lineNumber} - "${fieldName}" : valeur non numérique "${raw}"`;
  }
  if (value <= 0) {
    return `Ligne ${lineNumber} - "${fieldName}" : le montant doit être strictement positif (valeur reçue : "${raw}")`;
  }
  return null;
}
```

Composants — Layouts & Routing

Fichier : `src/components/AppLayout.tsx`

Lignes : 326 • Fonctions détectées : 17

IconGrid() — ligne 7

Description	(commentaire	d'origine)	:	■■	Icônes	SVG	inline
<pre>const IconGrid = () => (<svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL <rect x="3" y="3" width="7" height="7" rx="1"/><rect x="14" y="3" width="7" height="7" rx="1"/> <rect x="3" y="14" width="7" height="7" rx="1"/><rect x="14" y="14" width="7" height="7" rx="1"/> </svg>)</pre>							

IconBox() — ligne 13

Fonction : `icon box` — voir le code ci-dessous.

```
const IconBox = () => (  
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL  
    <path d="M21 16V8a2 2 0 0-1-1.73l-7-4a2 2 0 0-2 0l-7 4a2 2 0 003 8v8a2 2 0 01 1.73l7 4a2 2 0 02 0l7-4  
    <polyline points="3.27 6.96 12 12.01 20.73 6.96"/><line x1="12" y1="22.08" x2="12" y2="12"/>  
  </svg>  
)
```

IconCart() — ligne 19

Fonction : `icon cart` — voir le code ci-dessous.

```
const IconCart = () => (  
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL  
    <circle cx="9" cy="21" r="1"/><circle cx="20" cy="21" r="1"/>  
    <path d="M1 1h4l2.68 13.39a2 2 0 02 1.61h9.72a2 2 0 02-1.61L23 6H6"/>  
  </svg>  
)
```

IconUsers() — ligne 25

Fonction : `icon users` — voir le code ci-dessous.

```
const IconUsers = () => (  
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL  
    <path d="M17 21v-2a4 4 0 0-4-4H5a4 4 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/>  
    <path d="M23 21v-2a4 4 0 0-3-3.87"/><path d="M16 3.13a4 4 0 010 7.75"/>  
  </svg>  
)
```

IconPuzzle() — ligne 31

Fonction : `icon puzzle` — voir le code ci-dessous.

```
const IconPuzzle = () => (  
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
```

```

    <path d="M20.59 13.41l-7.17 7.17a2 2 0 01-2.83 0L2 12V2h10l8.59 8.59a2 2 0 010 2.82z"/>
    <line x1="7" y1="7" x2="7.01" y2="7"/>
  </svg>
);

```

IconChart() — ligne 37

Fonction : `icon chart` — voir le code ci-dessous.

```

const IconChart = () => (
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/>
    <line x1="6" y1="20" x2="6" y2="14"/><line x1="2" y1="20" x2="22" y2="20"/>
  </svg>
);

```

IconShop() — ligne 43

Fonction : `icon shop` — voir le code ci-dessous.

```

const IconShop = () => (
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <path d="M6 2L3 6v14a2 2 0 002 2h14a2 2 0 002-2V6l-3-4z"/>
    <line x1="3" y1="6" x2="21" y2="6"/>
    <path d="M16 10a4 4 0 01-8 0"/>
  </svg>
);

```

IconTrash() — ligne 50

Fonction : `icon trash` — voir le code ci-dessous.

```

const IconTrash = () => (
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H5 6"/>
    <path d="M10 11v6"/><path d="M14 11v6"/><path d="M9 6V4a1 1 0 011-1h4a1 1 0 011 1v2"/>
  </svg>
);

```

IconLayers() — ligne 56

Fonction : `icon layers` — voir le code ci-dessous.

```

const IconLayers = () => (
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <polygon points="12 2 2 7 12 12 22 7 12 2"/>
    <polyline points="2 17 12 22 22 17"/>
    <polyline points="2 12 12 17 22 12"/>
  </svg>
);

```

IconSettings() — ligne 63

Fonction : `icon settings` — voir le code ci-dessous.

```

const IconSettings = () => (
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <circle cx="12" cy="12" r="3"/>
    <path d="M19.4 15a1.65 1.65 0 00.33 1.82l.06.06a2 2 0 010 2.83 2 2 0 01-2.83 0l-.06-.06a1.65 1.65 0 00-1.
  </svg>
);

```

IconLogout() — ligne 69

Fonction : `icon logout` — voir le code ci-dessous.

```
const IconLogout = () => (
  <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <path d="M9 21H5a2 2 0 01-2-2V5a2 2 0 012-2h4"/>
    <polyline points="16 17 21 12 16 7"/><line x1="21" y1="12" x2="9" y2="12"/>
  </svg>
);
```

IconMenu() — ligne 75

Fonction : `icon menu` — voir le code ci-dessous.

```
const IconMenu = () => (
  <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeL
    <line x1="3" y1="12" x2="21" y2="12"/><line x1="3" y1="6" x2="21" y2="6"/><line x1="3" y1="18" x2="21" y2
  </svg>
);
```

IconChevron() — ligne 80

Fonction : `icon chevron` — voir le code ci-dessous.

```
const IconChevron = ({ open }: { open: boolean }) => (
  <svg
    width="14" height="14" viewBox="0 0 24 24" fill="none"
    stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
    style={{ transition: 'transform 0.2s', transform: open ? 'rotate(90deg)' : 'rotate(0deg)', flexShrink: 0 }}
  >
    <polyline points="9 18 15 12 9 6"/>
  </svg>
);
```

isModuleActive() — ligne 145

Description (commentaire d'origine) :

```
function isModuleActive(mod: NavModule, pathname: string): boolean {
  if (mod.children) {
    return mod.children.some((child) =>
      child.end ? pathname === child.path : pathname.startsWith(child.path)
    );
  }
  return mod.end ? pathname === mod.path : pathname.startsWith(mod.path);
}
```

AppLayout() — ligne 160

Fonction : `app layout` — voir le code ci-dessous.

```
const AppLayout: React.FC<AppLayoutProps> = ({ children, pageTitle }) => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const { pathname } = useLocation();
  const [collapsed, setCollapsed] = useState(false);

  // Groupes ouverts – initialisés avec les groupes actifs selon la route courante
  const [openGroups, setOpenGroups] = useState<Set<string>>(() => {
    const open = new Set<string>();
    for (const mod of navModules) {
```

```

    if (mod.children && isModuleActive(mod, pathname)) open.add(mod.label);
  }
  return open;
});

// Auto-ouvre le groupe parent lors d'une navigation vers un sous-item
useEffect(() => {
  for (const mod of navModules) {
    if (mod.children && isModuleActive(mod, pathname)) {
      setOpenGroups((prev) => new Set([...prev, mod.label]));
    }
  }
}, [pathname]);

const toggleGroup = (label: string) => {
  setOpenGroups((prev) => {
    const next = new Set(prev);
    next.has(label) ? next.delete(label) : next.add(label);
    return next;
  });
};

const handleLogout = async () => {
  await logout();
  navigate('/login');
};

const sections = navModules.reduce<Record<string, NavModule[]>>((acc, mod) => {
  const s = mod.section ?? 'Autre';
  if (!acc[s]) acc[s] = [];
  acc[s].push(mod);
  return acc;
}, {});

const userInitial = user?.email?.[0]?.toUpperCase() ?? '?';

return (
  <div className={`layout${collapsed ? ' layout--collapsed' : ''}`}>
    { /* ■■■ SIDEBAR ■■■ */ }
    <aside className="sidebar">
      <div className="sidebar-brand">
        <span className="sidebar-logo">PS</span>
        {!collapsed && <span className="sidebar-brand-name">PrestaShop</span>}
      </div>

      <nav className="sidebar-nav">
        {Object.entries(sections).map(([section, items]) => (
          <div key={section} className="nav-section">
            {!collapsed && <span className="nav-section-label">{section}</span>}

            {items.map((mod) => {
              if (mod.disabled) {
                return (
                  <span key={mod.label} className="nav-item nav-item--disabled" title={mod.label}>
                    <span className="nav-item-icon">{mod.icon}</span>
                    {!collapsed && <span className="nav-item-label">{mod.label}</span>}
                    {!collapsed && <span className="nav-badge-soon">Bientôt</span>}
                  </span>
                );
              }
            })}

            <div key={mod.label} className="nav-group">
              { /* Parent – bouton toggle (rétractable) */ }
              <button
                className={`nav-item nav-item--parent${active ? ' nav-item--active' : ''}`}
                onClick={() => toggleGroup(mod.label)}
              >

```

```

        title={mod.label}
      >
        <span className="nav-item-icon">{mod.icon}</span>
        {!collapsed && <span className="nav-item-label">{mod.label}</span>}
        {!collapsed && <IconChevron open={isOpen} />}
      </button>

      {/* Sous-menu – visible quand groupe ouvert et sidebar non réduite */}
      {isOpen && !collapsed && (
        <div className="nav-submenu">
          {mod.children.map((child) => (
            <NavLink
              key={child.path}
              to={child.path}
              end={child.end}
              className={({ isActive }) =>
                `nav-subitem${isActive ? ' nav-subitem--active' : ''}`
              }
            >
              <span className="nav-subitem-dot" />
              {child.label}
            </NavLink>
          ))}
        </div>
      )}
    </div>
  );
}

return (
  <NavLink
    key={mod.label}
    to={mod.path}
    end={mod.end}
    title={mod.label}
    className={({ isActive }) => `nav-item${isActive ? ' nav-item--active' : ''}`}
  >
    <span className="nav-item-icon">{mod.icon}</span>
    {!collapsed && <span className="nav-item-label">{mod.label}</span>}
  </NavLink>
);
  )}
</div>
)}
</nav>

<div className="sidebar-footer">
  <div className="sidebar-user">
    <div className="sidebar-avatar">{userInitial}</div>
    {!collapsed && (
      <div className="sidebar-user-info">
        <span className="sidebar-user-email" title={user?.email}>{user?.email}</span>
        <span className="sidebar-user-role">Administrateur</span>
      </div>
    )}
  </div>
  <button className="sidebar-logout-btn" onClick={handleLogout} title="Déconnexion">
    <IconLogout />
    {!collapsed && <span>Déconnexion</span>}
  </button>
</div>
</aside>

{/* ■■ CONTENU PRINCIPAL ■■ */}
<div className="layout-body">
  <header className="topbar">
    <button className="topbar-toggle" onClick={() => setCollapsed(!collapsed)} aria-label="Menu">
      <IconMenu />
    </button>
    {pageTitle && <h1 className="topbar-title">{pageTitle}</h1>}
    <div className="topbar-right">

```

}

toggleGroup() — ligne 184

Fonction : `toggle group` — voir le code ci-dessous.

}

handleLogout() — ligne 192

Fonction : `handle_logout` — voir le code ci-dessous.

}

Fichier :

src/components/ShopLayout.tsx

Lignes : 157 • Fonctions détectées : 2

ShopLayout() — ligne 9

Fonction : `shop layout` — voir le code ci-dessous.

```

<div className="mx-auto grid max-w-7xl items-center gap-4 px-6 py-4 lg:grid-cols-[auto,1fr,auto]">
  <Link to="/shop" className="flex items-center gap-3 text-slate-900 transition hover:text-slate-700">
    <span className="inline-flex h-10 w-10 items-center justify-center rounded-2xl border border-slate-200">
      <svg className="h-5 w-5" viewBox="0 0 32 32" fill="none" stroke="currentColor" strokeWidth="2">
        <path d="M2 4h28v8H2z" />
        <path d="M4 12v12c0 2 1 4 3 4h18c2 0 3 2 3 4v12" />
        <line x1="12" y1="12" x2="12" y2="28" />
        <line x1="20" y1="12" x2="20" y2="28" />
      </svg>
    </span>
    <span className="flex flex-col leading-none">
      <span className="font-display text-xl tracking-tight">ShopPro</span>
      <span className="text-[11px] text-slate-500">Boutique moderne</span>
    </span>
  </Link>

  <div className="relative hidden w-full max-w-xl lg:block">
    <svg className="absolute left-4 top-1/2 h-4 w-4 -translate-y-1/2 text-slate-400" viewBox="0 0 24 24">
      <circle cx="11" cy="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/>
    </svg>
    <input
      type="search"
      placeholder="Rechercher un produit"
      className="w-full rounded-full border border-slate-200 bg-slate-50 px-11 py-3 text-sm text-slate-900"
    />
  </div>

  <div className="flex items-center gap-2">
    {customer ? (
      <>
        <div className="hidden flex-col items-end gap-1 text-right sm:flex">
          <span className="text-[10px] uppercase tracking-[0.2em] text-slate-400">Bienvenue</span>
          <span className="font-display text-sm">{customer.firstname}</span>
        </div>
        <Link
          to="/shop/my-orders"
          className="hidden rounded-full border border-slate-200 px-4 py-2 text-xs font-semibold text-slate-900"
        >
          Mes commandes
        </Link>
        <button
          className="rounded-full bg-slate-900 px-4 py-2 text-xs font-semibold text-white shadow-sm transition"
          onClick={handleLogout}
        >
          Déconnexion
        </button>
      </>
    ) : (
      <Link
        to="/shop/auth"
        className="rounded-full bg-slate-900 px-4 py-2 text-xs font-semibold text-white shadow-sm transition"
      >
        Connexion
      </Link>
    )}

    <Link
      to="/shop/cart"
      className="relative inline-flex h-10 w-10 items-center justify-center rounded-full border border-slate-200"
      aria-label="Voir le panier"
    >
      <svg className="h-5 w-5" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
        <circle cx="9" cy="21" r="1" />
        <circle cx="20" cy="21" r="1" />
        <path d="M1 1h4l2.68 13.39a2 2 0 0 2 1.61h9.72a2 2 0 0 2 1.61L23 6H6" />
      </svg>
      {totalItems > 0 && (
        <span className="absolute -right-1 -top-1 inline-flex h-5 min-w-[20px] items-center justify-center text-xs">
          {totalItems}
        </span>
      )}
    </Link>
  </div>

```


Fichier :

src/components/ProtectedRoute.tsx

Lignes : 31 • Fonctions détectées : 1

ProtectedRoute() — ligne 8

Fonction : `protected route` — voir le code ci-dessous.

```
const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children }) => {
  const { user, isLoading } = useAuth();
  const location = useLocation();

  if (isLoading) {
    return (
      <div className="loading-container">
        <div className="spinner"></div>
        <p>Vérification de la session...</p>
      </div>
    );
  }

  if (!user) {
    // Mémoire l'URL cible pour rediriger après connexion
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  return <>{children}</>;
}
```

Composants — Authentification

Fichier : `src/components/Login.tsx`

Lignes : 110 • Fonctions détectées : 2

Login() — ligne 6

Fonction : `login` — voir le code ci-dessous.

```
const Login: React.FC = () => {
  const DEFAULT_EMAIL = import.meta.env.VITE_DEFAULT_EMAIL ?? 'admin@prestashop.com';
  const DEFAULT_PWD = import.meta.env.VITE_DEFAULT_PWD ?? '';

  const [email, setEmail] = useState(DEFAULT_EMAIL);
  const [password, setPassword] = useState(DEFAULT_PWD);
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

  const { user, login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  // Si déjà connecté, rediriger vers la page d'origine ou l'accueil
  useEffect(() => {
    if (user) {
      const from = (location.state as { from?: Location })?.from?.pathname || '/products';
      navigate(from, { replace: true });
    }
  }, [user, navigate, location]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);
    setLoading(true);

    try {
      await login(email, password);
      // La redirection est gérée par le useEffect ci-dessus
    } catch (err) {
      if (err instanceof AuthError) {
        setError(err.message);
      } else {
        setError('Une erreur inattendue est survenue. Veuillez réessayer.');
```

```

    })

    <form onSubmit={handleSubmit} className="login-form" noValidate>
      <div className="login-field">
        <label htmlFor="email">Adresse e-mail</label>
        <input
          id="email"
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
          placeholder="email@exemple.com"
          autoComplete="email"
          autoFocus
          disabled={loading}
        />
      </div>

      <div className="login-field">
        <label htmlFor="password">Mot de passe</label>
        <input
          id="password"
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
          placeholder="....."
          autoComplete="current-password"
          disabled={loading}
        />
      </div>

      <button type="submit" className="login-btn" disabled={loading}>
        {loading ? (
          <>
            <span className="login-spinner" aria-hidden="true"></span>
            Connexion en cours...
          </>
        ) : (
          'Se connecter'
        )}
      </button>
    </form>
  </div>
</div>
);
}

```

handleSubmit() — ligne 27

Fonction : `handle submit` — voir le code ci-dessous.

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setError(null);
  setLoading(true);

  try {
    await login(email, password);
    // La redirection est gérée par le useEffect ci-dessus
  } catch (err) {
    if (err instanceof AuthError) {
      setError(err.message);
    } else {
      setError('Une erreur inattendue est survenue. Veuillez réessayer.');
```

```
}
```

Fichier :

src/components/UserSelectPage.tsx

Lignes : 256 • Fonctions détectées : 5

formatDisplayName() — ligne 11

Fonction : `format display name` — voir le code ci-dessous.

```
function formatDisplayName(user: CustomerSummary): string {
  const name = `${user.firstname} ${user.lastname}`.trim();
  if (name) return name;
  if (user.email) return user.email;
  return `Utilisateur #${user.id}`;
}
```

initialsFrom() — ligne 18

Fonction : `initials from` — voir le code ci-dessous.

```
function initialsFrom(user: CustomerSummary): string {
  const first = user.firstname?.trim().charAt(0) ?? '';
  const last = user.lastname?.trim().charAt(0) ?? '';
  const email = user.email?.trim().charAt(0) ?? '';
  return (first + last || email || user.id.charAt(0)).toUpperCase();
}
```

UserSelectPage() — ligne 25

Fonction : `user select page` — voir le code ci-dessous.

```
const UserSelectPage: React.FC = () => {
  const navigate = useNavigate();
  const { setCustomer } = useCustomer();

  const [users, setUsers] = useState<CustomerSummary[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [selectingId, setSelectingId] = useState<string | null>(null);

  useEffect(() => {
    let active = true;
    setLoading(true);
    setError(null);

    fetchCustomerList()
      .then((list) => {
        if (!active) return;
        setUsers(list);
      })
      .catch(() => {
        if (!active) return;
        setError('Impossible de charger les utilisateurs.');
```

```
    })
    .finally(() => {
      if (!active) return;
      setLoading(false);
    });
  });

  return () => {
```

```

    active = false;
  };
}, []);

const sortedUsers = useMemo(() => {
  return [...users].sort((a, b) => {
    const nameA = formatDisplayName(a).toLowerCase();
    const nameB = formatDisplayName(b).toLowerCase();
    return nameA.localeCompare(nameB);
  });
}, [users]);

const handleSelectUser = async (user: CustomerSummary) => {
  setSelectingId(user.id);
  setError(null);
  try {
    const secureKey = await fetchSecureKey(user.id);
    setCustomer({
      id: user.id,
      email: user.email,
      firstname: user.firstname,
      lastname: user.lastname,
      secureKey,
    });
    navigate('/shop', { replace: true });
  } catch {
    setError('Impossible de demarrer la session.');
```

} finally {
 setSelectingId(null);
 }
 }
};

const handleAnonymous = () => {
 setCustomer(null);
 navigate('/shop', { replace: true });
};

const showEmpty = !loading && sortedUsers.length === 0 && !error;

return (
 <div className="relative min-h-screen overflow-hidden bg-gradient-to-br from-slate-50 via-white to-amber-50">
 <div className="pointer-events-none absolute -top-24 right-0 h-72 w-72 rounded-full bg-emerald-200/40 b">
 <div className="pointer-events-none absolute -bottom-32 left-12 h-72 w-72 rounded-full bg-amber-200/40 b">
 <div className="relative mx-auto flex max-w-6xl flex-col gap-10 px-6 py-12">
 <header className="flex flex-col gap-4">
 <div className="flex flex-wrap items-center justify-between gap-4">
 <div>
 <p className="text-xs font-semibold uppercase tracking-[0.3em] text-emerald-500">
 Demarrage rapide
 </p>
 <h1 className="mt-3 text-3xl font-semibold text-slate-900 sm:text-4xl">
 Choisir un utilisateur
 </h1>
 </div>
 <div>
 <Link
 to="/login"
 className="rounded-full border border-slate-300 bg-white px-4 py-2 text-xs font-semibold text-s">
 >
 Accés back-office
 </Link>
 </div>
 </div>
 <p className="max-w-2xl text-base text-slate-600">
 Sélectionnez un compte pour ouvrir la session ou continuez en mode anonyme.
 Vous pourrez toujours changer plus tard.
 </p>
 </header>

 {error && (
 <div className="rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-sm text-rose-700">
 {error}
 </div>
)}
 </div>
 </div>
 </div>
 </div>
);

```

    </div>
  )}

  <div className="grid gap-8 lg:grid-cols-[1.25fr,0.75fr]">
    <section className="space-y-6">
      <div className="flex items-center justify-between">
        <h2 className="text-lg font-semibold text-slate-900">Utilisateurs disponibles</h2>
        <span className="text-xs font-semibold uppercase tracking-[0.2em] text-slate-400">
          {loading ? 'Chargement' : `${sortedUsers.length} utilisateur(s)`}
        </span>
      </div>
    </div>

    {loading && (
      <div className="grid gap-4 sm:grid-cols-2">
        {skeletonItems.map((item) => (
          <div
            key={item}
            className="animate-pulse rounded-2xl border border-slate-200 bg-white p-4 shadow-sm"
          >
            <div className="flex items-center gap-3">
              <div className="h-12 w-12 rounded-2xl bg-slate-100" />
              <div className="flex-1 space-y-2">
                <div className="h-3 w-2/3 rounded-full bg-slate-100" />
                <div className="h-3 w-1/2 rounded-full bg-slate-100" />
              </div>
            </div>
            <div className="mt-4 h-9 w-full rounded-full bg-slate-100" />
          </div>
        ))}
      </div>
    )}

    {!loading && sortedUsers.length > 0 && (
      <div className="grid gap-4 sm:grid-cols-2">
        {sortedUsers.map((user) => {
          const isSelected = selectingId === user.id;
          return (
            <button
              key={user.id}
              type="button"
              onClick={() => handleSelectUser(user)}
              disabled={!isSelected}
              className="group flex h-full flex-col gap-4 rounded-2xl border border-slate-200 bg-white"
            >
              <div className="flex items-center gap-3">
                <div className="flex h-12 w-12 items-center justify-center rounded-2xl bg-emerald-100">
                  {initialsFrom(user)}
                </div>
                <div>
                  <p className="text-base font-semibold text-slate-900">
                    {formatDisplayName(user)}
                  </p>
                  <p className="text-sm text-slate-500">{user.email} || ID ${user.id}</p>
                </div>
              </div>
              <div className="mt-auto flex items-center justify-between">
                <span className="text-xs font-semibold uppercase tracking-[0.2em] text-slate-400">
                  {isSelected ? 'Ouverture...' : 'Connexion rapide'}
                </span>
                <span className="rounded-full border border-slate-200 px-3 py-1 text-xs font-semibold">
                  {isSelected ? 'Chargement' : 'Ouvrir'}
                </span>
              </div>
            </button>
          );
        })}
      </div>
    )}

    {showEmpty && (
      <div className="rounded-2xl border border-dashed border-slate-300 bg-white/70 p-6 text-sm text-slate-400">
        <p>Aucun utilisateur disponible.</p>
      </div>
    )}
  )}

```

```

    <p className="text-base font-semibold text-slate-900">Aucun utilisateur trouve</p>
    <p className="mt-2">
      Vous pouvez creer un compte via la page client ou importer des utilisateurs.
    </p>
    <div className="mt-4 flex flex-wrap gap-3">
      <Link
        to="/shop/auth"
        className="rounded-full bg-slate-900 px-4 py-2 text-xs font-semibold text-white transition
      >
        Creer un compte
      </Link>
      <Link
        to="/import"
        className="rounded-full border border-slate-300 bg-white px-4 py-2 text-xs font-semibold
      >
        Importer des utilisateurs
      </Link>
    </div>
  </div>
)}
</section>

<aside className="space-y-6">
  <div className="rounded-3xl border border-emerald-200 bg-gradient-to-br from-emerald-500 via-emera
    <div className="flex items-center justify-between">
      <p className="text-xs font-semibold uppercase tracking-[0.3em] text-emerald-100">Option rapide
      <span className="rounded-full border border-emerald-200/60 px-3 py-1 text-xs font-semibold tex
        Highlight
      </span>
    </div>
    <h3 className="mt-4 text-2xl font-semibold">Utilisateur anonyme</h3>
    <p className="mt-3 text-sm text-emerald-50/90">
      Accédez a la boutique sans compte. Vous pourrez finaliser plus tard si besoin.
    </p>
    <button
      type="button"
      onClick={handleAnonymous}
      className="mt-6 w-full rounded-full bg-white/95 px-4 py-3 text-sm font-semibold text-emerald-
    >
      Continuer sans compte
    </button>
  </div>

  <div className="rounded-2xl border border-slate-200 bg-white/80 p-5 text-sm text-slate-600 shadow
    <p className="text-base font-semibold text-slate-900">Pourquoi choisir un compte ?</p>
    <ul className="mt-3 space-y-2">
      <li>Suivi des commandes en un clic</li>
      <li>Synchronisation du panier avec PrestaShop</li>
      <li>Checkout plus rapide avec vos adresses</li>
    </ul>
  </div>
</aside>
</div>
</div>
</div>
);
}

```

handleSelectUser() — ligne 66

Fonction : `handle select user` — voir le code ci-dessous.

```

const handleSelectUser = async (user: CustomerSummary) => {
  setSelectingId(user.id);
  setError(null);
  try {
    const secureKey = await fetchSecureKey(user.id);
    setCustomer({
      id: user.id,

```

```

        email: user.email,
        firstname: user.firstname,
        lastname: user.lastname,
        secureKey,
    });
    navigate('/shop', { replace: true });
  } catch {
    setError('Impossible de demarrer la session.');
```

handleAnonymous() — ligne 86

Fonction : `handle anonymous` — voir le code ci-dessous.

```

const handleAnonymous = () => {
  setCustomer(null);
  navigate('/shop', { replace: true });
}
```

Fichier :

`src/components/CustomerAuthPage.tsx`

Lignes : 201 • Fonctions détectées : 3

CustomerAuthPage() — ligne 8

Fonction : `customer auth page` — voir le code ci-dessous.

```

const CustomerAuthPage: React.FC = () => {
  const { customer, setCustomer } = useCustomer();
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();
  const next = searchParams.get('next') ?? '/shop';

  const [tab, setTab] = useState<Tab>('login');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // Login fields
  const [loginEmail, setLoginEmail] = useState('');

  // Register fields
  const [regFirstname, setRegFirstname] = useState('');
  const [regLastname, setRegLastname] = useState('');
  const [regEmail, setRegEmail] = useState('');
  const [regPasswd, setRegPasswd] = useState('');

  useEffect(() => {
    if (customer) navigate(next, { replace: true });
  }, [customer, navigate, next]);

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!loginEmail.trim()) { setError('Veuillez saisir votre email.');
```

```

        return;
    }
    setCustomer(found);
} catch {
    setError('Erreur de connexion. Veuillez réessayer.');
```

```

    } finally {
        setLoading(false);
    }
};

const handleRegister = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!regFirstname.trim() || !regLastname.trim() || !regEmail.trim() || !regPasswd.trim()) {
        setError('Tous les champs sont obligatoires.');
```

```

        return;
    }
    if (regPasswd.length < 5) {
        setError('Le mot de passe doit comporter au moins 5 caractères.');
```

```

        return;
    }
    setLoading(true);
    setError(null);
    try {
        const existing = await findCustomerByEmail(regEmail.trim());
        if (existing) {
            setError('Un compte existe déjà avec cet email. Connectez-vous.');
```

```

            return;
        }
        const newCustomer = await registerCustomer({
            email: regEmail.trim(),
            firstname: regFirstname.trim(),
            lastname: regLastname.trim(),
            passwd: regPasswd,
        });
        setCustomer(newCustomer);
    } catch (err: unknown) {
        const msg = err instanceof Error ? err.message : 'Erreur lors de la création du compte.';
        setError(msg);
    } finally {
        setLoading(false);
    }
};

return (
    <div className="auth-page">
        <div className="auth-card">
            <div className="auth-brand">
                <Link to="/shop" className="auth-back"><← Retour à la boutique</Link>
                <h1 className="auth-title">Mon compte</h1>
            </div>

            {/ * ■■ Tabs ■■ */}
            <div className="auth-tabs">
                <button
                    className={`auth-tab${tab === 'login' ? ' auth-tab--active' : ''}`}
                    onClick={() => { setTab('login'); setError(null); }}
                >Se connecter</button>
                <button
                    className={`auth-tab${tab === 'register' ? ' auth-tab--active' : ''}`}
                    onClick={() => { setTab('register'); setError(null); }}
                >Créer un compte</button>
            </div>

            {error && <div className="auth-error">{error}</div>}

            {/ * ■■ Login ■■ */}
            {tab === 'login' && (
                <form className="auth-form" onSubmit={handleLogin}>
                    <div className="auth-field">
                        <label className="auth-label">Adresse email</label>
                        <input
```

```

        type="email"
        className="auth-input"
        placeholder="votre@email.com"
        value={loginEmail}
        onChange={e => setLoginEmail(e.target.value)}
        autoFocus
        required
    />
</div>
<p className="auth-hint">Authentification via l'API PrestaShop – aucun mot de passe requis.</p>
<button type="submit" className="auth-submit" disabled={loading}>
    {loading ? 'Connexion...' : 'Se connecter'}
</button>
<p className="auth-switch">
    Pas encore de compte ?{' '}
    <button type="button" className="auth-switch-link" onClick={() => { setTab('register'); setError('') }}>
        Créer un compte
    </button>
</p>
</form>
)}

{/* ■■ Register ■■ */}
{tab === 'register' && (
    <form className="auth-form" onSubmit={handleRegister}>
        <div className="auth-row">
            <div className="auth-field">
                <label className="auth-label">Prénom</label>
                <input
                    type="text"
                    className="auth-input"
                    placeholder="Jean"
                    value={regFirstname}
                    onChange={e => setRegFirstname(e.target.value)}
                    required
                />
            </div>
            <div className="auth-field">
                <label className="auth-label">Nom</label>
                <input
                    type="text"
                    className="auth-input"
                    placeholder="Dupont"
                    value={regLastname}
                    onChange={e => setRegLastname(e.target.value)}
                    required
                />
            </div>
        </div>
        <div className="auth-field">
            <label className="auth-label">Adresse email</label>
            <input
                type="email"
                className="auth-input"
                placeholder="votre@email.com"
                value={regEmail}
                onChange={e => setRegEmail(e.target.value)}
                required
            />
        </div>
        <div className="auth-field">
            <label className="auth-label">Mot de passe</label>
            <input
                type="password"
                className="auth-input"
                placeholder="5 caractères minimum"
                value={regPasswd}
                onChange={e => setRegPasswd(e.target.value)}
                minLength={5}
                required
            />
        </div>
    </form>
)}

```

```

    </div>
    <button type="submit" className="auth-submit" disabled={loading}>
      {loading ? 'Création...' : 'Créer mon compte'}
    </button>
    <p className="auth-switch">
      Déjà un compte ?{' '}
      <button type="button" className="auth-switch-link" onClick={() => { setTab('login'); setError(n
        Se connecter
      </button>
    </p>
  </form>
  )}
</div>
</div>
);
};

export default CustomerAuthPage;

```

handleLogin() — ligne 31

Fonction : `handle login` — voir le code ci-dessous.

```

const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!loginEmail.trim()) { setError('Veuillez saisir votre email.');
```

handleRegister() — ligne 50

Fonction : `handle register` — voir le code ci-dessous.

```

const handleRegister = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!regFirstname.trim() || !regLastname.trim() || !regEmail.trim() || !regPasswd.trim()) {
    setError('Tous les champs sont obligatoires.');
```

```
        passwd: regPasswd,  
    });  
    setCustomer(newCustomer);  
} catch (err: unknown) {  
    const msg = err instanceof Error ? err.message : 'Erreur lors de la création du compte.';  
    setError(msg);  
} finally {  
    setLoading(false);  
}  
}
```

Composants — Produits (BO)

Fichier :

`src/components/ProductList.tsx`

Lignes : 342 • Fonctions détectées : 9

`ProductList()` — ligne 20

Fonction : `product list` — voir le code ci-dessous.

```
const ProductList: React.FC = () => {
  const [products, setProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [filters, setFilters] = useState<Filters>({
    idMin: '',
    idMax: '',
    name: '',
    reference: '',
    category: '',
    priceMin: '',
    priceMax: '',
    quantityMin: '',
    quantityMax: '',
    status: 'all',
  });
  const [appliedFilters, setAppliedFilters] = useState<ProductFilters>({});

  const navigate = useNavigate();

  const loadProducts = async (filtersToApply: ProductFilters) => {
    try {
      setLoading(true);
      const productsData = await productService.getAllProducts(filtersToApply);
      setProducts(productsData);
      setError(null);
    } catch (err) {
      setError(
        'Erreur lors du chargement des produits : ' +
        (err as Error).message
      );
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    loadProducts({});
  }, []);

  const toNumber = (value: string): number | undefined => {
    const trimmed = value.trim();
    if (!trimmed) return undefined;
    const parsed = Number(trimmed);
    return Number.isNaN(parsed) ? undefined : parsed;
  };

  const toText = (value: string): string | undefined => {
    const trimmed = value.trim();
    return trimmed ? trimmed : undefined;
  };
};
```

```

const buildFilterPayload = (): ProductFilters => ({
  idMin: toNumber(filters.idMin),
  idMax: toNumber(filters.idMax),
  name: toText(filters.name),
  reference: toText(filters.reference),
  categoryId: toNumber(filters.category),
  priceMin: toNumber(filters.priceMin),
  priceMax: toNumber(filters.priceMax),
  quantityMin: toNumber(filters.quantityMin),
  quantityMax: toNumber(filters.quantityMax),
  active: filters.status === 'all' ? undefined : filters.status === 'active',
});

const handleDeleteProduct = async (productId: string) => {
  if (window.confirm("Êtes-vous sûr de vouloir supprimer ce produit ?")) {
    try {
      await productService.deleteProduct(productId);
      loadProducts(appliedFilters);
    } catch (error) {
      setError("Erreur lors de la suppression du produit.");
      console.error(error);
    }
  }
};

const formatPrice = (product: Product): string => {
  return new Intl.NumberFormat('fr-FR', {
    style: 'currency',
    currency: 'EUR',
  }).format(product.price);
};

const handleSearch = (event: React.FormEvent<HTMLFormElement>) => {
  event.preventDefault();
  const payload = buildFilterPayload();
  setAppliedFilters(payload);
  loadProducts(payload);
};

const handleAddProduct = () => {
  navigate('/products/add');
};

if (loading) {
  return (
    <div className="loading-container">
      <div className="spinner"></div>
      <p>Chargement des produits...</p>
    </div>
  );
}

if (error) {
  return (
    <div className="error-container">
      <p className="error-message">{error}</p>
      <button onClick={() => loadProducts(appliedFilters)} className="retry-button">
        Réessayer
      </button>
    </div>
  );
}

return (
  <div className="product-list-container">
    <div className="products-header">
      <h1>Liste des Produits ({products.length})</h1>

      <div className="header-actions">
        <button onClick={() => loadProducts(appliedFilters)} className="refresh-button">
          Actualiser
        </button>
      </div>
    </div>
  </div>
);

```

```

    </button>

    <button onClick={handleAddProduct} className="add-button">
      Ajouter un produit
    </button>
  </div>
</div>

<form className="product-filters" onSubmit={handleSearch}>
  <div className="product-filters-row">
    <div className="filter-field filter-field--range">
      <label>ID</label>
      <div className="filter-range">
        <input
          type="number"
          placeholder="Min."
          value={filters.idMin}
          onChange={(e) => setFilters((prev) => ({ ...prev, idMin: e.target.value })))}
        />
        <input
          type="number"
          placeholder="Max."
          value={filters.idMax}
          onChange={(e) => setFilters((prev) => ({ ...prev, idMax: e.target.value })))}
        />
      </div>
    </div>
    <div className="filter-field">
      <label>Nom</label>
      <input
        type="text"
        placeholder="Chercher un nom"
        value={filters.name}
        onChange={(e) => setFilters((prev) => ({ ...prev, name: e.target.value })))}
      />
    </div>
    <div className="filter-field">
      <label>Référence</label>
      <input
        type="text"
        placeholder="Chercher une référence"
        value={filters.reference}
        onChange={(e) => setFilters((prev) => ({ ...prev, reference: e.target.value })))}
      />
    </div>
    <div className="filter-field">
      <label>Catégorie</label>
      <input
        type="number"
        placeholder="ID catégorie"
        value={filters.category}
        onChange={(e) => setFilters((prev) => ({ ...prev, category: e.target.value })))}
      />
    </div>
    <div className="filter-field filter-field--range">
      <label>Montant HT</label>
      <div className="filter-range">
        <input
          type="number"
          min="0"
          step="0.01"
          placeholder="Min."
          value={filters.priceMin}
          onChange={(e) => setFilters((prev) => ({ ...prev, priceMin: e.target.value })))}
        />
        <input
          type="number"

```

```

        min="0"
        step="0.01"
        placeholder="Max."
        value={filters.priceMax}
        onChange={(e) => setFilters((prev) => ({ ...prev, priceMax: e.target.value })))}
      />
    </div>
  </div>

  <div className="filter-field filter-field--range">
    <label>Quantité</label>
    <div className="filter-range">
      <input
        type="number"
        min="0"
        placeholder="Min."
        value={filters.quantityMin}
        onChange={(e) => setFilters((prev) => ({ ...prev, quantityMin: e.target.value })))}
      />
      <input
        type="number"
        min="0"
        placeholder="Max."
        value={filters.quantityMax}
        onChange={(e) => setFilters((prev) => ({ ...prev, quantityMax: e.target.value })))}
      />
    </div>
  </div>

  <div className="filter-field">
    <label>État</label>
    <select
      value={filters.status}
      onChange={(e) => setFilters((prev) => ({ ...prev, status: e.target.value as Filters['status'] })
    >
      <option value="all">Tous</option>
      <option value="active">Actif</option>
      <option value="inactive">Inactif</option>
    </select>
  </div>

  <div className="filter-actions">
    <button type="submit" className="filter-search-btn">
      ■ Rechercher
    </button>
  </div>
</form>

<div className="stats">
  <p>{products.length} produit(s) trouvé(s)</p>
</div>

{products.length > 0 ? (
  <div className="products-grid">
    {products.map((product) => (
      <div key={product.id} className="product-card">
        <div className="product-image-wrapper">
          <ProductBadge
            dateAvailability={product.date_availability_produit}
            className="availability-badge--corner"
          />
          {product.imageUrl ? (
            <img
              src={product.imageUrl}
              alt={product.name}
              className="product-image"
              onError={(e) => {
                (e.currentTarget as HTMLImageElement).style.display = 'none';
                (e.currentTarget.nextElementSibling as HTMLElement | null)?.style.setProperty('display',
              }}
          ) : null}
        </div>
      </div>
    ))}
  </div>
) : null}

```

```

        />
      ) : null}
    <div
      className="product-image-placeholder"
      style={{ display: product.imageUrl ? 'none' : 'flex' }}
    >
      <svg width="32" height="32" viewBox="0 0 24 24" fill="none" stroke="#d1d5db" strokeWidth="1"
        <rect x="3" y="3" width="18" height="18" rx="2"/>
        <circle cx="8.5" cy="8.5" r="1.5"/>
        <polyline points="21 15 16 10 5 21"/>
      </svg>
    </div>
  </div>

  <div className={`product-badge ${product.active ? 'active' : 'inactive'}`}>
    {product.active ? '✓ Actif' : 'X Inactif'}
  </div>

  <div className="product-info">
    <h3 className="product-name">{product.name}</h3>

    <p className="product-reference">
      Réf : {product.reference || 'Non renseignée'}
    </p>

    <p className="product-price">{formatPrice(product)}</p>

    <div className="product-actions">
      <button
        className="view-button"
        onClick={() => navigate(`/products/${product.id}`)}
      >
        =■ Éditer
      </button>
      <button
        className="view-button"
        onClick={() => handleDeleteProduct(product.id)}
      >
        ■■ Supprimer
      </button>
    </div>
  </div>
  </div>
  )}
</div>
) : (
  <div className="no-results">
    <p>Aucun produit ne correspond à votre recherche.</p>
  </div>
)
</div>
);
}

```

loadProducts() — ligne 40

Fonction : `load products` — voir le code ci-dessous.

```

const loadProducts = async (filtersToApply: ProductFilters) => {
  try {
    setLoading(true);
    const productsData = await productService.getAllProducts(filtersToApply);
    setProducts(productsData);
    setError(null);
  } catch (err) {
    setError(
      'Erreur lors du chargement des produits : ' +
      (err as Error).message
    );
  }
};

```

```

        console.error(err);
    } finally {
        setLoading(false);
    }
}

```

toNumber() — ligne 61

Fonction : `to number` — voir le code ci-dessous.

```

const toNumber = (value: string): number | undefined => {
    const trimmed = value.trim();
    if (!trimmed) return undefined;
    const parsed = Number(trimmed);
    return Number.isNaN(parsed) ? undefined : parsed;
}

```

toText() — ligne 68

Fonction : `to text` — voir le code ci-dessous.

```

const toText = (value: string): string | undefined => {
    const trimmed = value.trim();
    return trimmed ? trimmed : undefined;
}

```

buildFilterPayload() — ligne 73

Fonction : `build filter payload` — voir le code ci-dessous.

```

const buildFilterPayload = (): ProductFilters => ({
    idMin: toNumber(filters.idMin),
    idMax: toNumber(filters.idMax),
    name: toText(filters.name),
    reference: toText(filters.reference),
    categoryId: toNumber(filters.category),
    priceMin: toNumber(filters.priceMin),
    priceMax: toNumber(filters.priceMax),
    quantityMin: toNumber(filters.quantityMin),
    quantityMax: toNumber(filters.quantityMax),
    active: filters.status === 'all' ? undefined : filters.status === 'active',
});

```

handleDeleteProduct() — ligne 86

Fonction : `handle delete product` — voir le code ci-dessous.

```

const handleDeleteProduct = async (productId: string) => {
    if (window.confirm("Êtes-vous sûr de vouloir supprimer ce produit ?")) {
        try {
            await productService.deleteProduct(productId);
            loadProducts(appliedFilters);
        } catch (error) {
            setError("Erreur lors de la suppression du produit.");
            console.error(error);
        }
    }
}

```

formatPrice() — ligne 98

Fonction : `format price` — voir le code ci-dessous.

```
const formatPrice = (product: Product): string => {
  return new Intl.NumberFormat('fr-FR', {
    style: 'currency',
    currency: 'EUR',
  }).format(product.price);
}
```

handleSearch() — ligne 105

Fonction : `handle search` — voir le code ci-dessous.

```
const handleSearch = (event: React.FormEvent<HTMLFormElement>) => {
  event.preventDefault();
  const payload = buildFilterPayload();
  setAppliedFilters(payload);
  loadProducts(payload);
}
```

handleAddProduct() — ligne 112

Fonction : `handle add product` — voir le code ci-dessous.

```
const handleAddProduct = () => {
  navigate('/products/add');
}
```

Fichier :

src/components/ProductCreate.tsx

Lignes : 292 • **Fonctions détectées :** 4

ProductCreate() — ligne 5

Fonction : `product create` — voir le code ci-dessous.

```
const ProductCreate: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const [loading, setLoading] = useState(false);
  const [loadingProduct, setLoadingProduct] = useState(!id);
  const [status, setStatus] = useState<{ type: 'success' | 'error' | '', msg: string }>({ type: '', msg: '' })

  // État initial basé sur votre structure XML
  const [formData, setFormData] = useState({
    name: '',
    reference: '',
    ean13: '',
    price: 0, // Prix de vente HT
    wholesale_price: 0, // Prix d'achat
    quantity: 0,
    description: '',
    description_short: '',
    meta_title: '',
    active: true,
    id_category_default: 2,
    id_tax_rules_group: 1
  });

  // Charger les données du produit si on est en mode édition
  useEffect(() => {
    if (id) {
```

```

        loadProductData(id);
    }
}, [id]));

const loadProductData = async (productId: string) => {
    try {
        setLoadingProduct(true);
        const product = await productService.getProduct(productId);
        if (product) {
            setFormData({
                name: product.name,
                reference: product.reference,
                ean13: product.ean13,
                price: product.price,
                wholesale_price: product.wholesale_price,
                quantity: product.quantity,
                description: product.description,
                description_short: product.description_short,
                meta_title: product.meta_title,
                active: product.active,
                id_category_default: product.id_category_default,
                id_tax_rules_group: product.id_tax_rules_group
            });
        } else {
            setStatus({ type: 'error', msg: 'Impossible de charger le produit' });
        }
    } catch (err) {
        setStatus({ type: 'error', msg: 'Erreur lors du chargement du produit' });
        console.error(err);
    } finally {
        setLoadingProduct(false);
    }
};

const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>) => {
    const { name, value, type } = e.target;
    setFormData(prev => ({
        ...prev,
        [name]: type === 'number' ? parseFloat(value) || 0 : value
    }));
};

const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setStatus({ type: '', msg: '' });

    try {
        let result;
        if (id) {
            // Mode édition
            result = await productService.update(id, formData);
            if (result) {
                setStatus({ type: 'success', msg: `Produit "${result.name}" modifié avec succès!` });
                setTimeout(() => navigate('/products'), 2000);
            }
        } else {
            // Mode création
            result = await productService.create(formData);
            if (result) {
                setStatus({ type: 'success', msg: `Produit "${result.name}" créé avec succès (ID: ${result.id})` });
                setTimeout(() => navigate('/products'), 2000);
            }
        }
    } catch (err) {
        setStatus({ type: 'error', msg: 'Erreur lors de la communication avec PrestaShop.' });
        console.error(err);
    } finally {
        setLoading(false);
    }
};

```

```

if (loadingProduct) {
  return (
    <div className="product-form-container">
      <div className="form-loading">
        <p>
          <span className="loading-spinner"></span>
          Chargement du produit...
        </p>
      </div>
    </div>
  );
}

return (
  <div className="product-form-container">
    <div className="form-header">
      <h2>{id ? '—■ Éditer le produit' : '■ Créer un produit PrestaShop'}</h2>
      {id && (
        <button type="button" onClick={() => navigate('/products')} className="btn-back">
          ← Retour
        </button>
      )}
    </div>

    {status.msg && (
      <div className={`form-status ${status.type}`}>
        {status.type === 'success' ? '✓' : '■'} {status.msg}
      </div>
    )}

    <form onSubmit={handleSubmit} className="product-form">
      {/* Section Informations de base */}
      <div className="section-title">■ Informations de base</div>

      <div className="form-group full">
        <label htmlFor="name">Nom du produit *</label>
        <input
          id="name"
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
          required
          placeholder="Ex: Chemise bleue"
        />
      </div>

      <div className="form-group">
        <label htmlFor="reference">Référence</label>
        <input
          id="reference"
          type="text"
          name="reference"
          value={formData.reference}
          onChange={handleChange}
          placeholder="Ex: SKU-001"
        />
      </div>

      <div className="form-group">
        <label htmlFor="ean13">Code EAN13</label>
        <input
          id="ean13"
          type="text"
          name="ean13"
          value={formData.ean13}
          onChange={handleChange}
          placeholder="13 chiffres"
        />
      </div>
    </form>
  </div>
);

```

```

{ /* Section Tarifs */}
<div className="section-title">■ Tarifs</div>

<div className="form-group">
  <label htmlFor="price">Prix de vente HT (€) *</label>
  <input
    id="price"
    type="number"
    step="0.01"
    name="price"
    value={formData.price}
    onChange={handleChange}
    required
    placeholder="0.00"
  />
</div>

<div className="form-group">
  <label htmlFor="wholesale_price">Prix d'achat (€)</label>
  <input
    id="wholesale_price"
    type="number"
    step="0.01"
    name="wholesale_price"
    value={formData.wholesale_price}
    onChange={handleChange}
    placeholder="0.00"
  />
</div>

{ /* Section Stock */}
<div className="section-title">■ Stock</div>

<div className="form-group">
  <label htmlFor="quantity">Quantité initiale *</label>
  <input
    id="quantity"
    type="number"
    name="quantity"
    value={formData.quantity}
    onChange={handleChange}
    required
    placeholder="0"
  />
</div>

<div className="form-group">
  <label htmlFor="id_category_default">ID Catégorie par défaut</label>
  <input
    id="id_category_default"
    type="number"
    name="id_category_default"
    value={formData.id_category_default}
    onChange={handleChange}
    placeholder="2"
  />
</div>

{ /* Section SEO & Descriptions */}
<div className="section-title">■ SEO & Descriptions</div>

<div className="form-group full">
  <label htmlFor="meta_title">Titre SEO (Meta Title)</label>
  <input
    id="meta_title"
    type="text"
    name="meta_title"
    value={formData.meta_title}
    onChange={handleChange}
    placeholder="Titre pour les moteurs de recherche"
  />
</div>

```

```

</div>

<div className="form-group full">
  <label htmlFor="description_short">Résumé (Description courte)</label>
  <textarea
    id="description_short"
    name="description_short"
    value={formData.description_short}
    onChange={handleChange}
    rows={3}
    placeholder="Brève description du produit..."
  />
</div>

<div className="form-group full">
  <label htmlFor="description">Description complète</label>
  <textarea
    id="description"
    name="description"
    value={formData.description}
    onChange={handleChange}
    rows={5}
    placeholder="Description détaillée (peut contenir du HTML)..."
  />
</div>

{/* Boutons d'action */}

```

loadProductData() — ligne 35

Fonction : `load product data` — voir le code ci-dessous.

```

const loadProductData = async (productId: string) => {
  try {
    setLoadingProduct(true);
    const product = await productService.getProduct(productId);
    if (product) {
      setFormData({
        name: product.name,
        reference: product.reference,
        ean13: product.ean13,
        price: product.price,
        wholesale_price: product.wholesale_price,
        quantity: product.quantity,
        description: product.description,
        description_short: product.description_short,
        meta_title: product.meta_title,
        active: product.active,
        id_category_default: product.id_category_default,
        id_tax_rules_group: product.id_tax_rules_group
      });
    } else {
      setStatus({ type: 'error', msg: 'Impossible de charger le produit' });
    }
  } catch (err) {
    setStatus({ type: 'error', msg: 'Erreur lors du chargement du produit' });
    console.error(err);
  } finally {
    setLoadingProduct(false);
  }
}

```

handleChange() — ligne 65

Fonction : `handle change` — voir le code ci-dessous.

```

const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>) => {
  const { name, value, type } = e.target;

```

```

    setFormData(prev => ({
      ...prev,
      [name]: type === 'number' ? parseFloat(value) || 0 : value
    }));
  }
}

```

handleSubmit() — ligne 73

Fonction : `handle submit` — voir le code ci-dessous.

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setLoading(true);
  setStatus({ type: '', msg: '' });

  try {
    let result;
    if (id) {
      // Mode édition
      result = await productService.update(id, formData);
      if (result) {
        setStatus({ type: 'success', msg: `Produit "${result.name}" modifié avec succès!` });
        setTimeout(() => navigate('/products'), 2000);
      }
    } else {
      // Mode création
      result = await productService.create(formData);
      if (result) {
        setStatus({ type: 'success', msg: `Produit "${result.name}" créé avec succès (ID: ${result.id})` });
        setTimeout(() => navigate('/products'), 2000);
      }
    }
  } catch (err) {
    setStatus({ type: 'error', msg: 'Erreur lors de la communication avec PrestaShop.' });
    console.error(err);
  } finally {
    setLoading(false);
  }
}

```

Fichier : src/components/ProductBadge.tsx

Lignes : 65 • **Fonctions détectées :** 4

startTimer() — ligne 8

Fonction : `start timer` — voir le code ci-dessous.

```

function startTimer() {
  if (timerId !== null) return;
  timerId = window.setInterval(() => {
    const now = Date.now();
    listeners.forEach((listener) => listener(now));
  }, REFRESH_MS);
}

```

stopTimer() — ligne 16

Fonction : `stop timer` — voir le code ci-dessous.

```
function stopTimer() {
  if (timerId === null) return;
  if (listeners.size > 0) return;
  window.clearInterval(timerId);
  timerId = null;
}
```

subscribe() — ligne 23

Fonction : `subscribe` — voir le code ci-dessous.

```
function subscribe(listener: (now: number) => void) {
  listeners.add(listener);
  startTimer();
  return () => {
    listeners.delete(listener);
    stopTimer();
  };
}
```

ProductBadge() — ligne 38

Fonction : `product badge` — voir le code ci-dessous.

```
const ProductBadge: React.FC<ProductBadgeProps> = ({ dateAvailability, className, nowMs: nowMsProp }) => {
  const [nowMsInternal, setNowMs] = useState(() => Date.now());
  const nowMs = nowMsProp ?? nowMsInternal;

  useEffect(() => {
    if (!dateAvailability || nowMsProp !== undefined) return;
    return subscribe(setNowMs);
  }, [dateAvailability, nowMsProp]);

  const badge = useMemo(
    () => getProductBadge(dateAvailability, nowMs),
    [dateAvailability, nowMs]
  );

  if (!badge) return null;

  const badgeClass = `availability-badge availability-badge--${badge.toLowerCase}`;
  const label = badge === 'HOT' ? '■ HOT' : '■ NEW';
  return (
    <span className={[[badgeClass, className].filter(Boolean).join(' ')}>
      {label}
    </span>
  );
}
```

Composants — Imports

Fichier :

src/components/ProductImport.tsx

Lignes : 418 • Fonctions détectées : 9

ProductImport() — ligne 10

Fonction : `product import` — voir le code ci-dessous.

[illegible]


```

<svg className="import-dropzone-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeW
  <polyline points="16 16 12 12 8 16"/>
  <line x1="12" y1="12" x2="12" y2="21"/>
  <path d="M20.39 18.39A5 5 0 0018 9h-1.26A8 8 0 103 16.3"/>
</svg>
{file ? (
  <p className="import-dropzone-text">
    <strong>{file.name}</strong> – {(file.size / 1024).toFixed(1)} Ko
  </p>
) : (
  <p className="import-dropzone-text">Glissez votre fichier CSV ici</p>
  <p className="import-dropzone-sub">ou cliquez pour parcourir</p>
</>
)}
<input
  ref={fileInputRef}
  type="file"
  accept=".csv"
  className="import-file-input"
  onChange={handleInputChange}
/>
</div>
)}

{/* ■■ PRÉVISUALISATION AVEC VALIDATION ■■ */}
{step === 'preview' && allRows.length > 0 && (
  <div className="import-preview">
    <h2 className="import-section-title">
      Pré-validation – {allRows.length} ligne{allRows.length > 1 ? 's' : ''} détectée{allRows.length > 1 ? 's' : ''}
    </h2>

    <div className="import-validation-summary">
      <div className="import-validation-chip import-validation-chip--valid">
        <span className="import-validation-chip-icon">✓</span>
        <span>{validCount} valide{validCount > 1 ? 's' : ''}</span>
      </div>
      {warningCount > 0 && (
        <div className="import-validation-chip import-validation-chip--warning">
          <span className="import-validation-chip-icon">■</span>
          <span>{warningCount} avertissement{warningCount > 1 ? 's' : ''}</span>
        </div>
      )}
      {errorCount > 0 && (
        <div className="import-validation-chip import-validation-chip--error">
          <span className="import-validation-chip-icon">X</span>
          <span>{errorCount} erreur{errorCount > 1 ? 's' : ''}</span>
        </div>
      )}
    </div>

    {hasBlockingErrors && (
      <div className="import-blocking-banner">
        Corrigez les erreurs dans le fichier CSV avant de lancer l'import.
      </div>
    )}

    <div className="import-table-wrapper" style={{ maxHeight: '480px', overflowY: 'auto' }}>
      <table className="import-table">
        <thead>
          <tr>
            <th>Ligne</th>
            <th>Statut</th>
            <th>Nom</th>
            <th>Référence</th>
            <th>Prix HT</th>
            <th>Prix achat</th>
            <th>Qté</th>
            <th>Catégorie</th>
          </tr>
        </thead>
      </table>
    </div>
  </div>
)
}

```



```
function handleFile(selectedFile: File) {
  if (!selectedFile.name.endsWith('.csv')) {
    alert('Veuillez sélectionner un fichier .csv');
    return;
  }

  setFile(selectedFile);

  selectedFile.text().then((content) => {
    const rows = parseCSV(content);
    setAllRows(rows);
    const vals = validateAllRows(rows);
    setValidations(vals);
    setStep('preview');
  });
}
```

handleInputChange() — ligne 44

Fonction : `handle input change` — voir le code ci-dessous.

```
function handleInputChange(e: React.ChangeEvent<HTMLInputElement>) {
  const selected = e.target.files?.[0];
  if (selected) handleFile(selected);
}
```

handleDrop() — ligne 49

Fonction : `handle drop` — voir le code ci-dessous.

```
function handleDrop(e: React.DragEvent<HTMLDivElement>) {
  e.preventDefault();
  setDragOver(false);
  const dropped = e.dataTransfer.files[0];
  if (dropped) handleFile(dropped);
}
```

handleDragOver() — ligne 56

Fonction : `handle drag over` — voir le code ci-dessous.

```
function handleDragOver(e: React.DragEvent<HTMLDivElement>) {
  e.preventDefault();
  setDragOver(true);
}
```

handleDragLeave() — ligne 61

Fonction : `handle drag leave` — voir le code ci-dessous.

```
function handleDragLeave() {
  setDragOver(false);
}
```

handleImport() — ligne 67

Description (commentaire d'origine) : ■■ Lancement de l'import

```
async function handleImport() {
  if (!file) return;
  setStep('importing');
```


[illegible]

```

setCleanStatus('running');
setCleanProgress({ done: 0, total: 0 });
setCleanResult(null);
try {
  const result = await config.clean((done, total) => setCleanProgress({ done, total }));
  setCleanResult(result);
} catch {
  setCleanResult({ total: 0, deleted: 0, errors: 1 });
}
setCleanStatus('done');
}

const successCount = results.filter((r) => r.success).length;
const errorCount = results.filter((r) => !r.success).length;
const progressPct = progress.total > 0 ? Math.round((progress.done / progress.total) * 100) : 0;

return (
  <div className="import-page">

    {/* ■■ SÉLECTEUR DE TYPE ■■ */}
    <div className="import-type-selector">
      <label className="import-type-label">Type d'entité à importer</label>
      <div className="import-type-tabs">
        {(Object.keys(IMPORT_TYPES) as ImportType[]).map((type) => (
          <button
            key={type}
            className={`import-type-tab${importType === type ? ' import-type-tab--active' : ''}`}
            onClick={() => handleTypeChange(type)}
          >
            {IMPORT_TYPES[type].label}
          </button>
        ))}
      </div>
      <p className="import-type-hint">
        Endpoint : <code>{config.endpoint}</code>
      </p>
    </div>

    {/* ■■ ZONE DE DÉPÔT ■■ */}
    {(step === 'idle' || step === 'preview') && (
      <div
        className={`import-dropzone${dragOver ? ' import-dropzone--over' : ''}`}
        onDrop={handleDrop}
        onDragOver={(e) => { e.preventDefault(); setDragOver(true); }}
        onDragLeave={() => setDragOver(false)}
        onClick={() => fileInputRef.current?.click()}
      >
        <svg className="import-dropzone-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeW
          <polyline points="16 16 12 12 8 16"/>
          <line x1="12" y1="12" x2="12" y2="21"/>
          <path d="M20.39 18.39A5 0 0018 9h-1.26A8 0 0103 16.3"/>
        </svg>
        {file ? (
          <p className="import-dropzone-text">
            <strong>{file.name}</strong> – {(file.size / 1024).toFixed(1)} Ko
          </p>
        ) : (
          <p className="import-dropzone-text">
            Glissez votre fichier <strong>{config.label}</strong> CSV ici
          </p>
          <p className="import-dropzone-sub">ou cliquez pour parcourir</p>
        )}
      <input
        ref={fileInputRef}
        type="file"
        accept=".csv"
        className="import-file-input"
        onChange={handleInputChange}
      />
    )}
  </div>

```

```

    </div>
  )}

  /* ■■ PRÉVISUALISATION ■■ */
  {step === 'preview' && previewRows.length > 0 && (
    <div className="import-preview">
      <h2 className="import-section-title">Aperçu – 5 premières lignes</h2>
      <div className="import-table-wrapper">
        <table className="import-table">
          <thead>
            <tr>
              {config.previewHeaders.map((h) => <th key={h}>{h}</th>)}
            </tr>
          </thead>
          <tbody>
            {previewRows.map((row, i) => (
              <tr key={i}>
                {config.previewHeaders.map((h) => (
                  <td key={h}>
                    {h === 'Actif' ? (
                      <span className={`import-badge ${row[h] === 'Oui' ? 'import-badge--success' : 'import-
                        {row[h]}
                      </span>
                    ) : (
                      <span title={row[h]} className="import-cell-truncate">{row[h]}</span>
                    )}
                  </td>
                ))}
              </tr>
            ))}
          </tbody>
        </table>
      </div>
      <div className="import-actions">
        <button className="btn btn-secondary" onClick={handleReset}>Changer de fichier</button>
        <button className="btn btn-primary" onClick={handleImport}>Lancer l'import</button>
      </div>
    </div>
  )}
}

```

handleFile() — ligne 114

Description (commentaire d'origine) : ■■ Sélection / drag-drop

```

function handleFile(f: File) {
  if (!f.name.endsWith('.csv')) {
    alert('Veuillez sélectionner un fichier .csv');
    return;
  }
  setFile(f);
  f.text().then((content) => {
    setPreviewRows(config.parse(content).slice(0, 5));
    setStep('preview');
  });
}

```

handleInputChange() — ligne 125

Fonction : `handle input change` — voir le code ci-dessous.

```

function handleInputChange(e: React.ChangeEvent<HTMLInputElement>) {
  const f = e.target.files?.[0];
  if (f) handleFile(f);
}

```

handleDrop() — ligne 130

Fonction : `handle drop` — voir le code ci-dessous.

```
function handleDrop(e: React.DragEvent<HTMLDivElement>) {
  e.preventDefault();
  setDragOver(false);
  const f = e.dataTransfer.files[0];
  if (f) handleFile(f);
}
```

handleImport() — ligne 139

Description (commentaire d'origine) :

```

async function handleImport() {
  if (!file) return;
  setStep('importing');
  setResults([]);
  setProgress({ done: 0, total: 0 });
  const res = await config.run(file, (done, total) => setProgress({ done, total }));
  setResults(res);
  setStep('done');
}

```

handleReset() — ligne 148

Fonction : `handle reset` — voir le code ci-dessous.

```
function handleReset() {
  setFile(null);
  setPreviewRows([]);
  setProgress({ done: 0, total: 0 });
  setResults([]);
  setStep('idle');
  if (fileInputRef.current) fileInputRef.current.value = '';
}
```

handleTypeChange() — ligne 157

Fonction : `handle type change` — voir le code ci-dessous.

```
function handleTypeChange(type: ImportType) {
  setImportType(type);
  handleReset();
  setCleanStatus('idle');
  setCleanResult(null);
}
```

handleClean() — ligne 164

Fonction : `handle_clean` — voir le code ci-dessous.

```

async function handleClean() {
  const confirmed = window.confirm(
    `Supprimer TOUTES les entrées de type "${config.label}" de la base de données ?\n\nCette action est irréversible`
  );
  if (!confirmed) return;
  setCleanStatus('running');
  setCleanProgress({ done: 0, total: 0 });
  setCleanResult(null);
  try {

```

```

    const result = await config.clean((done, total) => setCleanProgress({ done, total }));
    setCleanResult(result);
  } catch {
    setCleanResult({ total: 0, deleted: 0, errors: 1 });
  }
  setCleanStatus('done');
}

```

Fichier :

src/components/FichiersImport.tsx

Lignes : 505 • Fonctions détectées : 13

Dropzone() — ligne 41

Fonction : **dropzone** — voir le code ci-dessous.

```

const Dropzone: React.FC<DropzoneProps> = ({ accept, label, sublabel, file, disabled, onChange }) => {
  const inputRef = useRef<HTMLInputElement>(null);
  const [over, setOver] = useState(false);

  const handleDrop = (e: React.DragEvent) => {
    e.preventDefault();
    setOver(false);
    if (disabled) return;
    const f = e.dataTransfer.files[0];
    if (f) onChange(f);
  };

  return (
    <div
      className={`fz-dropzone${over ? ' fz-dropzone--over' : ''}${disabled ? ' fz-dropzone--disabled' : ''}`}
      onClick={() => !disabled && inputRef.current?.click()}
      onDragOver={(e) => { e.preventDefault(); if (!disabled) setOver(true); }}
      onDragLeave={() => setOver(false)}
      onDrop={handleDrop}
    >
      <input
        ref={inputRef}
        type="file"
        accept={accept}
        className="fz-file-input"
        disabled={disabled}
        onChange={(e) => { const f = e.target.files?.[0]; if (f) onChange(f); }}
      />
      <svg className="fz-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5">
        <path d="M4 16v1a3 3 0 03 3h10a3 3 0 03-3v-1M12 12v4m0 0l-3 3m3-3l3 3"/>
      </svg>
      {file ? (
        <p className="fz-filename">{file.name}</p>
      ) : (
        <>
          <p className="fz-label">{label}</p>
          <p className="fz-sublabel">{sublabel}</p>
        </>
      )}
    </div>
  );
}

```

handleDrop() — ligne 45

Fonction : **handle drop** — voir le code ci-dessous.

```

const handleDrop = (e: React.DragEvent) => {
  e.preventDefault();
  setOver(false);
  if (disabled) return;
  const f = e.dataTransfer.files[0];
  if (f) onChange(f);
}

```

ResultsTable() — ligne 90

Fonction : `results table` — voir le code ci-dessous.

```

const ResultsTable: React.FC<ResultsTableProps> = ({ results }) => {
  const ok = results.filter((r) => r.success).length;
  const err = results.filter((r) => !r.success).length;

  return (
    <div className="fz-results">
      <div className="fz-stats">
        <div className="fz-stat fz-stat--total"><span className="fz-stat-val">{results.length}</span><span cla
        <div className="fz-stat fz-stat--ok"> <span className="fz-stat-val">{ok}</span> <span cla
        {err > 0 && <div className="fz-stat fz-stat--err"><span className="fz-stat-val">{err}</span><span cla
      </div>
      {err > 0 && (
        <div className="fz-table-wrapper">
          <table className="fz-table">
            <thead><tr><th>N° CSV</th><th>Entrée</th><th>Statut</th><th>Détail</th></tr></thead>
            <tbody>
              {results.filter((r) => !r.success).map((r, i) => (
                <tr key={i} className="fz-row--err">
                  <td className="fz-line-num">{r.lineNumber != null ? `L.${r.lineNumber}` : '-'}</td>
                  <td>{r.label}</td>
                  <td><span className="fz-badge fz-badge--err">Erreur</span></td>
                  <td className="fz-detail">{r.error}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  );
}

```

ImageResultsTable() — ligne 126

Fonction : `image results table` — voir le code ci-dessous.

```

const ImageResultsTable: React.FC<ImageResultsTableProps> = ({ results }) => {
  const ok = results.filter((r) => r.success).length;
  const err = results.filter((r) => !r.success).length;
  const skip = 0;

  return (
    <div className="fz-results">
      <div className="fz-stats">
        <div className="fz-stat fz-stat--total"><span className="fz-stat-val">{results.length}</span><span cla
        <div className="fz-stat fz-stat--ok"> <span className="fz-stat-val">{ok}</span> <span cla
        {skip > 0 && <div className="fz-stat fz-stat--skip"><span className="fz-stat-val">{skip}</span><span cla
        {err > 0 && <div className="fz-stat fz-stat--err"><span className="fz-stat-val">{err}</span> <span cla
      </div>
    </div>
  );
}

```

ProgressBar() — ligne 150

Fonction : `progress bar` — voir le code ci-dessous.

```
const ProgressBar: React.FC<ProgressProps> = ({ pct, label }) => (
  <div className="fz-progress-block">
    <p className="fz-progress-label">{label || 'Traitement...'}</p>
    <div className="fz-progress-bar">
      <div className="fz-progress-fill" style={{ width: `${pct}%` }} />
    </div>
    <span className="fz-progress-pct">{pct}%</span>
  </div>
);
```

FichiersImport() — ligne 162

Description	(commentaire d'origine)	:	■ ■	Composant principal

```
const FichiersImport: React.FC = () => {
  const [z1, setZ1] = useState<ZoneState>(INITIAL_ZONE);
  const [z2, setZ2] = useState<ZoneState>(INITIAL_ZONE);
  const [z3, setZ3] = useState<ZoneState>(INITIAL_ZONE);
  const [zImg, setZImg] = useState<ZoneState>(INITIAL_ZONE);
  const [formPhase, setFormPhase] = useState<'idle' | 'running' | 'done' | 'error'>('idle');
  const [formError, setFormError] = useState('');

  const allIdle = [z1, z2, z3, zImg].every((z) => z.phase === 'idle');
  const hasAllFiles = Boolean(z1.file && z2.file && z3.file && zImg.file);
  const inputsLocked = formPhase === 'running' || !allIdle;
```

[illegible]

```
const runFichier1 = async (): Promise<boolean> => {
  if (!z1.file || z1.phase === 'running') return false;
  setZ1((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importFichier1(z1.file, (done, total, label) => {
      setZ1((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
    setZ1((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZ1((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
};
```

```
const runFichier2 = async (): Promise<boolean> => {
  if (!z2.file || z2.phase === 'running') return false;
  setZ2((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importFichier2(z2.file, (done, total, label) => {
      setZ2((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
    setZ2((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZ2((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
};
```

```

const runFichier3 = async (): Promise<boolean> => {
  if (!z3.file || z3.phase === 'running') return false;
  setZ3((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importFichier3(z3.file, (done, total, label) => {
      setZ3((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
    setZ3((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZ3((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
};

const runImages = async (): Promise<boolean> => {
  if (!zImg.file || zImg.phase === 'running') return false;
  setZImg((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importImagesZip(zImg.file, (done, total, label) => {
      setZImg((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
    setZImg((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZImg((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
};

const runPrevalidation = async (): Promise<boolean> => {
  if (!z1.file || !z2.file || !z3.file || !zImg.file) return false;

  setZ1((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));
  setZ2((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));
  setZ3((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));
  setZImg((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));

  const validation = await prevalidateFichiersImport(
    { fichier1: z1.file, fichier2: z2.file, fichier3: z3.file, images: zImg.file },
    {
      fichier1: (done, total, label) => {
        setZ1((p) => ({
          ...p,
          progress: total > 0 ? Math.round((done / total) * 100) : 0,
          progressLabel: label,
        }));
      },
      fichier2: (done, total, label) => {
        setZ2((p) => ({
          ...p,
          progress: total > 0 ? Math.round((done / total) * 100) : 0,
          progressLabel: label,
        }));
      },
      fichier3: (done, total, label) => {
        setZ3((p) => ({
          ...p,
          progress: total > 0 ? Math.round((done / total) * 100) : 0,
          progressLabel: label,
        }));
      },
      images: (done, total, label) => {

```


}

runFichier1() — ligne 176

Description (commentaire d'origine) :

```
const runFichier1 = async (): Promise<boolean> => {
  if (!z1.file || z1.phase === 'running') return false;
  setZ1((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importFichier1(z1.file, (done, total, label) => {
      setZ1((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
  } catch (e) {
    setZ1((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
}
```

runFichier2() — ligne 195

Fonction : `run fichier2` — voir le code ci-dessous.

```
const runFichier2 = async (): Promise<boolean> => {
  if (!z2.file || z2.phase === 'running') return false;
  setZ2((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importFichier2(z2.file, (done, total, label) => {
      setZ2((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
    setZ2((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZ2((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
}
```

runFichier3() — ligne 214

Fonction : `run fichier3` — voir le code ci-dessous.

```
const runFichier3 = async (): Promise<boolean> => {
  if (!z3.file || z3.phase === 'running') return false;
  setZ3((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importFichier3(z3.file, (done, total, label) => {
      setZ3((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
  }
}
```

```

    });
    setZ3((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZ3((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
}

```

runImages() — ligne 233

Fonction : `run images` — voir le code ci-dessous.

```

const runImages = async (): Promise<boolean> => {
  if (!zImg.file || zImg.phase === 'running') return false;
  setZImg((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: '', results: [] }));
  try {
    const results = await importImagesZip(zImg.file, (done, total, label) => {
      setZImg((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    });
    setZImg((p) => ({ ...p, phase: 'done', progress: 100, results }));
    return true;
  } catch (e) {
    setZImg((p) => ({ ...p, phase: 'error', progressLabel: String(e) }));
    return false;
  }
}

```

runPrevalidation() — ligne 252

Fonction : `run prevalidation` — voir le code ci-dessous.

```

const runPrevalidation = async (): Promise<boolean> => {
  if (!z1.file || !z2.file || !z3.file || !zImg.file) return false;

  setZ1((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));
  setZ2((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));
  setZ3((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));
  setZImg((p) => ({ ...p, phase: 'running', progress: 0, progressLabel: 'Pré-validation...', results: [] }));

  const validation = await prevalidateFichiersImport(
    { fichier1: z1.file, fichier2: z2.file, fichier3: z3.file, images: zImg.file },
    {
      fichier1: (done, total, label) => {
        setZ1((p) => ({
          ...p,
          progress: total > 0 ? Math.round((done / total) * 100) : 0,
          progressLabel: label,
        }));
      },
      fichier2: (done, total, label) => {
        setZ2((p) => ({
          ...p,
          progress: total > 0 ? Math.round((done / total) * 100) : 0,
          progressLabel: label,
        }));
      },
      fichier3: (done, total, label) => {
        setZ3((p) => ({
          ...p,
          progress: total > 0 ? Math.round((done / total) * 100) : 0,
          progressLabel: label,
        }));
      },
    },
  );
}

```

```

    images: (done, total, label) => {
      setZImg((p) => ({
        ...p,
        progress: total > 0 ? Math.round((done / total) * 100) : 0,
        progressLabel: label,
      }));
    },
  },
);

setZ1((p) => ({ ...p, phase: 'done', progress: 100, results: validation.fichier1 }));
setZ2((p) => ({ ...p, phase: 'done', progress: 100, results: validation.fichier2 }));
setZ3((p) => ({ ...p, phase: 'done', progress: 100, results: validation.fichier3 }));
setZImg((p) => ({ ...p, phase: 'done', progress: 100, results: validation.images }));

if (validation.hasErrors) {
  setFormError('Pré-validation échouée : aucune donnée n\'a été importée.');
```

```

  return false;
}
return true;
}

```

resetAll() — ligne 306

Fonction : `reset all` — voir le code ci-dessous.

```

const resetAll = () => {
  setZ1(INITIAL_ZONE);
  setZ2(INITIAL_ZONE);
  setZ3(INITIAL_ZONE);
  setZImg(INITIAL_ZONE);
  setFormPhase('idle');
  setFormError('');
}

```

runAll() — ligne 315

Fonction : `run all` — voir le code ci-dessous.

```

const runAll = async (e: React.FormEvent) => {
  e.preventDefault();
  if (formPhase === 'running') return;
  setFormError('');

  if (!hasAllFiles) {
    setFormError('Veuillez sélectionner les 4 fichiers avant de lancer l\'import.');
```

```

    return;
  }

```

```

  if (!allIdle) {
    setFormError('Réinitialisez le formulaire avant de relancer un import complet.');
```

```

    return;
  }

```

```

  setFormPhase('running');
  const prevalidationOk = await runPrevalidation();
  if (!prevalidationOk) { setFormPhase('error'); return; }

```

```

  const ok1 = await runFichier1();
  if (!ok1) { setFormPhase('error'); return; }

```

```

  const ok2 = await runFichier2();
  if (!ok2) { setFormPhase('error'); return; }

```

```

  const ok3 = await runFichier3();
  if (!ok3) { setFormPhase('error'); return; }

```

```

  const okImg = await runImages();

```

```

    if (!okImg) { setFormPhase('error'); return; }

    setFormPhase('done');
  }

```

Fichier : src/components/ImportAudit.tsx

Lignes : 465 • Fonctions détectées : 2

ImportAudit() — ligne 35

Fonction : `import audit` — voir le code ci-dessous.

```

const ImportAudit: React.FC = () => {
  const [products, setProducts] = useState<SectionState<ProductAudit>>(DEFAULT_STATE);
  const [categories, setCategories] = useState<SectionState<CategoryAudit>>(DEFAULT_STATE);
  const [customers, setCustomers] = useState<SectionState<CustomerAudit>>(DEFAULT_STATE);
  const [addresses, setAddresses] = useState<SectionState<AddressAudit>>(DEFAULT_STATE);
  const [suppliers, setSuppliers] = useState<SectionState<SupplierAudit>>(DEFAULT_STATE);
  const [brands, setBrands] = useState<SectionState<BrandAudit>>(DEFAULT_STATE);
  const [combinations, setCombinations] = useState<SectionState<CombinationAudit>>(DEFAULT_STATE);
  const [stocks, setStocks] = useState<SectionState<StockAudit>>(DEFAULT_STATE);
  const [taxes, setTaxes] = useState<SectionState<TaxAudit>>(DEFAULT_STATE);
  const [taxGroups, setTaxGroups] = useState<SectionState<TaxRuleGroupAudit>>(DEFAULT_STATE);
  const [taxRules, setTaxRules] = useState<SectionState<TaxRuleAudit>>(DEFAULT_STATE);

  const loadAll = useCallback(async () => {
    setProducts((p) => ({ ...p, loading: true, error: null }));
    setCategories((p) => ({ ...p, loading: true, error: null }));
    setCustomers((p) => ({ ...p, loading: true, error: null }));
    setAddresses((p) => ({ ...p, loading: true, error: null }));
    setSuppliers((p) => ({ ...p, loading: true, error: null }));
    setBrands((p) => ({ ...p, loading: true, error: null }));
    setCombinations((p) => ({ ...p, loading: true, error: null }));
    setStocks((p) => ({ ...p, loading: true, error: null }));
    setTaxes((p) => ({ ...p, loading: true, error: null }));
    setTaxGroups((p) => ({ ...p, loading: true, error: null }));
    setTaxRules((p) => ({ ...p, loading: true, error: null }));

    const tasks = await Promise.allSettled([
      fetchProductsSample(),
      fetchCategoriesSample(),
      fetchCustomersSample(),
      fetchAddressesSample(),
      fetchSuppliersSample(),
      fetchBrandsSample(),
      fetchCombinationsSample(),
      fetchStockSample(),
      fetchTaxesSample(),
      fetchTaxRuleGroupsSample(),
      fetchTaxRulesSample(),
    ]);

    const applyResult = <T,>(
      result: PromiseSettledResult<T[]>,
      setter: React.Dispatch<React.SetStateAction<SectionState<T>>>
    ) => {
      if (result.status === 'fulfilled') {
        setter({ items: result.value, loading: false, error: null });
      } else {
        setter({ items: [], loading: false, error: result.reason?.message ?? 'Erreur' });
      }
    };
  };

```

```

    applyResult(tasks[0] as PromiseSettledResult<ProductAudit[]>, setProducts);
    applyResult(tasks[1] as PromiseSettledResult<CategoryAudit[]>, setCategories);
    applyResult(tasks[2] as PromiseSettledResult<CustomerAudit[]>, setCustomers);
    applyResult(tasks[3] as PromiseSettledResult<AddressAudit[]>, setAddresses);
    applyResult(tasks[4] as PromiseSettledResult<SupplierAudit[]>, setSuppliers);
    applyResult(tasks[5] as PromiseSettledResult<BrandAudit[]>, setBrands);
    applyResult(tasks[6] as PromiseSettledResult<CombinationAudit[]>, setCombinations);
    applyResult(tasks[7] as PromiseSettledResult<StockAudit[]>, setStocks);
    applyResult(tasks[8] as PromiseSettledResult<TaxAudit[]>, setTaxes);
    applyResult(tasks[9] as PromiseSettledResult<TaxRuleGroupAudit[]>, setTaxGroups);
    applyResult(tasks[10] as PromiseSettledResult<TaxRuleAudit[]>, setTaxRules);
  }, []);

  useEffect(() => {
    loadAll();
  }, [loadAll]);

  const renderHeader = (title: string, count: number, loading: boolean) => (
    <div className="audit-section-header">
      <h2 className="audit-section-title">{title}</h2>
      <span className="audit-count">{loading ? '...' : count}</span>
    </div>
  );

  return (
    <div className="audit-page">
      <div className="audit-header">
        <div>
          <h1 className="audit-title">Audit import</h1>
          <p className="audit-subtitle">Verifier les donnees importees (extraits recents).</p>
        </div>
        <button className="btn btn-secondary" onClick={loadAll}>
          Actualiser
        </button>
      </div>

      <section className="audit-section">
        {renderHeader('Produits', products.items.length, products.loading)}
        {products.error && <p className="audit-error">{products.error}</p>}
        {!products.loading && products.items.length === 0 && !products.error && (
          <p className="audit-empty">Aucun produit.</p>
        )}
        {products.items.length > 0 && (
          <div className="audit-table-wrap">
            <table className="audit-table">
              <thead>
                <tr>
                  <th>ID</th>
                  <th>Nom</th>
                  <th>Reference</th>
                  <th>Prix HT</th>
                  <th>Taxe</th>
                  <th>Actif</th>
                </tr>
              </thead>
              <tbody>
                {products.items.map((p) => (
                  <tr key={p.id}>
                    <td>{p.id}</td>
                    <td>{p.name}</td>
                    <td>{p.reference}</td>
                    <td>{p.priceHt}</td>
                    <td>{p.taxRulesGroupId}</td>
                    <td>{p.active}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </section>
    </div>
  );

```

```

<section className="audit-section">
  {renderHeader('Declinaisons', combinations.items.length, combinations.loading)}
  {combinations.error && <p className="audit-error">{combinations.error}</p>}
  {!combinations.loading && combinations.items.length === 0 && !combinations.error && (
    <p className="audit-empty">Aucune declinaison.</p>
  )}
  {combinations.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>ID Produit</th>
            <th>Reference</th>
          </tr>
        </thead>
        <tbody>
          {combinations.items.map((c) => (
            <tr key={c.id}>
              <td>{c.id}</td>
              <td>{c.productId}</td>
              <td>{c.reference}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

```

```

<section className="audit-section">
  {renderHeader('Stock', stocks.items.length, stocks.loading)}
  {stocks.error && <p className="audit-error">{stocks.error}</p>}
  {!stocks.loading && stocks.items.length === 0 && !stocks.error && (
    <p className="audit-empty">Aucun stock.</p>
  )}
  {stocks.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>ID Produit</th>
            <th>Quantite</th>
          </tr>
        </thead>
        <tbody>
          {stocks.items.map((s) => (
            <tr key={s.id}>
              <td>{s.id}</td>
              <td>{s.productId}</td>
              <td>{s.quantity}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

```

```

<section className="audit-section">
  {renderHeader('Categories', categories.items.length, categories.loading)}
  {categories.error && <p className="audit-error">{categories.error}</p>}
  {!categories.loading && categories.items.length === 0 && !categories.error && (
    <p className="audit-empty">Aucune categorie.</p>
  )}
  {categories.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>

```

```

        <th>ID</th>
        <th>Nom</th>
    </tr>
</thead>
<tbody>
    {categories.items.map((c) => (
        <tr key={c.id}>
            <td>{c.id}</td>
            <td>{c.name}</td>
        </tr>
    ))}
</tbody>
</table>
</div>
    )}
</section>

<section className="audit-section">
    {renderHeader('Clients', customers.items.length, customers.loading)}
    {customers.error && <p className="audit-error">{customers.error}</p>}
    {!customers.loading && customers.items.length === 0 && !customers.error && (
        <p className="audit-empty">Aucun client.</p>
    )}
    {customers.items.length > 0 && (
        <div className="audit-table-wrap">
            <table className="audit-table">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Prenom</th>
                        <th>Nom</th>
                        <th>Email</th>
                    </tr>
                </thead>
                <tbody>
                    {customers.items.map((c) => (
                        <tr key={c.id}>
                            <td>{c.id}</td>
                            <td>{c.firstname}</td>
                            <td>{c.lastname}</td>
                            <td>{c.email}</td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    )}
</section>

<section className="audit-section">
    {renderHeader('Adresses', addresses.items.length, addresses.loading)}
    {addresses.error && <p className="audit-error">{addresses.error}</p>}
    {!addresses.loading && addresses.items.length === 0 && !addresses.error && (
        <p className="audit-empty">Aucune adresse.</p>
    )}
    {addresses.items.length > 0 && (
        <div className="audit-table-wrap">
            <table className="audit-table">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Alias</th>
                        <th>Ville</th>
                        <th>Code postal</th>
                        <th>Pays</th>
                        <th>Client</th>
                    </tr>
                </thead>
                <tbody>
                    {addresses.items.map((a) => (
                        <tr key={a.id}>

```

```

        <td>{a.id}</td>
        <td>{a.alias}</td>
        <td>{a.city}</td>
        <td>{a.postcode}</td>
        <td>{a.countryId}</td>
        <td>{a.customerId}</td>
    </tr>
  )}}
</tbody>
</table>
</div>
)}
</section>

<section className="audit-section">
  {renderHeader('Fournisseurs', suppliers.items.length, suppliers.loading)}
  {suppliers.error && <p className="audit-error">{suppliers.error}</p>}
  {!suppliers.loading && suppliers.items.length === 0 && !suppliers.error && (
    <p className="audit-empty">Aucun fournisseur.</p>
  )}
  {suppliers.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Nom</th>
          </tr>
        </thead>
        <tbody>
          {suppliers.items.map((s) => (
            <tr key={s.id}>
              <td>{s.id}</td>
              <td>{s.name}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

<section className="audit-section">
  {renderHeader('Marques', brands.items.length, brands.loading)}
  {brands.error && <p className="audit-error">{brands.error}</p>}
  {!brands.loading && brands.items.length === 0 && !brands.error && (
    <p className="audit-empty">Aucune marque.</p>
  )}
  {brands.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Nom</th>
          </tr>
        </thead>
        <tbody>
          {brands.items.map((b) => (
            <tr key={b.id}>
              <td>{b.id}</td>
              <td>{b.name}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

<section className="audit-section">

```

```

{renderHeader('Taxes', taxes.items.length, taxes.loading)}
{taxes.error && <p className="audit-error">{taxes.error}</p>}
{!taxes.loading && taxes.items.length === 0 && !taxes.error && (
  <p className="audit-empty">Aucune taxe.</p>
)}
{taxes.items.length > 0 && (
  <div className="audit-table-wrap">
    <table className="audit-table">
      <thead>
        <tr>
          <th>ID</th>
          <th>Nom</th>
          <th>Taux</th>
        </tr>
      </thead>
      <tbody>
        {taxes.items.map((t) => (
          <tr key={t.id}>
            <td>{t.id}</td>
            <td>{t.name}</td>
            <td>{t.rate}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
)}
</section>

<section className="audit-section">
  {renderHeader('Groupes de taxe', taxGroups.items.length, taxGroups.loading)}
  {taxGroups.error && <p className="audit-error">{taxGroups.error}</p>}
  {!taxGroups.loading && taxGroups.items.length === 0 && !taxGroups.error && (
    <p className="audit-empty">Aucun groupe de taxe.</p>
  )}
  {taxGroups.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Nom</th>
          </tr>
        </thead>
        <tbody>
          {taxGroups.items.map((g) => (
            <tr key={g.id}>
              <td>{g.id}</td>
              <td>{g.name}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

<section className="audit-section">
  {renderHeader('Regles de taxe', taxRules.items.length, taxRules.loading)}
  {taxRules.error && <p className="audit-error">{taxRules.error}</p>}
  {!taxRules.loading && taxRules.items.length === 0 && !taxRules.error && (
    <p className="audit-empty">Aucune regle de taxe.</p>
  )}
  {taxRules.items.length > 0 && (
    <div className="audit-table-wrap">
      <table className="audit-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Groupe</th>
            <th>Taxe</th>
          </tr>
        </thead>
        <tbody>
          {taxRules.items.map((r) => (
            <tr key={r.id}>
              <td>{r.id}</td>
              <td>{r.group}</td>
              <td>{r.rate}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</section>

```

```

        <th>Pays</th>
      </tr>
    </thead>
    <tbody>
      {taxRules.items.map((r) => (
        <tr key={r.id}>
          <td>{r.id}</td>
          <td>{r.groupId}</td>
          <td>{r.taxId}</td>
          <td>{r.countryId}</td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
)}
</section>
</div>
);
}

```

renderHeader() — ligne 103

Fonction : **render header** — voir le code ci-dessous.

```

const renderHeader = (title: string, count: number, loading: boolean) => (
  <div className="audit-section-header">
    <h2 className="audit-section-title">{title}</h2>
    <span className="audit-count">{loading ? '...' : count}</span>
  </div>
);

```



```

const [loading, setLoading]      = useState(true);
const [error, setError]          = useState<string | null>(null);

// Sélection en attente de confirmation
const [pendingChange, setPendingChange] = useState<{
  order: PSOrder;
  newValue: number;
  oldValue: number;
} | null>(null);
const [applying, setApplying] = useState(false);
const [deletingZombies, setDeletingZombies] = useState(false);

const load = useCallback(async () => {
  setLoading(true);
  setError(null);
  try {
    const ps = await fetchPSOrders();
    // Sort by date descending
    ps.sort((a, b) => (b.date ?? '').localeCompare(a.date ?? ''));
    setOrders(ps);
  } catch {
    setError('Impossible de charger les commandes.');
```

} finally {
 setLoading(false);
 }
 }
}, []);

useEffect(() => { load(); }, [load]);

// Vérifier si la transition est autorisée
const isTransitionAllowed = (oldState: number, newState: number): boolean => {
 // Règle: "dans le panier" (1) → paiement effectué (2) ou annulé (6) : OK
 if (oldState === 1 && (newState === 2 || newState === 6)) {
 return true;
 }
 // paiement effectué (2) → annulé (6) : OK
 if (oldState === 2 && newState === 6) {
 return true;
 }
 // annulé (6) → paiement effectué (2) : rare mais possible
 if (oldState === 6 && newState === 2) {
 return true;
 }
 // Même état : pas de changement
 if (oldState === newState) {
 return false;
 }
 // TOUT VERS "dans le panier" (1) : INTERDIT
 if (newState === 1) {
 return false;
 }
 // Autres cas non autorisés
 return false;
};

const handleSelectChange = (order: PSOrder, newValue: string) => {
 const newState = parseInt(newValue, 10);
 const oldState = order.currentState;

 // Vérifier si la transition est autorisée
 if (!isTransitionAllowed(oldState, newState)) {
 alert('Transition impossible : "\${psStatusLabel(oldState)}" → "\${psStatusLabel(newState)}" n'est pas au
 return;
 }

 setPendingChange({ order, newValue: newState, oldValue: oldState });
};

// Dans ton composant OrderList.tsx

const handleConfirmChange = async () => {

```

if (!pendingChange) return;
setApplying(true);
const { order, newValue, oldValue } = pendingChange;

try {
  let success = false;

  // CAS SPÉCIFIQUE : Panier (1) -> Commande (2 ou 6)
  if (oldValue === 1 && (newValue === 2 || newValue === 6)) {
    // Ici, on appelle une méthode spécifique du service
    // car transformer un panier nécessite souvent de créer l'objet Order
    success = await transformCartToOrder(order, newValue);
  } else {
    // CAS CLASSIQUE : Changement de statut d'une commande existante
    success = await updatePSOrderStatus(order.id, newValue);
  }

  if (success) {
    // Enregistrer les mouvements de stock
    const ref = order.reference || `#${order.id}`;
    if (newValue === 2) {
      // Sortie stock : commande validée
      const rows = order.cartRows?.length
        ? order.cartRows
        : await fetchOrderRows(order.id);
      stockService.recordOrderMovements(rows, ref, 'sortie').catch(() => {});
    } else if (newValue === 6 && oldValue === 2) {
      // Entrée stock : commande annulée (retour)
      const rows = await fetchOrderRows(order.id);
      stockService.recordOrderMovements(rows, ref, 'entree').catch(() => {});
    }

    // On rafraîchit la liste complète car l'ID de la commande
    // risque d'avoir changé (PrestaShop crée un nouvel ID Order différent du Cart ID)
    await load();
  }
} catch (err) {
  setError("L'opération a échoué.");
} finally {
  setApplying(false);
  setPendingChange(null);
}
};

const handleCancelChange = () => setPendingChange(null);

const zombieCount = orders.filter(o => o.currentState === 1 && o.totalPaid === 0).length;

const handleDeleteZombies = async () => {
  if (zombieCount === 0) return;
  if (!window.confirm(`Supprimer ${zombieCount} panier(s) vide(s) (0,00 €) ? Cette action est irréversible.
  setDeletingZombies(true);
  const { deleted, failed } = await deleteZombieCarts(orders);
  setDeletingZombies(false);
  if (failed > 0) {
    alert(`${deleted} panier(s) supprimé(s), ${failed} échec(s).`);
  }
  await load();
};

// Déterminer les options disponibles selon l'état actuel
const getAvailableOptions = (currentState: number): { value: number; label: string }[] => {
  const allOptions = ALLOWED_PS_STATES;

  // Filtrer selon les transitions autorisées
  return allOptions.filter(opt => {
    // Même état : on le garde (option courante)
    if (opt.value === currentState) return true;
    // Vérifier si la transition est autorisée
    return isTransitionAllowed(currentState, opt.value);
  });
};

```

```
// ■■■ Rendu ██████████
```

```
return (  
  <div className="orders-page">  
    /* Confirmation modale */  
    {pendingChange && (  
      <div className="orders-modal-overlay">  
        <div className="orders-modal">  
          <h3 className="orders-modal-title">Confirmer le changement</h3>  
          <p className="orders-modal-text">  
            Modifier le statut de la commande <strong>#{pendingChange.order.reference || pendingChange orde  
          </p>  
          <div className="orders-modal-state-change">  
            <span className="old-state">{psStatusLabel(pendingChange.oldValue)}</span>  
            <span className="arrow">-></span>  
            <span className="new-state">{psStatusLabel(pendingChange.newValue)}</span>  
          </div>  
          <div className="orders-modal-actions">  
            <button className="btn btn-secondary" onClick={handleCancelChange} disabled={applying}>  
              Annuler  
            </button>  
            <button className="btn btn-primary" onClick={handleConfirmChange} disabled={applying}>  
              {applying ? 'En cours...' : 'Confirmer'}  
            </button>  
          </div>  
        </div>  
      )}  
    )  
  
    <div className="orders-toolbar">  
      <button className="btn btn-secondary" onClick={load} disabled={loading}>  
        {loading ? 'Chargement...' : 'Actualiser'}  
      </button>  
      {zombieCount > 0 && (  
        <button  
          className="btn btn-danger"  
          onClick={handleDeleteZombies}  
          disabled={deletingZombies || loading}  
          title="Supprimer tous les paniers sans produit (0,00 €)"  
        >  
          {deletingZombies ? 'Suppression...' : `Supprimer ${zombieCount} panier(s) vide(s)`}  
        </button>  
      )}  
      <span className="orders-count">  
        {orders.length} élément(s) ({orders.filter(o => o.currentState === 1).length} panier(s), {orders.fi  
      </span>  
    </div>  
  
    {error && <p className="orders-error">{error}</p>}  
  
    {loading && orders.length === 0 ? (  
      <div className="orders-loading">Chargement des commandes...</div>  
    ) : orders.length === 0 ? (  
      <div className="orders-empty">Aucune commande ou panier trouvé.</div>  
    ) : (  
      <div className="orders-table-wrapper">  
        <table className="orders-table">  
          <thead>  
            <tr>  
              <th>N°</th>  
              <th>Client</th>  
              <th>Montant TTC</th>  
              <th>Date</th>  
              <th>Statut</th>  
              <th>Modifier</th>  
            </tr>  
          </thead>  
          <tbody>  
            {orders.map((order) => {
```

```

const currentValue = String(order.currentState);
const availableOptions = getAvailableOptions(order.currentState);

return (
  <tr key={order.id} className={order.currentState === 1 ? 'order-row--cart' : ''}>
    <td className="orders-cell-ref">
      {order.currentState === 1 && <span className="cart-icon">■</span>}
      #{order.reference || order.id}
    </td>
    <td>{order.customerName}</td>
    <td className="orders-cell-amount">{formatAmount(order.totalPaid)}</td>
    <td>{formatDate(order.date)}</td>
    <td>
      <span className={`status-badge ${orderStatusClass(order.currentState)} `}>
        {psStatusLabel(order.currentState)}
      </span>
    </td>
    <td>
      <select
        className="orders-status-select"
        value={currentValue}
        onChange={(e) => handleSelectChange(order, e.target.value)}
      >
        {availableOptions.map((opt) => (
          <option key={opt.value} value={opt.value}>
            {opt.label}
          </option>
        ))}
      </select>
    </td>
  </tr>
);
  )}
</tbody>
</table>
</div>
)}
</div>
);
}

```

isTransitionAllowed() — ligne 75

Description (commentaire d'origine) : Vérifier si la transition est autorisée

```

const isTransitionAllowed = (oldState: number, newState: number): boolean => {
  // Règle: "dans le panier" (1) → paiement effectué (2) ou annulé (6) : OK
  if (oldState === 1 && (newState === 2 || newState === 6)) {
    return true;
  }
  // paiement effectué (2) → annulé (6) : OK
  if (oldState === 2 && newState === 6) {
    return true;
  }
  // annulé (6) → paiement effectué (2) : rare mais possible
  if (oldState === 6 && newState === 2) {
    return true;
  }
  // Même état : pas de changement
  if (oldState === newState) {
    return false;
  }
  // TOUT VERS "dans le panier" (1) : INTERDIT
  if (newState === 1) {
    return false;
  }
  // Autres cas non autorisés
  return false;
}

```

handleSelectChange() — ligne 99

Fonction : `handle select change` — voir le code ci-dessous.

```
const handleSelectChange = (order: PSOrder, newValue: string) => {
  const newState = parseInt(newValue, 10);
  const oldState = order.currentState;

  // Vérifier si la transition est autorisée
  if (!isTransitionAllowed(oldState, newState)) {
    alert(`Transition impossible : "${psStatusLabel(oldState)}" → "${psStatusLabel(newState)}" n'est pas au
    return;
  }

  setPendingChange({ order, newValue: newState, oldValue: oldState });
}
```

handleConfirmChange() — ligne 114

Description (commentaire d'origine) : Dans ton composant OrderList.tsx

```
const handleConfirmChange = async () => {
  if (!pendingChange) return;
  setApplying(true);
  const { order, newValue, oldValue } = pendingChange;

  try {
    let success = false;

    // CAS SPÉCIFIQUE : Panier (1) -> Commande (2 ou 6)
    if (oldValue === 1 && (newValue === 2 || newValue === 6)) {
      // Ici, on appelle une méthode spécifique du service
      // car transformer un panier nécessite souvent de créer l'objet Order
      success = await transformCartToOrder(order, newValue);
    } else {
      // CAS CLASSIQUE : Changement de statut d'une commande existante
      success = await updatePSOrderStatus(order.id, newValue);
    }

    if (success) {
      // Enregistrer les mouvements de stock
      const ref = order.reference || `#${order.id}`;
      if (newValue === 2) {
        // Sortie stock : commande validée
        const rows = order.cartRows?.length
          ? order.cartRows
          : await fetchOrderRows(order.id);
        stockService.recordOrderMovements(rows, ref, 'sortie').catch(() => {});
      } else if (newValue === 6 && oldValue === 2) {
        // Entrée stock : commande annulée (retour)
        const rows = await fetchOrderRows(order.id);
        stockService.recordOrderMovements(rows, ref, 'entree').catch(() => {});
      }

      // On rafraîchit la liste complète car l'ID de la commande
      // risque d'avoir changé (PrestaShop crée un nouvel ID Order différent du Cart ID)
      await load();
    }
  } catch (err) {
    setError("L'opération a échoué.");
  } finally {
    setApplying(false);
    setPendingChange(null);
  }
}
```

handleDeleteZombies() — ligne 163

Fonction : `handle delete zombies` — voir le code ci-dessous.

```
const handleDeleteZombies = async () => {
  if (zombieCount === 0) return;
  if (!window.confirm(`Supprimer ${zombieCount} panier(s) vide(s) (0,00 €) ? Cette action est irréversible.`))
    return;
  setDeletingZombies(true);
  const { deleted, failed } = await deleteZombieCarts(orders);
  setDeletingZombies(false);
  if (failed > 0) {
    alert(`${deleted} panier(s) supprimé(s), ${failed} échec(s).`);
  }
  await load();
}
```

getAvailableOptions() — ligne 177

Description (commentaire d'origine) : Déterminer les options disponibles selon l'état actuel

```
const getAvailableOptions = (currentState: number): { value: number; label: string }
```

Fichier :
src/components/StockUpdate.tsx

Lignes : 356 • Fonctions détectées : 6

QtyBadge() — ligne 13

Description	(commentaire	d'origine)	:
Sous-composant : Badge stock			

```
function QtyBadge({ qty }: { qty: number }) {
  const cls =
    qty === 0 ? 'badge-qty zero'
    : qty <= LOW_STOCK_THRESHOLD ? 'badge-qty low'
    : 'badge-qty ok';
  return <span className={cls}>{qty}</span>;
}
```

StockUpdate() — ligne 24

[illegible]

```
export function StockUpdate() {
  const [lines, setLines] = useState<StockLine[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [tab, setTab] = useState<Tab>('stock');
  const [movements, setMovements] = useState<StockMovement[]>([]);

  // Filtrer
  const [search, setSearch] = useState('');
  const [stockFilter, setStockFilter] = useState<StockFilter>('all');

  // État par ligne (clé = StockLine.key)
  const [qtyInputs, setQtyInputs] = useState<Record<string, string>>({});
  const [noteInputs, setNoteInputs] = useState<Record<string, string>>({});
  const [updating, setUpdating] = useState<Record<string, boolean>>({});
```



```
{/* ■■■■ ONGLET STOCKS ■■■■ */}
{tab === 'stock' && (
</>
    { /* ■■ Barre d'outils ■■ */ }
    <div className="su-toolbar">
        <div className="su-search-wrap">
            <span className="su-search-icon">🔍</span>
            <input
                type="search"
                placeholder="Nom, référence, EAN..."
                value={search}
                onChange={e => setSearch(e.target.value)}
                className="su-search"
            />
        </div>
        <div className="su-filter-pills">
            {[('all', 'zero', 'low') as StockFilter[]].map(f => (
                <button
                    key={f}
                    className={`su-pill ${stockFilter === f ? 'active' : ''} pill-${f}`}
                    onClick={() => setStockFilter(f)}
                >
                    {f === 'all'   && `Tous (${lines.length})`}
                    {f === 'zero' && `Épuisé (${counts.zero})`}
                    {f === 'low'  && `Bas ≤${LOW_STOCK_THRESHOLD} (${counts.low})`}
                </button>
            ))}
        </div>
    </div>

    { /* ■■ Liste des produits ■■ */ }
    <div className="su-list">
        {grouped.size === 0 && (
            <div className="su-empty">Aucun produit trouvé pour cette recherche.</div>)
        }

        [...grouped.entries()].map(([productId, productLines]) => {
            const first = productLines[0];
            const isCombinations = first.productType === 'combinations';

            return (
                <div key={productId} className="su-group">
                    { /* En-tête groupe produit */ }
                    <div className="su-group-header">
                        <span className="su-group-name">{first.productName}</span>
                        {isCombinations && (
                            <span className="su-tag su-tag-combo">
                                {productLines.length} déclinaison{productLines.length > 1 ? 's' : ''}
                            </span>
                        )}
                    </div>

                    { /* Lignes (1 par produit simple, N par déclinaison) */ }
                    {productLines.map(line => (
                        <div
                            key={line.key}
                            className={`su-line ${line.quantity === 0 ? 'is-zero' : line.quantity <= LOW_STOCK_THRESHOLD ? 'warning' : ''}}`
                        >
                            { /* Infos */ }
                            <div className="su-line-info">
                                {line.combinationLabel
                                    ? <span className="su-combo-label">{line.combinationLabel}</span>
                                      : <span className="su-combo-label su-combo-label--simple">Produit simple</span>}
                            </div>
                            <div className="su-line-meta">
                                {line.reference && <span>Réf : {line.reference}</span>}
                                {line.ean13 && <span>EAN : {line.ean13}</span>}
                            </div>
                        </div>
                    ))}

                    { /* Stock actuel */ }
```



```

return (
  <div className="reset-page">
    <div className="reset-idle-card">
      <div className="reset-warning-banner">
        <span className="reset-warning-icon">!</span>
        <div>
          <p className="reset-warning-title">Zone de danger</p>
          <p className="reset-warning-text">
            Cette opération supprime toutes les données de la boutique : produits, catégories,
            clients, adresses, fournisseurs, marques et déclinaisons.
            Cette action est <strong>irréversible</strong>.
          </p>
        </div>
      </div>

      <div className="reset-entity-list">
        <h3 className="reset-entity-title">Données qui seront supprimées</h3>
        <ul className="reset-entity-items">
          {RESET_STEPS.map((s) => (
            <li key={s.label} className="reset-entity-item">
              <span className="reset-entity-dot" />
              {s.label}
            </li>
          ))}
        </ul>
      </div>

      <button className="btn btn-danger-large" onClick={handleConfirm}>
        Réinitialiser toutes les données
      </button>
    </div>
  </div>
);
}

```

handleReset () — ligne 52

Fonction : `handle reset` — voir le code ci-dessous.

```

const handleReset = async () => {
  setPhase('running');
  setCurrentStep(0);
  setStepProgress(0);
  setStepResults([]);

  const results: StepResult[] = [];

  for (let i = 0; i < RESET_STEPS.length; i++) {
    const step = RESET_STEPS[i];
    setCurrentStep(i);
    setStepProgress(0);

    try {
      const res = await step.run((done, total) => {
        setStepProgress(total > 0 ? Math.round((done / total) * 100) : 0);
      });
      results.push({ label: step.label, result: res });
    } catch (err) {
      results.push({
        label: step.label,
        result: null,
        error: err instanceof Error ? err.message : 'Erreur inconnue',
      });
    }

    setStepResults([...results]);
  }

  setPhase('done');
}

```

```
}
```

handleRetry() — ligne 84

Fonction : `handle retry` — voir le code ci-dessous.

```
const handleRetry = () => {  
  setPhase('idle');  
  setStepResults([]);  
}
```

Composants — Boutique (FrontOffice)

Fichier : `src/components/ShopHome.tsx`

Lignes : 500 • Fonctions détectées : 6

ProductImage() — ligne 19

Description	(commentaire	d'origine)	:	■■	Image	avec	fallback

```
const ProductImage: React.FC<{ src?: string; alt: string; className?: string }> = ({ src, alt, className }) => {
  const [error, setError] = useState(false);
  if (!src || error) {
    return (
      <div className={`flex h-full w-full items-center justify-center rounded-2xl border border-dashed border-gray-200`} >
        <svg className="h-8 w-8" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.2">
          <rect x="3" y="3" width="18" height="18" rx="2"/>
          <circle cx="8.5" cy="8.5" r="1.5"/>
          <polyline points="21 15 16 10 5 21"/>
        </svg>
      </div>
    );
  }
  return <img src={src} alt={alt} className={`h-full w-full object-cover ${className ?? ''}`} onError={() => setError(true)} />;
}
```

StockBadge() — ligne 37

[illegible]

```
const StockBadge: React.FC<{ qty: number }> = ({ qty }) => {
  const { label, level } = stockStatus(qty);
  const styles = { ok: 'border-emerald-200 bg-emerald-50 text-emerald-700', low: 'border-amber-200 bg-amber-50 text-amber-700' };
  return <span className={`inline-flex items-center rounded-full border px-3 py-1 text-xs font-semibold ${styles[level]}`}>{label}</span>;
}
```

SkeletonCard() — ligne 45

Description	(commentaire	d'origine)	:	■■	Skeleton	card

```
const SkeletonCard = () => (
  <div className="animate-pulse rounded-2xl border border-slate-200 bg-white p-4 shadow-sm">
    <div className="aspect-[4/3] rounded-2xl bg-slate-100" />
    <div className="mt-4 space-y-3">
      <div className="h-4 w-2/3 rounded-full bg-slate-100" />
      <div className="h-3 w-1/3 rounded-full bg-slate-100" />
      <div className="h-10 w-full rounded-full bg-slate-100" />
    </div>
  </div>
);
```

ShopHome() — ligne 69

```
addItem({ id: product.id, name: product.name, priceHt: product.priceHt, priceTtc: product.priceTtc, taxRa
```

```

// ■■ Date de référence ■■
const nowMs = useMemo(() => {
  if (!dateRef) return Date.now();
  const d = new Date(dateRef);
  d.setHours(23, 59, 59, 0);
  return Number.isNaN(d.getTime()) ? Date.now() : d.getTime();
}, [dateRef]);

const isSimulated = dateRef !== '';

// ■■ Validation prix ■■
const priceMinNum = priceMin !== '' ? parseFloat(priceMin.replace(',', '.')) : null;
const priceMaxNum = priceMax !== '' ? parseFloat(priceMax.replace(',', '.')) : null;
const priceError = priceMinNum !== null && priceMaxNum !== null && priceMinNum > priceMaxNum;

// ■■ Filtrage + tri client-side ■■
const filtered = useMemo(() => {
  let result = allProducts;

  if (debouncedSearch) {
    const q = debouncedSearch.toLowerCase();
    result = result.filter(p => p.name.toLowerCase().includes(q) || p.reference.toLowerCase().includes(q));
  }
  if (priceMinNum !== null && !Number.isNaN(priceMinNum)) {
    result = result.filter(p => p.priceTtc >= priceMinNum);
  }
  if (priceMaxNum !== null && !Number.isNaN(priceMaxNum) && !priceError) {
    result = result.filter(p => p.priceTtc <= priceMaxNum);
  }
  if (badgeFilter !== 'all') {
    result = result.filter(p => getProductBadge(p.date_availability_produit, nowMs) === badgeFilter.toUpperCase());
  }

  switch (sortBy) {
    case 'price-asc': return [...result].sort((a, b) => a.priceTtc - b.priceTtc);
    case 'price-desc': return [...result].sort((a, b) => b.priceTtc - a.priceTtc);
    case 'newest': return [...result].sort((a, b) => {
      const da = parseAvailabilityDate(a.date_availability_produit)?.getTime() ?? 0;
      const db = parseAvailabilityDate(b.date_availability_produit)?.getTime() ?? 0;
      return db - da;
    });
    default: return result;
  }
}, [allProducts, debouncedSearch, priceMinNum, priceMaxNum, priceError, badgeFilter, nowMs, sortBy]);

const hotCount = useMemo(() => allProducts.filter(p => getProductBadge(p.date_availability_produit, nowMs) === 'hot'));
const newCount = useMemo(() => allProducts.filter(p => getProductBadge(p.date_availability_produit, nowMs) === 'new'));

// ■■ Filtres actifs ■■
const hasActiveFilters = !(debouncedSearch || categoryId || priceMin || priceMax || badgeFilter !== 'all');
const selectedCategoryName = categories.find(c => c.id === categoryId)?.name;

const resetFilters = () => {
  setSearchInput(''); setDebSearch(''); setCategoryId(null);
  setPriceMin(''); setPriceMax(''); setSortBy('default');
  setBadgeFilter('all'); setDateRef('');
};

return (
  <div className="space-y-12 pb-20 pt-10">

    {/* ■■ Hero ■■ */}
    <section id="collection" className="relative overflow-hidden rounded-3xl border border-slate-200 bg-gray-50">
      <div className="grid gap-10 lg:grid-cols-[1.1fr,0.9fr]">
        <div className="space-y-6">
          <span className="inline-flex w-fit items-center rounded-full bg-emerald-100 px-4 py-1 text-xs font-medium">
            <h1 className="font-display text-4xl leading-tight text-slate-900 sm:text-5xl">Une boutique moderne
            <p className="max-w-xl text-lg leading-relaxed text-slate-600">Des pièces sélectionnées, une navigation
            <div className="flex flex-wrap gap-3">
              <a href="#essentiels" className="rounded-full bg-slate-900 px-5 py-3 text-sm font-semibold text-white">Essentiels
              <Link to="/shop/cart" className="rounded-full border border-slate-300 bg-white px-5 py-3 text-sm font-medium">Ajouter au panier
            </div>
          </span>
        </div>
      </div>
    </section>
  </div>
);

```

```

    </div>
    <div className="grid gap-4 sm:grid-cols-3">
      {[{ label: 'Livraison', value: '48h' }, { label: 'Retours', value: '30 jours' }, { label: 'Supp
        <div key={item.label} className="rounded-2xl border border-slate-200 bg-white/70 px-4 py-3 sh
          <p className="text-xs uppercase tracking-[0.2em] text-slate-400">{item.label}</p>
          <p className="font-display text-base text-slate-900">{item.value}</p>
        </div>
      )]}
    </div>
  </div>
  <div className="flex flex-col gap-4">
    <div className="grid grid-cols-2 gap-4">
      {[
        { label: 'Produits', value: loading ? '...' : String(allProducts.length) },
        { label: 'Catégories', value: loading ? '...' : String(categories.length) },
        { label: 'Nouveautés', value: loading ? '...' : String(newCount) },
        { label: 'Tendances', value: loading ? '...' : String(hotCount) },
      ].map(item => (
        <div key={item.label} className="rounded-2xl border border-slate-200 bg-white/80 px-4 py-4 sh
          <p className="text-xs uppercase tracking-[0.2em] text-slate-400">{item.label}</p>
          <p className="mt-1 font-display text-3xl text-slate-900">{item.value}</p>
        </div>
      )]}
    </div>
    <div className="rounded-2xl border border-slate-200 bg-white/70 px-4 py-3 text-sm text-slate-600">
      Catalogue mis à jour · Édition 2026
    </div>
  </div>
</div>
</section>

{/* ■■ Panneau de filtres ■■ */}
<section id="essentiels" className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm space-y-4">

  {/* Ligne 1 : recherche + tri */}
  <div className="flex flex-col gap-3 sm:flex-row sm:items-center">
    <div className="relative flex-1">
      <svg className="absolute left-4 top-1/2 h-4 w-4 -translate-y-1/2 text-slate-400" viewBox="0 0 24 24">
        <circle cx="11" cy="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/>
      </svg>
      <input
        type="text"
        value={searchInput}
        onChange={e => setSearchInput(e.target.value)}
        placeholder="Rechercher un produit, une référence..."
        className="w-full rounded-full border border-slate-200 bg-slate-50 py-2.5 pl-11 pr-10 text-sm text-slate-900"
      />
      {searchInput && (
        <button type="button" onClick={() => setSearchInput('')} className="absolute right-3 top-1/2 -translate-y-1/2"
          <svg className="h-4 w-4" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
            <path d="M16.65 16.65 21 21" />
          </svg>
        </button>
      )}
    </div>
    <select
      value={sortBy}
      onChange={e => setSortBy(e.target.value as SortKey)}
      className="rounded-full border border-slate-200 bg-slate-50 px-4 py-2.5 text-sm font-medium text-slate-900"
    >
      <option value="default">Tri par défaut</option>
      <option value="price-asc">Prix croissant</option>
      <option value="price-desc">Prix décroissant</option>
      <option value="newest">Nouveautés d'abord</option>
    </select>
  </div>

  {/* Ligne 2 : catégorie + prix + badges */}
  <div className="flex flex-wrap items-center gap-3">
    {/* Catégorie */}
    <select
      value={categoryId ?? ''}
      onChange={e => setCategoryId(e.target.value ? Number(e.target.value) : null)}
    >

```

```

        className="rounded-full border border-slate-200 bg-slate-50 px-4 py-2 text-sm font-medium text-slate-700"
      >
        <option value="">Toutes catégories</option>
        {categories.map(c => <option key={c.id} value={c.id}>{c.name}</option>)}
      </select>

    { /* Prix min / max */ }
    <div className={`flex items-center gap-2 rounded-full border px-3 py-1.5 ${priceError ? 'border-rose-500' : 'border-slate-200'} bg-slate-50`} >
      <span className="text-xs text-slate-400">€</span>
      <input
        type="number"
        min="0"
        value={priceMin}
        onChange={e => setPriceMin(e.target.value)}
        placeholder="Min"
        className="w-16 bg-transparent text-sm text-slate-700 placeholder:text-slate-400 focus:outline-none"
      />
      <span className="text-slate-300">—</span>
      <input
        type="number"
        min="0"
        value={priceMax}
        onChange={e => setPriceMax(e.target.value)}
        placeholder="Max"
        className="w-16 bg-transparent text-sm text-slate-700 placeholder:text-slate-400 focus:outline-none"
      />
    </div>

    { /* Séparateur */ }
    <div className="hidden h-6 w-px bg-slate-200 sm:block" />

    { /* Badges HOT / NEW */ }
    <div className="flex gap-2">
      {[
        { key: 'all' as BadgeFilter, label: 'Tout', count: allProducts.length },
        { key: 'hot' as BadgeFilter, label: '■ HOT', count: hotCount },
        { key: 'new' as BadgeFilter, label: '■ NEW', count: newCount },
      ]}.map(({ key, label, count }) => {
        const active = badgeFilter === key;
        return (
          <button
            key={key}
            type="button"
            onClick={() => setBadgeFilter(key)}
            className={`rounded-full border px-3 py-1.5 text-xs font-semibold transition ${active ? 'border-rose-500' : 'border-slate-200'} ${active ? 'bg-white' : 'bg-slate-50'}`}
          >
            {label}
            {count > 0 && (
              <span className={`ml-1 rounded-full px-1.5 py-0.5 text-[10px] ${active ? 'bg-white/20' : 'bg-slate-200'} font-size-sm`}>{count}</span>
            )}
          </button>
        );
      })}
    </div>
  </div>

  { /* Erreur prix */ }
  {priceError && (
    <p className="flex items-center gap-1.5 text-xs text-rose-600">
      <svg className="h-3.5 w-3.5" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
        <circle />
      </svg>
      Le prix minimum ne peut pas être supérieur au prix maximum.
    </p>
  )}

  { /* Ligne 3 : chips actifs + compteur + reset */ }
  <div className="flex flex-wrap items-center justify-between gap-2">
    <div className="flex flex-wrap gap-2">
      {debouncedSearch && <FilterChip label={`"${debouncedSearch}"`} onRemove={() => setSearchInput('')} />}
      {selectedCategoryName && <FilterChip label={selectedCategoryName} onRemove={() => setCategoryId(null)} />}
      {priceMin && !priceError && <FilterChip label={`≥ ${priceMin} €`} onRemove={() => setPriceMin('')} />}
      {priceMax && !priceError && <FilterChip label={`≤ ${priceMax} €`} onRemove={() => setPriceMax('')} />}
    </div>
    <div>
      <span>{count}</span>
      <button type="button" onClick={() => reset()}>Reset</button>
    </div>
  </div>

```

```

        {badgeFilter !== 'all' && <FilterChip label={badgeFilter === 'hot' ? '■ HOT' : '■ NEW'} onRemove=
        {sortBy !== 'default' && <FilterChip label={sortBy === 'price-asc' ? 'Prix ↑' : sortBy === 'price
    </div>
    <div className="flex items-center gap-3 text-xs text-slate-400">
        <span>{filtered.length} résultat{filtered.length !== 1 ? 's' : ''}</span>
        {hasActiveFilters && (
            <button type="button" onClick={resetFilters} className="rounded-full border border-slate-200 bg-
                Réinitialiser
            </button>
        )}
    </div>
</div>

/* Ligne 4 : date de référence (simulation badges) */
<div className={`flex flex-wrap items-center gap-3 rounded-xl border px-4 py-2.5 text-sm transition-c
    <svg className={`h-4 w-4 shrink-0 ${isSimulated ? 'text-amber-500' : 'text-slate-400'}`} viewBox="0
        <rect x="3" y="4" width="18" height="18" rx="2"/><line x1="16" y1="2" x2="16" y2="6"/><line x1="8
    </svg>
    <span className={`text-xs font-medium ${isSimulated ? 'text-amber-700' : 'text-slate-500'}`}>Simuler
    <input
        type="date"
        value={dateRef}
        onChange={e => { setDateRef(e.target.value); setBadgeFilter('all'); }}
        className={`rounded-lg border px-3 py-1 text-xs font-medium focus:outline-none focus:ring-2 ${isS
    />
    {isSimulated ? (
        <>
            <span className="text-xs text-amber-600">Badges calculés à cette date</span>
            <button type="button" onClick={() => { setDateRef(''); setBadgeFilter('all'); }} className="ml-1
                Aujourd'hui
            </button>
        </>
    ) : (
        <span className="text-xs text-slate-400">Modifie la date pour tester les badges HOT/NEW à n'import
    )}
</div>
</section>

/* ■■ Promo ■■ */
<section id="promos" className="flex flex-col gap-6 rounded-3xl border border-slate-200 bg-white px-6 py-
    <div>
        <p className="text-xs uppercase tracking-[0.2em] text-slate-400">Offres du moment</p>
        <h2 className="mt-2 font-display text-2xl text-slate-900">Jusqu'à -20% sur les essentiels urbains</h2>
        <p className="mt-2 text-sm text-slate-600">Valable cette semaine sur une sélection de produits phar
    </div>
    <a href="#essentiels" className="rounded-full bg-emerald-500 px-5 py-3 text-sm font-semibold text-whi
</section>

/* ■■ Erreur ■■ */
{error && (
    <div className="rounded-2xl border border-rose-200 bg-rose-50 px-4 py-3 text-sm text-rose-700">{error}
)}

/* ■■ En-tête grille ■■ */
<div className="flex flex-col gap-3 border-b border-slate-200 pb-4 md:flex-row md:items-end md:justify-f
    <div>
        <p className="text-xs uppercase tracking-[0.2em] text-slate-400">Catalogue</p>
        <h2 className="font-display text-2xl text-slate-900">
            {selectedCategoryName ?? 'Les collections du moment'}
        </h2>
    </div>
    <div>
        <p className="text-xs uppercase tracking-[0.2em] text-slate-400">
            {filtered.length} référence{filtered.length !== 1 ? 's' : ''}
        </p>
    </div>
</div>

/* ■■ Grille produits ■■ */
<div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
    {loading
        ? Array.from({ length: 6 }).map((_, i) => <SkeletonCard key={i} />)
        : filtered.length === 0
    }

```

```

? (
  <div className="col-span-full flex flex-col items-center gap-3 rounded-2xl border border-dashed"
    <svg className="h-8 w-8 text-slate-300" viewBox="0 0 24 24" fill="none" stroke="currentColor"
    <p className="text-sm font-medium text-slate-500">
      {badgeFilter === 'hot' ? 'Aucun produit HOT en ce moment (moins de 24h).'
      : badgeFilter === 'new' ? 'Aucun produit NEW en ce moment (moins de 7 jours).'
      : hasActiveFilters ? 'Aucun résultat pour ces filtres.'
      : 'Aucun produit disponible.'}
    </p>
    {hasActiveFilters && (
      <button type="button" onClick={resetFilters} className="rounded-full border border-slate-200"
        Réinitialiser les filtres
      </button>
    )}
  </div>
)
: filtered.map(product => {
  const isAdded = added === product.id;
  const isDisabled = product.quantity === 0;
  const btnCls = isDisabled
    ? 'cursor-not-allowed border-slate-200 bg-slate-100 text-slate-400'
    : isAdded
      ? 'border-emerald-500 bg-emerald-500 text-white'
      : 'border-slate-900 bg-slate-900 text-white hover:bg-slate-800';

  return (
    <div key={product.id} className="group flex h-full flex-col overflow-hidden rounded-2xl border"
      <Link to={` /shop/${product.id}`} className="relative">
        <div className="relative aspect-[4/3] overflow-hidden border-b border-slate-200 bg-slate-100"
          <ProductBadge dateAvailability={product.date_availability_produit} className="availab"
          <ProductImage src={product.imageUrl} alt={product.name} className="transition duration"
        </div>
        <div className="pointer-events-none absolute inset-0 flex items-end justify-end p-4 opa"
          <span className="rounded-full bg-slate-900/90 px-3 py-1 text-[11px] font-semibold tex"
        </div>
      </Link>

      <div className="flex flex-1 flex-col gap-3 p-5">
        <p className="text-xs uppercase tracking-[0.2em] text-slate-400">
          {product.reference ? `Réf · ${product.reference}` : 'Sélection'}
        </p>
        <h3 className="font-display text-lg leading-snug text-slate-900" title={product.name}>{
        <div className="flex items-center justify-between">
          <span className="text-base font-semibold">{formatPrice(product.priceTtc)}</span>
          <StockBadge qty={product.quantity} />
        </div>
        <div className="mt-auto flex flex-wrap gap-3">
          <Link to={` /shop/${product.id}`} className="inline-flex items-center justify-center r
            Détails
          </Link>
          <button
            className={`inline-flex items-center justify-center gap-2 rounded-full border px-4 p
            disabled={isDisabled}
            onClick={() => handleAddToCart(product)}
          >
            {isAdded ? 'Ajouté' : (
              <>
                <svg className="h-4 w-4" viewBox="0 0 24 24" fill="none" stroke="currentColor"
                  <circle cx="9" cy="21" r="1"/><circle cx="20" cy="21" r="1"/>
                  <path d="M1 1h4l2.68 13.39a2 2 0 002 1.61h9.72a2 2 0 002-1.61L23 6H6"/>
                </svg>
                Ajouter
              </>
            )}
          </button>
        </div>
      </div>
    </div>
  );
}
}

```

```

        </div>
    </div>
);
};

```

handleAddToCart() — ligne 132

Description (commentaire d'origine) : Panier

```
const handleAddToCart = (product: ShopProduct) => {
  if (product.quantity === 0) return;
  addItem({ id: product.id, name: product.name, priceHt: product.priceHt, priceTtc: product.priceTtc, taxRate: product.taxRate });
  setAdded(product.id);
  setTimeout(() => setAdded(null), 1500);
}
```

resetFilters() — ligne 190

Fonction : `reset filters` — voir le code ci-dessous.

```
const resetFilters = () => {
  setSearchInput(''); setDebSearch(''); setCategoryId(null);
  setPriceMin(''); setPriceMax(''); setSortBy('default');
  setBadgeFilter('all'); setDateRef('');
}
```

Fichier :

src/components/ProductDetail.tsx

Lignes : 419 • Fonctions détectées : 5

ImgPlaceholder() — ligne 16

Description	(commentaire	d'origine)	:	■ ■	Image	placeholder

```
const ImgPlaceholder = () => (
  <div className="detail-img-placeholder">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.2">
      <rect x="3" y="3" width="18" height="18" rx="2" />
      <circle cx="8.5" cy="8.5" r="1.5" />
      <polyline points="21 15 16 10 5 21" />
    </svg>
  </div>
);
```

Gallery() — ligne 27

[illegible]

```
const Gallery: React.FC<{ images: string[] }> = ({ images }) => {
  const [active, setActive] = useState(0);
  const [imgError, setImgError] = useState<Record<number, boolean>>({});

  if (images.length === 0) {
    return (
```

```

    <div className="detail-gallery">
      <ImgPlaceholder />
    </div>
  );
}

return (
  <div className="detail-gallery">
    <div className="detail-main-img-wrap">
      {imgError[active] ? (
        <ImgPlaceholder />
      ) : (
        <img
          src={images[active]}
          alt="Produit"
          className="detail-main-img"
          onError={() => setImgError((p) => ({ ...p, [active]: true })))}
        />
      )}
    </div>

    {images.length > 1 && (
      <div className="detail-thumbnails">
        {images.map((url, i) => (
          <button
            key={i}
            className={`detail-thumb${active === i ? ' detail-thumb--active' : ''}`}
            onClick={() => setActive(i)}
          >
            {imgError[i] ? (
              <div className="detail-thumb-placeholder" />
            ) : (
              <img
                src={url}
                alt={`Vue ${i + 1}`}
                onError={() => setImgError((p) => ({ ...p, [i]: true })))}
              />
            )}
          </button>
        ))}
      </div>
    )}
  </div>
);
}

```

QtySelector() — ligne 85

Fonction : `qty selector` — voir le code ci-dessous.

```

const QtySelector: React.FC<QtyProps> = ({ value, max, onChange }) => (
  <div className="qty-selector">
    <button
      className="qty-btn"
      onClick={() => onChange(Math.max(1, value - 1))}
      disabled={value <= 1}
    >
      -
    </button>
    <input
      type="number"
      className="qty-input"
      value={value}
      min={1}
      max={max}
      onChange={(e) => {
        const n = parseInt(e.target.value, 10);
        if (!isNaN(n)) onChange(Math.min(max, Math.max(1, n)));
      }}
    />
  </div>
);

```

```

/>
<button
  className="qty-btn"
  onClick={() => onChange(Math.min(max, value + 1))}
  disabled={value >= max}
>
  +
</button>
</div>
);

```

ProductDetail() — ligne 117

Description (commentaire d'origine) : ■■ Page ██████████
 ██████████

```
const ProductDetail: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const { addItem } = useCart();

  const [product, setProduct] = useState<ShopProduct | null>(null);
  const [images, setImages] = useState<string[]>([]);
  const [combinations, setCombinations] = useState<ShopCombination[]>([]);
  const [selectedCombo, setSelectedCombo] = useState<ShopCombination | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [qty, setQty] = useState(1);
  const [added, setAdded] = useState(false);

  // États pour le stock réel
  const [realStock, setRealStock] = useState(0);
  const [stockLoading, setStockLoading] = useState(true);
  const [stockError, setStockError] = useState<string | null>(null);

  // 1. Chargement des infos produit
  useEffect(() => {
    if (!id) return;
    setLoading(true);
    setError(null);
    fetchShopProductDetail(id)
      .then((data) => {
        if (!data) {
          setError('Produit introuvable.');
```

```

    })
    .finally(() => setStockLoading(false));
}, [id, selectedCombo]);

// Calculs de prix
const effectivePriceHt = product ? product.priceHt + (selectedCombo?.priceImpact ?? 0) : 0;
const effectivePriceTtc = product ? effectivePriceHt * (1 + product.taxRate) : 0;
const effectiveQty = realStock;
const hasCombinations = combinations.length > 0;

// Condition pour ajouter au panier
const canAdd =
  !stockLoading &&
  effectiveQty > 0 &&
  (!hasCombinations || selectedCombo !== null);
const maxQty = Math.max(1, Math.min(effectiveQty, 99));

const handleAddToCart = () => {
  if (!product || !canAdd) return;
  addItem(
    {
      id: product.id,
      attributeId: selectedCombo?.id,
      variantLabel: selectedCombo?.label,
      name: product.name,
      priceHt: effectivePriceHt,
      priceTtc: effectivePriceTtc,
      taxRate: product.taxRate,
      imageUrl: images[0],
    },
    qty,
  );
  setAdded(true);
  setTimeout(() => setAdded(false), 2000);
};

// Affichage du squelette de chargement
if (loading) {
  return (
    <div className="detail-page">
      <div className="detail-skeleton">
        <div className="detail-skeleton-img" />
        <div className="detail-skeleton-info">
          <div className="sk-line sk-line--title" />
          <div className="sk-line sk-line--price" />
          <div className="sk-line sk-line--desc" />
          <div className="sk-line sk-line--desc" />
          <div className="sk-line sk-line--btn" />
        </div>
      </div>
    </div>
  );
}

if (error || !product) {
  return (
    <div className="detail-page">
      <div className="detail-error">
        <p>{error ?? 'Produit introuvable.'}</p>
        <button className="btn-back" onClick={() => navigate('/shop')}>
          ← Retour à la boutique
        </button>
      </div>
    </div>
  );
}

const { label: stockLabel, level: stockLevel } = stockStatus(effectiveQty);
const taxPct = Math.round(product.taxRate * 10000) / 100;

// Message d'information stock selon contexte

```

```

let stockMessage = '';
if (hasCombinations && !selectedCombo) {
  stockMessage = 'Choisissez une déclinaison';
} else if (stockLoading) {
  stockMessage = 'Vérification du stock...';
} else if (stockError) {
  stockMessage = '■ Stock indisponible';
} else if (effectiveQty === 0) {
  stockMessage = 'Rupture de stock';
} else {
  stockMessage = stockLabel;
}

return (
  <div className="detail-page">
    { /* ■ Fil d'Ariane ■ */ }
    <nav className="detail-breadcrumb">
      <Link to="/shop">Boutique</Link>
      <span>/</span>
      <span>{product.name}</span>
    </nav>

    <div className="detail-layout">
      <Gallery images={images} />

      <div className="detail-info">
        <div className="detail-info-top">
          <h1 className="detail-name">{product.name}</h1>
          {product.reference && <p className="detail-ref">Réf : {product.reference}</p>}
          <ProductBadge
            dateAvailability={product.date_availability_produit}
            className="availability-badge--inline"
          />
        </div>

        <div className="detail-price">{formatPrice(effectivePriceTtc)}</div>
        <div className="detail-tax">TVA: {taxPct}%</div>

        { /* Indicateur de stock (avec gestion des déclinaisons) */ }
        {hasCombinations && !selectedCombo ? (
          <div className="stock-badge stock-badge--info">■ {stockMessage}</div>
        ) : stockLoading ? (
          <div className="stock-badge stock-badge--loading">{stockMessage}</div>
        ) : stockError ? (
          <div className="stock-badge stock-badge--error">{stockMessage}</div>
        ) : (
          <>
            <div className={`stock-badge stock-badge--${stockLevel}`}>{stockMessage}</div>
            {effectiveQty > 0 && !hasCombinations && (
              <div className="detail-stock-count">Stock: {effectiveQty} disponible(s)</div>
            )}
            {effectiveQty > 0 && hasCombinations && selectedCombo && (
              <div className="detail-stock-count">Stock: {effectiveQty} disponible(s)</div>
            )}
          </>
        )}

        { /* Sélecteur de déclinaisons */ }
        {hasCombinations && (
          <div className="detail-combinations">
            <p className="detail-combo-label">Déclinaison</p>
            <div className="detail-combo-options">
              {combinations.map((combo) => {
                // Note: `combo.quantity` vient du service shop (valeur indicative)
                const isOut = combo.quantity === 0;
                return (
                  <button
                    key={combo.id}
                    type="button"
                    className={[
                      'detail-combo-btn',

```

```

        selectedCombo?.id === combo.id ? 'detail-combo-btn--selected' : '',
        isOut ? 'detail-combo-btn--out' : '',
    ]
    .join(' ')
    .trim())
    onClick={() => setSelectedCombo(combo)}
  >
    {combo.label}
    {combo.priceImpact > 0 && (
      <span className="detail-combo-delta">
        {` +${formatPrice(combo.priceImpact * (1 + product.taxRate))}`}
      </span>
    )}
    {combo.priceImpact < 0 && (
      <span className="detail-combo-delta detail-combo-delta--neg">
        {` ${formatPrice(combo.priceImpact * (1 + product.taxRate))}`}
      </span>
    )}
  </button>
);
  })}
</div>
  {!selectedCombo && (
    <p className="detail-combo-hint">Veuillez sélectionner une déclinaison.</p>
  )}
</div>
)}

{/* Description courte */}
{product.description_short && (
  <div
    className="detail-desc-short"
    dangerouslySetInnerHTML={{ __html: product.description_short }}
  />
)}

{/* Zone d'achat */}
<div className="detail-purchase">
  {canAdd && !stockLoading && !stockError && (
    <div className="detail-qty-row">
      <label className="detail-qty-label">Quantité</label>
      <QtySelector value={qty} max={maxQty} onChange={setQty} />
    </div>
  )}

  <div className={`detail-stock-row detail-stock-row--${stockLevel}`}>
    <span
      className={`detail-stock-dot${stockLevel === 'low' ? ' detail-stock-dot--pulse' : ''}`}
    />
    <span className="detail-stock-text">
      {effectiveQty === 0
        ? "Ce produit n'est plus disponible actuellement"
        : hasCombinations && !selectedCombo
        ? "Sélectionnez une déclinaison pour voir la disponibilité"
        : stockMessage}
    </span>
  </div>

  <button
    className={`btn-add-cart${added ? ' btn-add-cart--added' : ''}`}
    disabled={!canAdd || stockLoading}
    onClick={handleAddToCart}
  >
    {added ? (
      '✓ Ajouté au panier !'
    ) : stockLoading ? (
      'Vérification...'
    ) : effectiveQty === 0 ? (
      'Rupture de stock'
    ) : hasCombinations && !selectedCombo ? (
      'Choisissez une déclinaison'
    )
  }
  </button>

```

```

    ) : (
      <>
        <svg
          width="18"
          height="18"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          strokeWidth="2"
        >
          <circle cx="9" cy="21" r="1" />
          <circle cx="20" cy="21" r="1" />
          <path d="M1 1h4l2.68 13.39a2 2 0 002 1.61h9.72a2 2 0 002-1.61L23 6H6" />
        </svg>
        Ajouter au panier
      </>
    )}
  </button>
</div>

  { /* Description longue */
  {product.description && (
    <div className="detail-desc">
      <h3 className="detail-desc-title">Description</h3>
      <div
        className="detail-desc-body"
        dangerouslySetInnerHTML={{ __html: product.description }}
      />
    </div>
  )}
  </div>
</div>
</div>
);
}

```

handleAddToCart() — ligne 186

Fonction : `handle add to cart` — voir le code ci-dessous.

```

const handleAddToCart = () => {
  if (!product || !canAdd) return;
  addItem(
    {
      id: product.id,
      attributeId: selectedCombo?.id,
      variantLabel: selectedCombo?.label,
      name: product.name,
      priceHt: effectivePriceHt,
      priceTtc: effectivePriceTtc,
      taxRate: product.taxRate,
      imageUrl: images[0],
    },
    qty,
  );
  setAdded(true);
  setTimeout(() => setAdded(false), 2000);
}

```

Fichier : `src/components/CartPage.tsx`

Lignes : 131 • Fonctions détectées : 2

CartPage() — ligne 7

Fonction : **cart page** — voir le code ci-dessous.

```
const CartPage: React.FC = () => {
  const { items, removeItem, updateQty, totalPrice, totalPriceHt, totalTax, totalItems } = useCart();
  const { customer } = useCustomer();
  const navigate = useNavigate();

  const handleCheckout = () => {
    if (!customer) {
      navigate('/shop/auth?next=/shop/checkout');
    } else {
      navigate('/shop/checkout');
    }
  };

  if (items.length === 0) {
    return (
      <div className="cart-empty">
        <svg width="64" height="64" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.2">
          <circle cx="9" cy="21" r="1"/><circle cx="20" cy="21" r="1"/>
          <path d="M1 1h4l2.68 13.39a2 2 0 02 1.61h9.72a2 2 0 02-1.61L23 6H6"/>
        </svg>
        <p>Votre panier est vide</p>
        <Link to="/shop" className="cart-btn-shop">Découvrir nos produits</Link>
      </div>
    );
  }

  return (
    <div className="cart-page">
      <nav className="cart-breadcrumb">
        <Link to="/shop">Boutique</Link>
        <span>/</span>
        <span>Panier</span>
      </nav>

      <h1 className="cart-title">Panier <span className="cart-count">{{totalItems}} article{{totalItems > 1 ? 's' : ''}}</span></h1>

      <div className="cart-layout">
        { /* ■■ Liste des articles ■■ */ }
        <div className="cart-items">
          {items.map(item => (
            <div key={` ${item.id}:: ${item.attributeId ?? ''} `} className="cart-item">
              <div className="cart-item-img">
                {item.imageUrl ? (
                  <img src={item.imageUrl} alt={item.name} onError={e => (e.currentTarget.style.display = 'none')} />
                ) : (
                  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.2">
                    <rect x="3" y="3" width="18" height="18" rx="2"/>
                    <circle cx="8.5" cy="8.5" r="1.5"/>
                    <polyline points="21 15 16 10 5 21"/>
                  </svg>
                )}
              </div>

              <div className="cart-item-info">
                <Link to={` /shop/${item.id} `} className="cart-item-name">{item.name}</Link>
                {item.variantLabel && (
                  <span className="cart-item-variant">{item.variantLabel}</span>
                )}
                <span className="cart-item-price">{formatPrice(item.priceTtc)}</span>
              </div>

              <div className="cart-item-qty">
                <button
                  className="cart-qty-btn"
                  onClick={() => item.qty > 1
                    ? updateQty(item.id, item.qty - 1, item.attributeId)
                    : removeItem(item.id, item.attributeId)}
                ></button>
                <span className="cart-qty-val">{item.qty}</span>
              </div>
            </div>
          )})}
        </div>
      </div>
    </div>
  );
}
```

```

        <button
          className="cart-qty-btn"
          onClick={() => updateQty(item.id, item.qty + 1, item.attributeId)}
        >+</button>
      </div>

      <div className="cart-item-subtotal">{formatPrice(item.priceTtc * item.qty)}</div>

      <button className="cart-item-remove" onClick={() => removeItem(item.id, item.attributeId)} title="Retirer"
        <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
          <line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/>
        </svg>
      </button>
    </div>
  )}
</div>

{/* ■■ Récapitulatif ■■ */}
<div className="cart-summary">
  <h2 className="cart-summary-title">Récapitulatif</h2>

  <div className="cart-summary-row">
    <span>Sous-total HT</span>
    <span>{formatPrice(totalPriceHt)}</span>
  </div>
  <div className="cart-summary-row">
    <span>TVA</span>
    <span>{formatPrice(totalTax)}</span>
  </div>
  <div className="cart-summary-row cart-summary-row--muted">
    <span>Livraison</span>
    <span>Calculé à l'étape suivante</span>
  </div>

  <div className="cart-summary-total">
    <span>Total TTC</span>
    <span>{formatPrice(totalPrice)}</span>
  </div>

  <button className="cart-checkout-btn" onClick={handleCheckout}>
    Commander
    <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
      <polyline points="9 18 15 12 9 6"/>
    </svg>
  </button>

  <Link to="/shop" className="cart-continue-link"><← Continuer mes achats</Link>
</div>
</div>
</div>
);
};

export default CartPage;

```

handleCheckout() — ligne 12

Fonction : `handle checkout` — voir le code ci-dessous.

```

const handleCheckout = () => {
  if (!customer) {
    navigate('/shop/auth?next=/shop/checkout');
  } else {
    navigate('/shop/checkout');
  }
}

```

src/components/CheckoutPage.tsx

Lignes : 429 • Fonctions détectées : 4

emptyForm() — ligne 33

Fonction : `empty form` — voir le code ci-dessous.

```
const emptyForm = (): AddressFormState => ({
  alias: 'Domicile', address1: '', address2: '', postcode: '', city: '', phone: '',
});
```

CheckoutPage() — ligne 39

Description (commentaire d'origine) :

[illegible]


```

        <strong className="carrier-name">{c.name}</strong>
        <span className="carrier-desc">{c.desc}</span>
      </div>
      <span className="carrier-price">
        {c.price === 0 ? 'Gratuit' : formatPrice(c.price)}
      </span>
    </label>
  )}}
</div>

<div className="checkout-nav">
  <button className="checkout-back-btn" onClick={() => setStep('address')}><- Adresse</button>
  <button className="checkout-next-btn" onClick={() => setStep('review')}>
    Continuer →
  </button>
</div>
)}}
```

STEP 3 — REVIEW & CONFIRM

```

{step === 'review' && (
  <div className="checkout-section">
    <h2 className="checkout-section-title">Récapitulatif</h2>

    {/* Articles */}
    <div className="review-block">
      <h3 className="review-block-title">Articles ({items.length})</h3>
      {items.map(item => (
        <div key={`_${item.id}::${item.attributeId ?? ''}`} className="review-item">
          <span className="review-item-name">
            {item.name}{item.variantLabel ? ` - ${item.variantLabel}` : ''}
          </span>
          <span className="review-item-qty">x {item.qty}</span>
          <span className="review-item-price">{formatPrice(item.priceTtc * item.qty)}</span>
        </div>
      ))}
    </div>

    {/* Adresse */}
    {selectedAddress && (
      <div className="review-block">
        <h3 className="review-block-title">
          Adresse de livraison
          <button className="review-edit-btn" onClick={() => setStep('address')}>Modifier</button>
        </h3>
        <p className="review-addr">
          {selectedAddress.firstname} {selectedAddress.lastname}<br />
          {selectedAddress.address1}{selectedAddress.address2 ? `, ${selectedAddress.address2}` : ''}
          {selectedAddress.postcode} {selectedAddress.city}
          {selectedAddress.phone && <><br />{selectedAddress.phone}</>}
        </p>
      </div>
    )}

    {/* Transporteur */}
    <div className="review-block">
      <h3 className="review-block-title">
        Livraison
        <button className="review-edit-btn" onClick={() => setStep('delivery')}>Modifier</button>
      </h3>
      <p className="review-addr">
        {selectedCarrier.name} – {selectedCarrier.price === 0 ? 'Gratuit' : formatPrice(selectedCarri
      </p>
    </div>

    {/* Paiement */}
    <div className="review-block review-block--payment">
      <div className="cod-badge">
        <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth=

```

```

        <rect x="1" y="4" width="22" height="16" rx="2"/>
        <line x1="1" y1="10" x2="23" y2="10"/>
      </svg>
      Paiement à la livraison (espèces)
    </div>
    <p className="cod-note">Vous payez le livreur en espèces à la réception de votre colis.</p>
  </div>

  {placeError && <div className="checkout-error">{placeError}</div>}

  <div className="checkout-nav">
    <button className="checkout-back-btn" onClick={() => setStep('delivery')}>← Livraison</button>
    <button className="checkout-confirm-btn" onClick={handlePlaceOrder} disabled={placing}>
      {placing ? (
        <<span className="btn-spinner" /> Commande en cours...</>
      ) : (
        `Confirmer la commande – ${formatPrice(total)}`
      )}
    </button>
  </div>
</div>
)}
</div>

{/* ■■ Order sidebar ■■ */}
<div className="checkout-sidebar">
  <h3 className="sidebar-title">Votre commande</h3>
  {items.map(item => (
    <div key={` ${item.id}:: ${item.attributeId ?? ''}`} className="sidebar-item">
      <span className="sidebar-item-name">
        {item.name}{item.variantLabel ? ` – ${item.variantLabel}` : ''}
      </span>
      <span className="sidebar-item-qty">×{item.qty}</span>
      <span className="sidebar-item-price">{formatPrice(item.priceTtc * item.qty)}</span>
    </div>
  ))}
  <div className="sidebar-sep" />
  <div className="sidebar-row">
    <span>Sous-total HT</span>
    <span>{formatPrice(totalPriceHt)}</span>
  </div>
  <div className="sidebar-row">
    <span>TVA</span>
    <span>{formatPrice(totalTaxAmount)}</span>
  </div>
  <div className="sidebar-row">
    <span>Livraison</span>
    <span>{shippingCost === 0 ? 'Gratuit' : formatPrice(shippingCost)}</span>
  </div>
  <div className="sidebar-total">
    <span>Total TTC</span>
    <span>{formatPrice(total)}</span>
  </div>
</div>
</div>
</div>
);
}

```

Description	(commentaire d'origine)	:	■■	Step	1:	Address


```

    setPlacing(false);
  }
}

```

Fichier : src/components/OrderConfirmation.tsx

Lignes : 115 • Fonctions détectées : 1

OrderConfirmation() — ligne 20

Fonction : `order confirmation` — voir le code ci-dessous.

```

const OrderConfirmation: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const { customer } = useCustomer();
  const [order, setOrder] = useState<OrderSummary | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (!id) return;
    api.get(`/orders/${id}?display=[id,reference,total_paid,current_state,date_add]`)
      .then(res => {
        const doc = new DOMParser().parseFromString(res.data, 'text/xml');
        const el = doc.querySelector('order');
        if (el) {
          setOrder({
            id,
            reference: el.querySelector('reference')?.textContent?.trim() || `#${id}`,
            totalPaid: parseFloat(el.querySelector('total_paid')?.textContent ?? '0'),
            currentState: parseInt(el.querySelector('current_state')?.textContent ?? '0', 10),
            dateAdd: el.querySelector('date_add')?.textContent?.trim() ?? '',
          });
        }
      })
      .catch(() => { /* show generic success even if fetch fails */ })
      .finally(() => setLoading(false));
  }, [id]);

  return (
    <div className="confirm-page">
      <div className="confirm-card">
        <div className="confirm-icon">
          <svg width="40" height="40" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" >
            <circle cx="12" cy="12" r="10"/>
            <polyline points="9 12 11 14 15 10"/>
          </svg>
        </div>

        <h1 className="confirm-title">Commande confirmée !</h1>
        <p className="confirm-sub">Merci {customer?.firstname ?? ''} pour votre commande.</p>

        {!loading && order && (
          <div className="confirm-details">
            <div className="confirm-row">
              <span>Référence</span>
              <strong>{order.reference}</strong>
            </div>
            <div className="confirm-row">
              <span>Total</span>
              <strong>{formatPrice(order.totalPaid)}</strong>
            </div>
            <div className="confirm-row">
              <span>Paieement</span>
              <strong>À la livraison</strong>
            </div>
          </div>
        )}
      </div>
    </div>
  );
}

```

```

    </div>
  </div>
)}

{loading && (
  <div className="confirm-details">
    <div className="confirm-row">
      <span>Numéro de commande</span>
      <strong>#{id}</strong>
    </div>
    <div className="confirm-row">
      <span>Paielement</span>
      <strong>À la livraison</strong>
    </div>
  </div>
)}

<div className="confirm-cod-box">
  <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
    <rect x="1" y="4" width="22" height="16" rx="2"/>
    <line x1="1" y1="10" x2="23" y2="10"/>
  </svg>
  <div>
    <strong>Paielement à la livraison</strong>
    <p>Préparez le règlement en espèces lors de la réception de votre colis.</p>
  </div>
</div>

<div className="confirm-actions">
  <Link to="/shop/my-orders" className="confirm-btn confirm-btn--outline">
    Voir mes commandes
  </Link>
  <Link to="/shop" className="confirm-btn confirm-btn--primary">
    Continuer mes achats
  </Link>
</div>
</div>
</div>
);
}

```

Fichier : src/components/MyOrders.tsx

Lignes : 126 • Fonctions détectées : 2

formatDate() — ligne 24

Fonction : **format date** — voir le code ci-dessous.

```

function formatDate(dateStr: string): string {
  if (!dateStr || dateStr === '0000-00-00 00:00:00') return '-';
  const d = new Date(dateStr.replace(' ', 'T'));
  return d.toLocaleDateString('fr-FR', { day: '2-digit', month: '2-digit', year: 'numeric' });
}

```

MyOrders() — ligne 30

Fonction : **my orders** — voir le code ci-dessous.

```

const MyOrders: React.FC = () => {
  const { customer } = useCustomer();
  const navigate = useNavigate();
  const [orders, setOrders] = useState<PSCustomerOrder[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
}

```

```

useEffect(() => {
  if (!customer) {
    navigate('/shop/auth?next=/shop/my-orders', { replace: true });
    return;
  }
  getCustomerOrders(customer.id)
    .then(setOrders)
    .catch(() => setError('Impossible de charger vos commandes.'))
    .finally(() => setLoading(false));
}, [customer, navigate]);

return (
  <div className="myorders-page">
    <nav className="myorders-breadcrumb">
      <Link to="/shop">Boutique</Link>
      <span>/</span>
      <span>Mes commandes</span>
    </nav>

    <div className="myorders-header">
      <h1 className="myorders-title">Mes commandes</h1>
      {customer && (
        <span className="myorders-customer">
          {customer.firstname} {customer.lastname}
        </span>
      )}
    </div>

    {loading && (
      <div className="myorders-loading">
        <div className="myorders-spinner" />
        <p>Chargement...</p>
      </div>
    )}

    {!loading && error && (
      <div className="myorders-error">{error}</div>
    )}

    {!loading && !error && orders.length === 0 && (
      <div className="myorders-empty">
        <svg width="56" height="56" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.2">
          <path d="M6 2L3 6v14a2 2 0 002 2h14a2 2 0 002-2V6l-3-4z"/>
          <line x1="3" y1="6" x2="21" y2="6"/>
          <path d="M16 10a4 4 0 01-8 0"/>
        </svg>
        <p>Vous n'avez pas encore de commandes.</p>
        <Link to="/shop" className="myorders-shop-btn">Découvrir nos produits</Link>
      </div>
    )}

    {!loading && !error && orders.length > 0 && (
      <div className="myorders-table-wrap">
        <table className="myorders-table">
          <thead>
            <tr>
              <th>Référence</th>
              <th>Date</th>
              <th>Statut</th>
              <th>Total</th>
              <th>Paieement</th>
            </tr>
          </thead>
          <tbody>
            {orders.map(order => (
              <tr key={order.id}>
                <td>
                  <span className="order-ref">{order.reference}</span>
                </td>
                <td>{formatDate(order.dateAdd)}</td>
                <td>

```

```

        <span className={`order-badge order-badge--${STATE_LEVEL[order.currentState] ?? 'pending'}
          {STATE_LABELS[order.currentState] ?? `État ${order.currentState}`}
        </span>
      </td>
      <td><strong>{formatPrice(order.totalPaid)}</strong></td>
      <td>À la livraison</td>
    </tr>
  )}
</tbody>
</table>
</div>
  )}
</div>
);
};

export default MyOrders;

```

Composants — Tableau de bord

Fichier :

`src/components/dashboardPage.tsx`

Lignes : 116 • Fonctions détectées : 3

IconRefresh() — ligne 11

Fonction : `icon refresh` — voir le code ci-dessous.

```
const IconRefresh = () => (  
  <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.2" strokeL  
    <polyline points="23 4 23 10 17 10"/>  
    <path d="M20.49 15a9 9 0 1 1-2.12-9.36L23 10"/>  
  </svg>  
);
```

IconCalendar() — ligne 18

Fonction : `icon calendar` — voir le code ci-dessous.

```
const IconCalendar = () => (  
  <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLin  
    <rect x="3" y="4" width="18" height="18" rx="2" ry="2"/>  
    <line x1="16" y1="2" x2="16" y2="6"/><line x1="8" y1="2" x2="8" y2="6"/>  
    <line x1="3" y1="10" x2="21" y2="10"/>  
  </svg>  
);
```

DashboardPage() — ligne 26

Fonction : `dashboard page` — voir le code ci-dessous.

```
export default function DashboardPage() {  
  const [filters, setFilters] = useState<DashboardFilters>({ selectedDate: null });  
  const { data, loading, error, refetch } = useDashboardData(filters, 60000);  
  
  if (loading) return <DashboardSkeleton />;  
  
  if (error) {  
    return (  
      <div className="db-page">  
        <div className="db-error">  
          <span>Erreur de chargement : {error}</span>  
        </div>  
      </div>  
    );  
  }  
  
  if (!data) return null;  
  
  const globalTotalOrders = data.dailyStats.reduce((s, d) => s + d.orderCount, 0);  
  const globalTotalRevenue = data.dailyStats.reduce((s, d) => s + d.totalAmount, 0);  
  const globalAvg = globalTotalOrders === 0 ? 0 : globalTotalRevenue / globalTotalOrders;  
  
  const today = new Date().toLocaleDateString('fr-FR', { weekday: 'long', day: 'numeric', month: 'long', year  
  
  return (  

```

```

<div className="db-page">

  {/* ■■ En-tête ■■ */}
  <div className="db-header">
    <div className="db-header-left">
      <h1 className="db-title">Tableau de bord</h1>
      <p className="db-subtitle">{today}</p>
    </div>
    <button className="db-refresh-btn" onClick={refetch}>
      <IconRefresh />
      Actualiser
    </button>
  </div>

  {/* ■■ Filtre date ■■ */}
  <div className="db-filter-bar">
    <span className="db-filter-icon"><IconCalendar /></span>
    <span className="db-filter-label">Filtrer par date :</span>
    <input
      type="date"
      className="db-filter-input"
      value={filters.selectedDate ?? ''}
      onChange={(e) => setFilters({ selectedDate: e.target.value || null })}
    />
    {filters.selectedDate && (
      <button className="db-filter-clear" onClick={() => setFilters({ selectedDate: null })}>
        X Effacer
      </button>
    )}
    {filters.selectedDate && (
      <span className="db-filter-active-badge">
        {filters.selectedDate}
      </span>
    )}
  </div>

  {/* ■■ KPI globaux ■■ */}
  <div>
    <p className="db-section-label">Vue globale – toutes commandes</p>
    <StatsCards
      totalOrders={globalTotalOrders}
      totalRevenue={globalTotalRevenue}
      averageOrderValue={globalAvg}
    />
  </div>

  {/* ■■ KPI filtrés ■■ */}
  {filters.selectedDate && (
    <div>
      <p className="db-section-label">Résultats du {filters.selectedDate}</p>
      <StatsCards
        totalOrders={data.stats.totalOrders}
        totalRevenue={data.stats.totalRevenue}
        averageOrderValue={data.stats.averageOrderValue}
      />
    </div>
  )}

  {/* ■■ Tableau par jour ■■ */}
  <OrdersTable dailyStats={data.dailyStats} />

</div>
);
}

```

Fichier : src/components/dashboard/DashboardSkeleton.tsx

Lignes : 14 • Fonctions détectées : 1

DashboardSkeleton() — ligne 1

Fonction : **dashboard skeleton** — voir le code ci-dessous.

```
export function DashboardSkeleton() {
  return (
    <div className="db-skeleton">
      <div className="db-skel-block db-skel-header" />
      <div className="db-skel-grid">
        {[0, 1, 2].map((i) => (
          <div key={i} className="db-skel-block db-skel-card" />
        ))}
      </div>
      <div className="db-skel-block db-skel-table" />
    </div>
  );
}
```

Fichier : src/components/dashboard/DatePickerInput.tsx

Lignes : 29 • Fonctions détectées : 1

DatePickerInput() — ligne 6

Fonction : **date picker input** — voir le code ci-dessous.

```
export function DatePickerInput({ selectedDate, onChange }: DatePickerInputProps) {
  return (
    <div className="mb-6">
      <label className="block text-sm font-medium text-gray-700 mb-2">
        Filtrer par date précise :
      </label>
      <input
        type="date"
        value={selectedDate || ''}
        onChange={(e) => onChange(e.target.value || null)}
        className="w-full md:w-64 px-3 py-2 border border-gray-300 rounded-lg shadow-sm focus:ring-blue-500 f
      />
      {selectedDate && (
        <button
          onClick={() => onChange(null)}
          className="ml-3 text-sm text-red-600 hover:text-red-800"
        >
          ✕ Effacer
        </button>
      )}
    </div>
  );
}
```

Fichier : src/components/dashboard/OrdersTable.tsx

Lignes : 53 • Fonctions détectées : 2

formatDate() — ligne 7

Fonction : `format date` — voir le code ci-dessous.

```
function formatDate(raw: string): string {
  const d = new Date(raw + 'T00:00:00');
  if (isNaN(d.getTime())) return raw;
  return d.toLocaleDateString('fr-FR', { weekday: 'short', day: 'numeric', month: 'short', year: 'numeric' });
}
```

OrdersTable() — ligne 13

Fonction : `orders table` — voir le code ci-dessous.

```
export function OrdersTable({ dailyStats }: OrdersTableProps) {
  const sorted = [...dailyStats].sort((a, b) => b.date.localeCompare(a.date));

  return (
    <div className="db-table-card">
      <div className="db-table-head">
        <h2 className="db-table-title">Détail par jour</h2>
        <span className="db-table-pill">{sorted.length} jour{sorted.length !== 1 ? 's' : ''}</span>
      </div>

      {sorted.length === 0 ? (
        <div className="db-table-empty">Aucune donnée disponible</div>
      ) : (
        <table className="db-table">
          <thead>
            <tr>
              <th>Date</th>
              <th>Commandes</th>
              <th>Montant total</th>
            </tr>
          </thead>
          <tbody>
            {sorted.map((day) => (
              <tr key={day.date}>
                <td>
                  <span className="db-date-badge">{formatDate(day.date)}</span>
                </td>
                <td className="db-td-orders">
                  <span className="db-order-badge">{day.orderCount}</span>
                </td>
                <td className="db-td-amount">{formatCurrency(day.totalAmount)}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
    </div>
  );
}
```

Fichier : `src/components/dashboard/SalesChart.tsx`

Lignes : 35 • **Fonctions détectées :** 1

SalesChart() — ligne 10

Fonction : `sales chart` — voir le code ci-dessous.

```

export function SalesChart({ data }: SalesChartProps) {
  const formattedData = data.map(day => ({
    date: day.date,
    'Montant total (€)': day.totalAmount,
    'Nb commandes': day.orderCount,
  }));

  return (
    <div className="bg-white rounded-2xl shadow p-4 border border-gray-100 mb-8">
      <h2 className="text-lg font-semibold text-gray-700 mb-4">Ventes par jour</h2>
      <ResponsiveContainer width="100%" height={300}>
        <LineChart data={formattedData}>
          <CartesianGrid strokeDasharray="3 3" />
          <XAxis dataKey="date" />
          <YAxis yAxisId="left" tickFormatter={(value) => formatCurrency(value)} />
          <YAxis yAxisId="right" orientation="right" />
          <Tooltip formatter={(value, name) => name === 'Montant total (€)' ? formatCurrency(value as number) : value} />
          <Legend />
          <Line yAxisId="left" type="monotone" dataKey="Montant total (€)" stroke="#10b981" strokeWidth={2} />
          <Line yAxisId="right" type="monotone" dataKey="Nb commandes" stroke="#3b82f6" strokeWidth={2} />
        </LineChart>
      </ResponsiveContainer>
    </div>
  );
}

```

Fichier : src/components/dashboard/StatsCards.tsx

Lignes : 68 • Fonctions détectées : 4

IconOrders() — ligne 8

Fonction : **icon orders** — voir le code ci-dessous.

```

const IconOrders = () => (
  <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" />
    <path d="M6 2L3 6v14a2 2 0 02 2h14a2 2 0 02-2v6l-3-4z"/>
    <line x1="3" y1="6" x2="21" y2="6"/>
    <path d="M16 10a4 4 0 01-8 0"/>
  </svg>
);

```

IconRevenue() — ligne 16

Fonction : **icon revenue** — voir le code ci-dessous.

```

const IconRevenue = () => (
  <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" />
    <line x1="12" y1="1" x2="12" y2="23"/>
    <path d="M17 5H9.5a3.5 3.5 0 00 7h5a3.5 3.5 0 010 7H6"/>
  </svg>
);

```

IconAvg() — ligne 23

Fonction : **icon avg** — voir le code ci-dessous.

```

const IconAvg = () => (
  <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" />
    <line x1="18" y1="20" x2="18" y2="10"/>
  </svg>
);

```

```

    <line x1="12" y1="20" x2="12" y2="4"/>
    <line x1="6" y1="20" x2="6" y2="14"/>
    <line x1="2" y1="20" x2="22" y2="20"/>
  </svg>
);

```

StatsCards() — ligne 32

Fonction : `stats cards` — voir le code ci-dessous.

```

export function StatsCards({ totalOrders, totalRevenue, averageOrderValue }: StatsCardsProps) {
  return (
    <div className="db-kpi-grid">
      <div className="db-kpi-card">
        <div className="db-kpi-icon db-kpi-icon--indigo">
          <IconOrders />
        </div>
        <div className="db-kpi-body">
          <div className="db-kpi-label">Commandes</div>
          <div className="db-kpi-value">{totalOrders}</div>
        </div>
      </div>

      <div className="db-kpi-card">
        <div className="db-kpi-icon db-kpi-icon--green">
          <IconRevenue />
        </div>
        <div className="db-kpi-body">
          <div className="db-kpi-label">Chiffre d'affaires</div>
          <div className="db-kpi-value db-kpi-value--green">{formatCurrency(totalRevenue)}</div>
        </div>
      </div>

      <div className="db-kpi-card">
        <div className="db-kpi-icon db-kpi-icon--pink">
          <IconAvg />
        </div>
        <div className="db-kpi-body">
          <div className="db-kpi-label">Panier moyen</div>
          <div className="db-kpi-value db-kpi-value--pink">{formatCurrency(averageOrderValue)}</div>
        </div>
      </div>
    </div>
  );
}

```

Total : 349 fonctions extraites du code source.